L’IA AU SERVICE DES DEVELOPPEURS

Jean-Christophe RANGON

2026



email: jc.rangon.formateur@gmail.com

linkedIn: www.linkedin.com/in/jean-christophe-rangon-dev-web

SUPPORT FORMATEUR ENRICHI

J1

Comprendre, cadrer et utiliser l'IA correctement en C++ industriel

IA, robotique, prompts, génération contrôlée, tests et double revue

FIL ROUGE

Robot porte-palette : toutes les notions de la journée sont reliées à un même contexte industriel afin de raisonner sur la criticité, la qualité du prompt, la vérification du code et la décision humaine.

DURÉE

7 h 30 de formation - 09:00-12:30 et 13:30-17:30.

Document de préparation et d'animation - version dense et détaillée

0. Comment utiliser ce support

Ce document est un support formateur enrichi. Il n'est pas conçu comme un diaporama : il contient volontairement davantage d'explications, d'exemples, de contre-exemples, d'éléments de langage, de consignes d'atelier et de corrections que ce qui sera projeté. Le but est de pouvoir préparer puis animer le J1 sans devoir reconstruire les explications au dernier moment.

PRINCIPE DE LA JOURNÉE

L'IA propose. Le développeur spécifie, vérifie, teste et décide. Plus une erreur peut avoir un impact industriel important, plus la validation humaine et les contrôles doivent être renforcés.

0.1 Progression pédagogique du J1

Horaire	Séquence	Question centrale	Production
09:00-10:00	IA, robotique et responsabilité	Que peut-on déléguer à l'IA, et que ne prouve jamais une réponse plausible ?	Règle de base : proposition != validation
10:00-11:15	Cartographie des zones du robot	Toutes les parties du système ont-elles le même niveau de risque ?	Cartographie vert / orange / rouge
11:15-12:30	Règles internes d'utilisation	Comment transformer la prudence en règles opérationnelles ?	Ébauche de charte IA
13:30-15:00	Prompt technique	Comment réduire l'ambiguïté avant de générer ?	Gabarit de prompt réutilisable
15:00-16:30	Code C++ et tests	Comment produire une petite unité de code sans perdre le contrôle ?	Code + tests + liste de contrôles
16:30-17:30	Revue IA + revue humaine	Comment utiliser une IA pour relire sans lui déléguer la décision ?	Checklist de double revue

0.2 Ce que l'on doit éviter pendant ce J1

- Faire un cours général sur l'intelligence artificielle sans lien avec la production de code C++.
- Présenter une réponse de Copilot ou d'un autre assistant comme correcte simplement parce qu'elle compile ou paraît élégante.
- Transformer la classification vert / orange / rouge en norme de sûreté industrielle : il s'agit ici d'un outil pédagogique de raisonnement, pas d'une certification.
- Introduire des architectures ou outils complexes non nécessaires à la journée.
- Choisir un framework de tests externe alors que le déroulé ne l'impose pas : l'exemple principal utilise donc des tests C++ autonomes avec assert.
- Confondre revue assistée et validation : la seconde reste une décision humaine appuyée sur des critères vérifiables.

0.3 Prérequis utiles pour suivre les démonstrations

Le déroulé validé vise des développeurs ou ingénieurs ayant déjà les bases de C++ : fonctions, classes, types, compilation et lecture de code. Les exemples du J1 restent volontairement simples : l'objectif n'est pas d'enseigner C++, mais d'enseigner une méthode de travail contrôlée avec l'IA.

09:00-10:00 - Introduction : IA, robotique et responsabilité

OBJECTIF DE LA SÉQUENCE

Poser le contexte industriel et distinguer clairement assistance, génération et validation.

La première heure sert à installer le cadre mental de toute la formation. Avant de parler de prompt ou de code, le groupe doit comprendre qu'un assistant IA est utile précisément parce qu'il accélère certaines tâches - et qu'il devient dangereux si l'on confond vitesse de production et preuve de correction.

1.1 Partir du système réel : le robot porte-palette

Le fil rouge est un robot porte-palette. On peut le représenter simplement comme un système capable de recevoir une mission, de connaître ou estimer son état, de décider d'un déplacement, d'interagir avec des capteurs et des actionneurs, de communiquer avec son environnement et de remonter des informations de supervision ou de diagnostic.

Pour la journée, nous n'avons pas besoin de modéliser toute la robotique. Le robot sert surtout à rendre visibles les conséquences d'une erreur. Une faute dans un commentaire ou dans un outil de diagnostic n'a pas forcément le même impact qu'une faute dans une décision qui autorise un mouvement alors qu'un frein est indisponible.

À DIRE TRÈS SIMPLEMENT

Dans un projet industriel, toutes les lignes de code ne se valent pas. Avant de demander quelque chose à une IA, nous devons savoir ce que cette réponse pourrait casser si elle est fausse.

Schéma conceptuel : de la mission au mouvement

Étape	Exemple sur le robot	Question de contrôle
1. Mission	Transporter une palette de A vers B	La mission est-elle valide ?
2. État	Mode Idle, batterie, capteurs, freins, erreurs	Les données nécessaires sont-elles disponibles et cohérentes ?
3. Décision	Autoriser ou refuser le mouvement	Toutes les contraintes sont-elles respectées ?
4. Action	Commander le déplacement	La commande est-elle admissible dans l'état courant ?
5. Observation	Logs, diagnostic, supervision	Peut-on expliquer ce qui s'est produit ?

1.2 Que signifie “utiliser une IA” dans le développement ?

Dans ce cours, le mot IA désigne un assistant capable de produire ou transformer du texte et du code à partir d’instructions. Il peut proposer une fonction, reformuler une documentation, suggérer des tests, commenter une interface, expliquer un message d’erreur ou aider à relire une portion de code.

La propriété importante à faire comprendre n’est pas la manière interne dont le modèle fonctionne, mais le type de confiance que l’on peut raisonnablement lui accorder : une sortie peut être fluide, cohérente en apparence et pourtant être partiellement fausse, incomplète ou inadaptée au contexte du projet.

Assistance, génération, validation : trois verbes à ne pas confondre

Niveau	Ce que fait l’outil	Exemple	Ce que cela ne prouve pas
Assistance	Aide à comprendre, reformuler, chercher des pistes ou comparer des options.	Expliquer une fonction existante ou proposer des cas de test.	Que l’explication couvre toutes les contraintes réelles.
Génération	Produit une nouvelle sortie : code, test, documentation, configuration.	Générer une classe de décision pour autoriser un déplacement.	Que le code est correct, sûr, complet ou conforme au projet.
Validation	Établit qu’un résultat satisfait des critères définis.	Revue humaine + tests + règles projet + contrôles nécessaires.	La validation ne peut pas être remplacée par une simple affirmation de l’IA.

RÈGLE DE MÉMORISATION

Un assistant IA peut participer au travail de validation en suggérant des contrôles. Il ne devient pas pour autant l’autorité qui déclare le code conforme.

1.3 Pourquoi l’IA peut réellement aider

Le gain vient surtout de la réduction du coût des tâches de préparation, d’exploration et de première rédaction. L’IA peut produire rapidement une première structure que le développeur va ensuite contraindre et vérifier.

- Accélérer une première version d’une petite fonction ou classe.
- Proposer des tests auxquels le développeur n’a pas encore pensé.
- Reformuler une interface ou une documentation pour rendre un contrat plus explicite.
- Comparer deux variantes de code et faire ressortir des différences.
- Aider à construire une checklist de revue.
- Expliquer un morceau de code existant pour préparer une revue humaine.

Le bénéfice principal n’est donc pas de supprimer la réflexion. Il est de déplacer une partie du temps passé à écrire mécaniquement vers du temps passé à spécifier, comparer, tester et décider.

1.4 Pourquoi la même IA peut devenir dangereuse

Le danger apparaît quand la fluidité de la réponse est interprétée comme un indice de vérité. Un code peut compiler et rester incorrect. Un test peut passer parce qu’il teste la même hypothèse erronée que le code. Une suggestion peut être localement élégante mais incompatible avec une convention d’architecture, une contrainte de temps réel ou une règle de sûreté non mentionnée dans le prompt.

Risque	Exemple	Pourquoi c'est trompeur	Réponse pédagogique
Omission	La vérification du capteur frontal manque.	Le reste du code semble propre et masque l'absence.	Comparer systématiquement la sortie à une liste de contraintes.
Hypothèse inventée	L'IA suppose qu'une batterie à 10 % est suffisante.	Le seuil paraît raisonnable mais n'a jamais été fourni.	Refuser toute valeur métier inventée ; demander l'origine de chaque seuil.
API fictive	Une fonction ou bibliothèque inexistante est utilisée.	Le nom semble plausible.	Compiler, consulter la documentation du projet, vérifier les symboles.
Mauvaise généralisation	Une règle valable en mode Idle est appliquée dans d'autres modes.	Le code "marche" dans le cas démontré.	Tester les états non admissibles et les cas limites.
Fausse confiance	L'outil affirme que le code est sûr.	La formulation est catégorique.	Exiger des preuves vérifiables : tests, invariants, critères, revue.

1.5 Démonstration guidée : proposition != preuve

Demander à un assistant : "Écris une fonction C++ qui dit si le robot peut avancer." La réponse sera souvent rapide. L'objectif n'est pas de juger la syntaxe. Le formateur demande ensuite au groupe : "De quelles informations avons-nous besoin pour décider si cette fonction est correcte ?"

- Quel est le seuil de batterie ?
- Que signifie exactement "freins disponibles" ?
- Que se passe-t-il si le capteur frontal ne répond pas ?
- Quels modes du robot autorisent le mouvement ?
- Existe-t-il une erreur critique active ?
- Quelle valeur faut-il renvoyer en cas d'information inconnue ?
- Quels tests sont attendus ?

La démonstration fait apparaître un point fondamental : le problème principal n'est pas seulement de produire du code, mais de produire une spécification suffisamment claire pour pouvoir juger le code.

1.6 Questions au groupe et réponses attendues

Question	Réponse courte attendue	Relance si le groupe hésite
Une IA peut-elle écrire du code qui compile ?	Oui.	Compiler prouve-t-il que le comportement métier est correct ? Non.
Une IA peut-elle générer des tests ?	Oui.	Ses tests couvrent-ils nécessairement les cas oubliés ? Non.
Qui valide l'intégration ?	L'équipe / le responsable humain selon le processus.	La décision doit être fondée sur des critères vérifiables.
Qu'est-ce qui doit venir avant la génération ?	Le besoin et les contraintes.	Sans critères, on ne sait pas juger la sortie.

1.7 Mini-synthèse 09:00-10:00

À RETENIR

L'IA est un accélérateur de proposition. Dans un contexte industriel, elle doit être entourée d'une spécification explicite, de contrôles et d'une décision humaine.

10:00-11:15 - Cartographie des zones du robot

OBJECTIF DE LA SÉQUENCE

Savoir raisonner par zones de criticité avant de décider comment utiliser l'IA.

La deuxième séquence introduit une idée simple : on n'applique pas le même niveau de confiance ni le même niveau de contrôle à toutes les parties d'un système. Le classement vert / orange / rouge est un outil de discussion pour ajuster le degré de prudence.

2.1 Les sept couches du fil rouge

Les couches ci-dessous reprennent le découpage du déroulé. Elles ne prétendent pas décrire une architecture universelle de robot mobile. Elles servent à organiser la discussion et à distinguer des responsabilités.

Couche	Rôle simplifié	Exemples de responsabilités	Questions de risque
Hardware	Accès aux éléments physiques.	Lecture capteurs, commande actionneurs, état batterie.	Une erreur peut-elle produire une commande physique incorrecte ?
Safety	Règles qui empêchent ou arrêtent un comportement dangereux.	Arrêt, conditions d'autorisation, interverrouillages.	Une erreur peut-elle autoriser une action interdite ?
Navigation	Déplacement et localisation.	Trajectoire, position, évitement.	Une décision erronée peut-elle provoquer une collision ou un blocage ?
Mission	Décide quoi faire et dans quel ordre.	Prendre palette, livrer, attendre, reprendre.	Une mission incohérente peut-elle demander un mouvement inadmissible ?
Supervision	Vue globale et pilotage opérationnel.	État machine, alarmes, commandes opérateur.	Une information erronée peut-elle induire une mauvaise décision humaine ?
Diagnostic	Aide à comprendre défauts et événements.	Codes erreurs, journaux, analyse de panne.	L'erreur affecte-t-elle l'action ou surtout l'analyse ?
Communication	Échanges avec autres modules ou systèmes.	Messages, protocoles, synchronisation.	Une donnée mal interprétée peut-elle contaminer plusieurs modules ?

2.2 Lire la pile de couches

Niveau	Couche	Idée à pointer
7	Supervision	Ce que voit ou commande l'opérateur / le système global.
6	Mission	Pourquoi le robot agit : objectif et séquence métier.
5	Navigation	Comment le robot se déplace.
4	Safety	Ce qui doit empêcher un mouvement inadmissible.
3	Communication	Comment les informations circulent entre parties.
2	Diagnostic	Comment on explique l'état et les défauts.
1	Hardware	L'interface avec le monde physique.

Au tableau, on peut dessiner les couches sous forme de boîtes empilées. Le but n'est pas de faire croire que toutes les architectures suivent exactement cet ordre, mais d'obliger le groupe à nommer la responsabilité de chaque zone avant de parler d'IA.

2.3 Le classement vert / orange / rouge

Le code source ne définit pas lui-même sa couleur. Le classement dépend du contexte, de l'impact possible d'une erreur, de la proximité avec une action physique, de la capacité de détection et de la possibilité de revenir en arrière. Une même fonction peut donc changer de niveau selon son usage.

Couleur pédagogique	Usage de l'IA possible	Niveau de contrôle attendu	Exemple indicatif
VERT	Assistance large sur tâches à faible conséquence directe.	Relecture normale, tests adaptés, vérification habituelle.	Reformulation d'un message de diagnostic ou d'une documentation interne.
ORANGE	Génération possible mais avec contraintes explicites et revue renforcée.	Tests ciblés, comparaison au contrat, validation par un développeur compétent.	Transformation d'une structure de données de supervision.
ROUGE	Usage très encadré ; l'IA ne doit jamais être considérée comme autorité de validation.	Exigences explicites, tests négatifs et limites, revue humaine rigoureuse, processus projet.	Décision qui autorise un mouvement alors qu'une condition de sécurité peut être absente.

POINT DE VIGILANCE

Vert / orange / rouge est ici un outil pédagogique de priorisation du contrôle. Ce classement ne remplace ni une analyse de risques formelle, ni les exigences de sûreté, ni les normes applicables au produit réel.

2.4 Une règle de décision simple pour discuter la criticité

Pour chaque fonction, demander successivement :

1. Que se passe-t-il si cette fonction se trompe ?
2. L'erreur peut-elle déclencher ou autoriser une action physique ?
3. L'erreur sera-t-elle détectée automatiquement avant d'avoir un effet ?
4. Peut-on revenir en arrière facilement ?
5. La fonction dépend-elle d'informations qui peuvent être absentes ou incohérentes ?
6. Une personne valide-t-elle la décision avant exécution ?

Plus les réponses augmentent l'impact potentiel ou diminuent la possibilité de détection/récupération, plus le niveau de prudence doit monter.

2.5 Atelier : classer des fonctions

CONSIGNE

En binômes, classez chaque fonction en vert, orange ou rouge. Le classement seul ne suffit pas : écrivez une justification en une phrase en commençant par "si cette fonction se trompe, alors…".

Fonction à classer	Couleur proposée	Justification attendue
Générer un résumé lisible des logs de diagnostic	VERT à ORANGE selon usage	Faible impact si le résumé n'est qu'une aide ; plus critique s'il déclenche une décision sans retour aux logs bruts.
Calculer l'autorisation de mouvement selon l'état des freins	ROUGE	Une erreur peut autoriser un mouvement quand une condition essentielle n'est pas satisfaite.
Proposer le nom d'une variable ou d'une fonction	VERT	Erreur facile à détecter et corriger ; pas d'effet physique direct.
Convertir un message de mission en commande interne	ORANGE à ROUGE	Une mauvaise interprétation peut propager une décision erronée vers des couches d'action.
Construire un message d'état pour la supervision	ORANGE	Une information erronée peut influencer une décision opérateur.
Suggérer des tests pour une règle de safety	ROUGE pour la règle, IA utile pour l'assistance	L'IA peut suggérer des tests, mais la couverture et l'acceptation restent humaines.

Correction : ne pas chercher une "bonne couleur" sans contexte

La correction doit surtout évaluer la qualité du raisonnement. Une équipe peut défendre orange là où une autre propose rouge si elle explicite les hypothèses : existence d'un contrôle indépendant, caractère purement informatif du résultat, impossibilité d'exécution automatique, etc.

2.6 Cartographie de référence pour la suite du J1

Zone	Exemple IA acceptable	Contrôle minimal à exiger
Faible conséquence directe	Documentation, explication, noms, synthèse de données non décisionnelle.	Lecture humaine et cohérence générale.

Zone	Exemple IA acceptable	Contrôle minimal à exiger
Conséquence métier modérée	Transformation de données, logique de mission non critique, interfaces.	Contrat clair, tests positifs/négatifs, revue ciblée.
Conséquence potentiellement critique	Autorisation de mouvement, conditions liées à capteurs/freins/erreurs.	Spécification explicite, tests limites, revue humaine indépendante, règles projet.

2.7 Mini-synthèse 10:00-11:15

À RETENIR

Avant de choisir comment utiliser l'IA, on qualifie l'impact d'une erreur. La criticité guide le niveau de détail du prompt, la profondeur des tests et la force de la revue humaine.

11:15-12:30 - Règles internes d'utilisation de l'IA

OBJECTIF DE LA SÉQUENCE

Construire des règles d'usage explicites pour éviter les usages non maîtrisés.

La prudence n'est utile que si elle devient opérationnelle. Une charte efficace ne dit pas seulement "faites attention". Elle précise ce qui est autorisé, interdit ou soumis à validation, et laisse une trace vérifiable de la décision.

3.1 Confidentialité : la première question avant le prompt

Avant d'envoyer un extrait de code, un log ou une spécification à un outil externe, le développeur doit savoir quelles informations peuvent être partagées. Le support ne fixe pas la politique de l'entreprise : il apprend à poser la question et à appliquer la règle locale.

- Code source propriétaire ou non publié.
- Secrets : clés, tokens, mots de passe, chaînes de connexion.
- Informations client, données personnelles ou informations contractuelles.
- Détails d'architecture considérés comme sensibles.
- Logs contenant des identifiants, adresses ou données opérationnelles.
- Extraits issus d'un composant tiers soumis à des conditions de licence.

FORMULATION OPÉRATIONNELLE

Je ne colle dans un assistant que les informations autorisées par la politique du projet. Je retire systématiquement les secrets et je préfère un exemple minimal ou anonymisé quand le contexte complet n'est pas nécessaire.

3.2 Code critique : augmenter les contrôles, pas seulement la prudence verbale

Dire qu'un code est critique n'a de valeur que si cela change le processus. Par exemple : prompt plus détaillé, interdiction de seuil inventé, tests négatifs obligatoires, revue humaine distincte, comparaison avec la spécification et traçabilité de la décision.

Formulation vague	Pourquoi elle est faible	Formulation opérationnelle
“Faire attention au code safety.”	Impossible à vérifier.	“Toute modification d’une règle d’autorisation de mouvement doit être comparée à la liste des contraintes et relue par un second développeur.”
“Ne pas faire confiance à l’IA.”	N’indique pas quoi faire.	“Chaque condition métier du prompt doit avoir au moins un test qui montre le comportement attendu lorsqu’elle est satisfaite et lorsqu’elle ne l’est pas.”
“Vérifier le code.”	Trop général.	“Compiler, exécuter les tests, vérifier les seuils, les états autorisés, les erreurs et les cas limites.”

3.3 Prompts standards : pourquoi standardiser ?

Un gabarit de prompt réduit le risque d’oublier systématiquement la même information. Il ne garantit pas une bonne réponse, mais il améliore la qualité des données d’entrée et facilite la revue : deux développeurs peuvent vérifier si contexte, entrées, sorties, erreurs, contraintes, tests et critères d’acceptation sont présents.

3.4 Limites de Copilot et assistants similaires

Dans ce J1, on retient des limites indépendantes d’une version précise du produit : l’assistant ne connaît pas automatiquement toutes les règles tacites du projet, peut produire une réponse plausible mais fausse, peut omettre une contrainte, peut reproduire une hypothèse incorrecte dans ses tests et peut proposer une solution techniquement valide mais architecturalement inadaptée.

Limite	Symptôme visible	Réflexe à enseigner
Contexte incomplet	La réponse choisit des conventions non prévues.	Fournir les règles utiles et demander à l’outil de lister ses hypothèses.
Confiance excessive	Réponse formulée sans nuance.	Demander les points d’incertitude et vérifier indépendamment.
Omission silencieuse	Une contrainte n’apparaît plus dans le code.	Tracer chaque exigence vers un test ou un contrôle.
Tests biaisés	Les tests reproduisent exactement l’implémentation proposée.	Déduire les tests du besoin avant ou indépendamment du code.
Refactoring trop large	Beaucoup de fichiers changent pour un petit besoin.	Limiter le périmètre, demander des changements atomiques et justifiés.

3.5 Validation humaine : ce que “humain” doit vraiment signifier

Une validation humaine ne consiste pas à cliquer sur “accept”. Elle suppose qu’une personne compétente peut expliquer pourquoi la solution satisfait les critères. Pour une petite unité de code, elle doit pouvoir retrouver le besoin, les contraintes, les hypothèses, les tests, les écarts éventuels et la décision finale.

3.6 Traçabilité : pouvoir reconstruire la décision

La traçabilité répond à la question : “Pourquoi ce code a-t-il été accepté ?” Il n’est pas nécessaire d’archiver une conversation gigantesque si le projet ne le demande pas. En revanche, il faut conserver les éléments utiles : ticket, contraintes, tests, décision de revue, éventuelles modifications apportées à la sortie IA.

Élément	Trace utile
Besoin	Ticket ou description du comportement attendu.
Contraintes	Liste explicite : seuil batterie, freins, capteur, erreur critique, mode.
Génération	Prompt ou résumé des instructions déterminantes.
Vérification	Tests, résultat de compilation, points de revue.
Décision	Accepté / modifié / refusé, avec justification quand nécessaire.

3.7 Atelier : rédiger une mini-charte de 6 règles

CONSIGNE

Rédigez six règles courtes : une sur la confidentialité, une sur le code critique, une sur les prompts, une sur la validation humaine, une sur la traçabilité, une sur l’usage des suggestions IA. Chaque règle doit être contrôlable.

Proposition de correction

- Ne jamais transmettre de secret, token, mot de passe ou donnée non autorisée dans un prompt.
- Pour une fonction rouge, la sortie IA est toujours revue à partir de la spécification et non seulement à partir du code généré.
- Utiliser le gabarit de prompt du projet pour toute génération de code : contexte, criticité, entrées, sorties, erreurs, contraintes, tests, critères d’acceptation.
- Toute valeur métier ou seuil non fourni dans le besoin doit être signalé comme hypothèse et ne doit pas être inventé silencieusement.
- Conserver dans le ticket les critères qui ont permis d’accepter ou de refuser la proposition.
- Une suggestion de revue IA n’est appliquée que si le développeur peut expliquer son effet et vérifier qu’elle respecte le besoin.

3.8 Checklist de qualité d’une règle interne

Question	Oui / Non
La règle indique-t-elle une action observable ?	
Le développeur sait-il quand elle s’applique ?	
Peut-on vérifier qu’elle a été respectée ?	
Évite-t-elle les mots vagues comme “faire attention” sans préciser comment ?	

Question	Oui / Non
Est-elle compatible avec la politique réelle de l'entreprise ?	
Indique-t-elle quand une validation supplémentaire est nécessaire ?	

3.9 Mini-synthèse 11:15-12:30

À RETENIR

Une bonne règle interne transforme un risque en comportement vérifiable : ce que l'on peut partager, ce que l'on peut générer, ce que l'on doit tester, qui décide et quelle trace conserver.

13:30-15:00 - Écrire un bon prompt technique

OBJECTIF DE LA SÉQUENCE

Passer d'une demande vague à une spécification exploitable par une IA.

L'après-midi commence par la compétence centrale du J1 : formuler une demande qui permet ensuite de juger la réponse. Un prompt technique n'est pas une formule magique. C'est une mini-spécification qui réduit l'ambiguïté et rend les critères de contrôle visibles.

4.1 Le problème d'une demande vague

DEMANDE VAGUE	PROBLÈMES CRÉÉS
"Fais une classe C++ qui vérifie si le robot peut avancer."	Seuil de batterie inconnu ; états autorisés inconnus ; comportement si capteur indisponible non précisé ; type de retour non défini ; tests absents ; hypothèses laissées à l'outil.

Le prompt vague n'est pas seulement moins précis. Il déplace des décisions métier vers l'assistant. Or ces décisions sont exactement celles que l'équipe doit maîtriser.

4.2 Anatomie du prompt technique du cours

Bloc	Question à laquelle il répond	Exemple
Contexte	Dans quel système et pour quel objectif ?	Robot porte-palette, logique d'autorisation de mouvement.
Module	Où se situe la modification ?	Composant de décision / mission, sans accès direct au hardware.
Criticité	Quel niveau de prudence ?	Élevée : une erreur peut autoriser un mouvement inadmissible.
Entrées	Quelles données sont disponibles ?	Batterie, freins, capteur frontal, erreur critique, mode.

Bloc	Question à laquelle il répond	Exemple
Sorties	Que doit produire le code ?	Décision autorisé/refusé + raison du refus.
Erreurs / cas invalides	Que faire si l'information est incohérente ?	Refuser dans l'exemple pédagogique et signaler la raison.
Contraintes C++	Quelles conventions techniques ?	C++17, enum class, pas de dépendance externe pour l'exemple.
Tests	Quels comportements doivent être prouvés ?	Un test par condition + seuil batterie + cas nominal.
Critères d'acceptation	Quand considère-t-on la réponse exploitable ?	Toutes les contraintes tracées, code compilable, tests exécutables.

4.3 Pourquoi chaque bloc est utile

Contexte et module

Le contexte évite une solution générique déconnectée. Le module fixe une frontière : on ne veut pas qu'une simple politique de décision commence soudain à commander directement un moteur ou à lire un bus matériel.

Criticité

La criticité influence la profondeur de la réponse demandée. Pour une fonction sensible, on demande explicitement les hypothèses, les cas de refus, les tests négatifs et les limites. On évite un prompt qui demande seulement "le code le plus simple".

Entrées et sorties

Les entrées et sorties forment un contrat. Une entrée doit avoir un sens clair et un type adapté. Une sortie doit permettre au code appelant de comprendre la décision. Un simple booléen peut suffire techniquement, mais un résultat structuré est souvent plus explicite pour l'analyse et le diagnostic.

Erreurs et cas invalides

Un bon prompt ne décrit pas seulement le scénario heureux. Il précise ce qui se passe quand une condition manque. Dans l'exemple pédagogique, l'absence d'une condition nécessaire conduit au refus de mouvement : c'est un choix explicite du support, pas une règle industrielle universelle.

Tests et critères d'acceptation

Les tests rendent le prompt falsifiable : une réponse peut échouer à un test. Les critères d'acceptation évitent une discussion subjective sur le fait que le code "a l'air propre".

4.4 Gabarit de prompt réutilisable

Gabarit à distribuer / recopier

```
CONTEXTE
- Système : <décrire le système>
- Objectif métier : <ce que le composant doit décider ou produire>

MODULE ET PÉRIMÈTRE
- Module concerné : <nom / responsabilité>
- Ne doit PAS : <responsabilités hors périmètre>
```

CRITICITÉ
- Niveau : <vert / orange / rouge dans le cadre pédagogique>
- Conséquence principale d'une erreur : <impact>

ENTRÉES
- <entrée 1> : <type + signification>
- <entrée 2> : <type + signification>

SORTIE ATTENDUE
- <type / structure + signification>

RÈGLES ET CONTRAINTES
1. <règle 1>
2. <règle 2>
...

CAS D'ERREUR / CAS LIMITES
- <cas> -> <comportement attendu>

CONTRAINTES C++
- Version : C++17
- Dépendances : <autorisées / interdites>
- Style : <règles utiles>

TESTS ATTENDUS
- Cas nominal
- Un test par condition de refus
- Cas limite sur les seuils

CRITÈRES D'ACCEPTATION
- Le code compile
- Chaque règle métier est traçable vers un test
- Aucune valeur métier n'est inventée silencieusement
- Les hypothèses sont listées

4.5 Exemple complet sur l'autorisation de mouvement

Prompt technique de démonstration

CONTEXTE
Nous travaillons sur un robot porte-palette. Nous voulons une petite unité de décision qui indique si un mouvement peut être autorisé.

MODULE ET PÉRIMÈTRE
Le composant ne commande aucun moteur et ne lit aucun capteur. Il reçoit uniquement un état déjà construit et renvoie une décision.

CRITICITÉ
Rouge dans notre classification pédagogique, car une décision erronée peut autoriser un mouvement inadmissible.

ENTRÉES
- batteryPercent : entier de 0 à 100
- brakesAvailable : bool
- frontSensorAvailable : bool
- criticalError : bool
- mode : enum RobotMode { Idle, Moving, Maintenance, Error }

SORTIE
Une structure MoveDecision contenant :
- allowed : bool

```
- reason : enum MoveDenialReason

RÈGLES
Le mouvement est autorisé seulement si :
1. batteryPercent >= 20
2. brakesAvailable == true
3. frontSensorAvailable == true
4. criticalError == false
5. mode == RobotMode::Idle

CONTRAINTES C++
- C++17
- pas de bibliothèque externe
- enum class pour les énumérations
- aucune valeur métier autre que celles fournies

TESTS
- cas nominal autorisé
- batterie à 19 -> refus
- batterie à 20 -> cas limite autorisé
- freins indisponibles -> refus
- capteur frontal indisponible -> refus
- erreur critique -> refus
- mode différent de Idle -> refus

CRITÈRES D'ACCEPTATION
Le code compile avec g++ -std=c++17 et tous les tests passent.
```

4.6 Faire apparaître les hypothèses

Même un prompt détaillé peut laisser des zones ouvertes. Demander à l'IA de commencer par lister les hypothèses qu'elle estime nécessaires. Si elle ajoute un comportement non demandé, on peut l'identifier avant de fusionner le code.

QUESTION À POSER À L'IA

Avant de générer le code, liste les informations manquantes et les hypothèses que tu serais obligé de faire. N'invente aucune valeur métier sans la signaler.

4.7 Atelier : transformer une demande vague

CONSIGNE

Point de départ : "Ajoute une fonction pour savoir si le robot peut partir." En binômes, produisez un prompt complet avec les neuf blocs vus précédemment. Vous pouvez utiliser les contraintes du fil rouge, mais vous devez distinguer clairement ce qui est fourni et ce que vous proposez comme choix pédagogique.

Grille de correction

Critère	0	1	2
Contexte / module	Absent	Partiel	Responsabilité et périmètre explicites

Critère	0	1	2
Règles	Générales	Plusieurs règles	Toutes les contraintes sont testables
Cas d'erreur	Absent	1 cas	Cas négatifs et limites précisés
Tests	Absent	Cas nominal	Cas nominal + refus + limite
Acceptation	Subjective	Partielle	Critères vérifiables

4.8 Mini-synthèse 13:30-15:00

À RETENIR

Un bon prompt technique ne garantit pas une bonne solution. Il garantit surtout que nous savons quelles informations nous avons données et selon quels critères nous allons juger la réponse.

15:00-16:30 - Générer une petite unité de code C++ et ses tests

OBJECTIF DE LA SÉQUENCE

Utiliser l'IA pour produire une petite unité de code tout en gardant le contrôle sur les règles métier et les tests.

La démonstration de code suit exactement les cinq contraintes prévues : batterie minimale, freins disponibles, capteur frontal disponible, absence d'erreur critique et état Idle. L'objectif n'est pas de produire une architecture robotique réelle, mais de montrer comment tracer chaque contrainte jusqu'au code et aux tests.

5.1 Choix pédagogique : une politique de décision pure

Pour rendre la démonstration contrôlable, le composant ne lit pas lui-même les capteurs et ne commande pas le robot. Il reçoit un état et retourne une décision. Cela limite le nombre de responsabilités et permet de tester la logique sans hardware réel.

Entrée	Type pédagogique	Rôle
batteryPercent	int	Niveau de batterie déjà mesuré / fourni au composant.
brakesAvailable	bool	Indique si la condition "freins disponibles" est satisfaite.
frontSensorAvailable	bool	Indique si le capteur frontal est disponible.
criticalError	bool	Signale la présence d'une erreur critique.
mode	RobotMode	État logique du robot ; seul Idle est admis par la règle.

5.2 Version fragile : un booléen compact et difficile à diagnostiquer

Exemple volontairement fragile

```
bool canMove(int battery, bool brakes, bool sensor, bool error, int mode) {
    return battery >= 20 && brakes && sensor && !error && mode == 0;
}
```

Cette version peut sembler séduisante parce qu'elle est courte. Pourtant elle contient plusieurs problèmes pédagogiques : les paramètres booléens sont peu expressifs au point d'appel, mode == 0 est un nombre magique, aucune raison de refus n'est disponible et une modification future peut rendre l'expression difficile à relire.

FRAGILE	MIEUX
canMove(25, true, false, false, 0) - il faut mémoriser la signification de chaque booléen et de 0.	evaluate(status) - l'état est regroupé et la décision contient une raison explicite de refus.

5.3 Version améliorée : types et raison de refus

Partie 1 - modèle et politique de décision

```
#include <cassert>
#include <iostream>
#include <string>

enum class RobotMode {
    Idle,
    Moving,
    Maintenance,
    Error
};

enum class MoveDenialReason {
    None,
    BatteryTooLow,
    BrakesUnavailable,
    FrontSensorUnavailable,
    CriticalError,
    RobotNotIdle
};

struct RobotStatus {
    int batteryPercent;
    bool brakesAvailable;
    bool frontSensorAvailable;
    bool criticalError;
    RobotMode mode;
};

struct MoveDecision {
    bool allowed;
    MoveDenialReason reason;
};

class MovementAuthorizationPolicy {
public:
    explicit MovementAuthorizationPolicy(int minBatteryPercent)
        : minBatteryPercent_(minBatteryPercent) {}
```

```
    MoveDecision evaluate(const RobotStatus& status) const {
        if (status.batteryPercent < minBatteryPercent_) {
            return {false, MoveDenialReason::BatteryTooLow};
        }
        if (!status.brakesAvailable) {
            return {false, MoveDenialReason::BrakesUnavailable};
        }
        if (!status.frontSensorAvailable) {
            return {false, MoveDenialReason::FrontSensorUnavailable};
        }
        if (status.criticalError) {
            return {false, MoveDenialReason::CriticalError};
        }
        if (status.mode != RobotMode::Idle) {
            return {false, MoveDenialReason::RobotNotIdle};
        }
        return {true, MoveDenialReason::None};
    }

private:
    int minBatteryPercent_;
};
```

5.4 Lecture ligne par ligne : ce que le code exprime

RobotMode

enum class donne des noms aux états. On préfère RobotMode::Idle à un entier 0 car le nom porte le sens. Le compilateur aide aussi à éviter certains mélanges de valeurs.

MoveDenialReason

La raison du refus rend la décision observable. Elle pourra être utilisée par un test, un log ou une couche de supervision. Elle évite le piège du simple booléen qui indique “non” sans expliquer pourquoi.

RobotStatus

La structure regroupe les données utilisées par la décision. Dans un vrai système, l’origine et la validité de ces données devraient être définies ailleurs. Ici, on garde volontairement un objet simple pour se concentrer sur la logique métier.

MovementAuthorizationPolicy

Le seuil de batterie est passé au constructeur au lieu d’être caché dans la méthode. Cela rend la dépendance visible. Pour la démonstration, nous utiliserons 20, conformément au prompt d’exemple. Si le projet réel possède une configuration officielle, c’est cette source qui doit faire autorité.

Ordre des conditions

Chaque condition de refus est séparée. Cela facilite la lecture et les tests. L’ordre détermine la première raison renvoyée si plusieurs conditions échouent en même temps ; c’est donc un comportement à connaître et, si nécessaire, à spécifier.

5.5 Les tests : partir des règles, pas du code

La meilleure manière d’éviter des tests qui répètent aveuglément l’implémentation est de dériver d’abord la liste des cas à partir du prompt. Chaque règle métier devient au moins un cas de test.

Règle	Cas de test	Résultat attendu
Batterie minimale	battery = 19	Refus : BatteryTooLow
Borne du seuil	battery = 20, autres conditions OK	Autorisé
Freins disponibles	brakesAvailable = false	Refus : BrakesUnavailable
Capteur frontal disponible	frontSensorAvailable = false	Refus : FrontSensorUnavailable
Pas d'erreur critique	criticalError = true	Refus : CriticalError
Mode Idle	mode = Moving	Refus : RobotNotIdle
Cas nominal	Toutes les conditions satisfaites	Autorisé : None

5.6 Code complet exécutable avec tests assert

main.cpp - version complète à copier-coller

```
#include <cassert>
#include <iostream>

enum class RobotMode { Idle, Moving, Maintenance, Error };

enum class MoveDenialReason {
    None,
    BatteryTooLow,
    BrakesUnavailable,
    FrontSensorUnavailable,
    CriticalError,
    RobotNotIdle
};

struct RobotStatus {
    int batteryPercent;
    bool brakesAvailable;
    bool frontSensorAvailable;
    bool criticalError;
    RobotMode mode;
};

struct MoveDecision {
    bool allowed;
    MoveDenialReason reason;
};

class MovementAuthorizationPolicy {
public:
    explicit MovementAuthorizationPolicy(int minBatteryPercent)
        : minBatteryPercent_(minBatteryPercent) {}

    MoveDecision evaluate(const RobotStatus& status) const {
        if (status.batteryPercent < minBatteryPercent_)
            return {false, MoveDenialReason::BatteryTooLow};
        if (!status.brakesAvailable)
            return {false, MoveDenialReason::BrakesUnavailable};
        if (!status.frontSensorAvailable)
            return {false, MoveDenialReason::FrontSensorUnavailable};
        if (status.criticalError)
            return {false, MoveDenialReason::CriticalError};
        if (status.mode != RobotMode::Idle)
```

```
        return {false, MoveDenialReason::RobotNotIdle};
    return {true, MoveDenialReason::None};
  }

private:
  int minBatteryPercent_;
};

RobotStatus validStatus() {
  return {50, true, true, false, RobotMode::Idle};
}

void testNominalCase() {
  MovementAuthorizationPolicy policy(20);
  const auto decision = policy.evaluate(validStatus());
  assert(decision.allowed);
  assert(decision.reason == MoveDenialReason::None);
}

void testBatteryTooLow() {
  MovementAuthorizationPolicy policy(20);
  auto status = validStatus();
  status.batteryPercent = 19;
  const auto decision = policy.evaluate(status);
  assert(!decision.allowed);
  assert(decision.reason == MoveDenialReason::BatteryTooLow);
}

void testBatteryBoundary() {
  MovementAuthorizationPolicy policy(20);
  auto status = validStatus();
  status.batteryPercent = 20;
  assert(policy.evaluate(status).allowed);
}

void testBrakesUnavailable() {
  MovementAuthorizationPolicy policy(20);
  auto status = validStatus();
  status.brakesAvailable = false;
  assert(policy.evaluate(status).reason == MoveDenialReason::BrakesUnavailable);
}

void testFrontSensorUnavailable() {
  MovementAuthorizationPolicy policy(20);
  auto status = validStatus();
  status.frontSensorAvailable = false;
  assert(policy.evaluate(status).reason == MoveDenialReason::FrontSensorUnavailable);
}

void testCriticalError() {
  MovementAuthorizationPolicy policy(20);
  auto status = validStatus();
  status.criticalError = true;
  assert(policy.evaluate(status).reason == MoveDenialReason::CriticalError);
}

void testRobotNotIdle() {
  MovementAuthorizationPolicy policy(20);
  auto status = validStatus();
  status.mode = RobotMode::Moving;
  assert(policy.evaluate(status).reason == MoveDenialReason::RobotNotIdle);
}
```

```
int main() {
    testNominalCase();
    testBatteryTooLow();
    testBatteryBoundary();
    testBrakesUnavailable();
    testFrontSensorUnavailable();
    testCriticalError();
    testRobotNotIdle();
    std::cout << "Tous les tests sont passes.\n";
}
```

Compilation et exécution

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o robot_policy
./robot_policy
```

Sous Windows avec g++ disponible dans le terminal, l'exécutable pourra être lancé par robot_policy.exe. Le résultat attendu est : "Tous les tests sont passes."

IMPORTANT

Ces tests démontrent le comportement de l'exemple pédagogique. Ils ne constituent pas une validation de sûreté d'un robot réel.

5.7 Faire travailler l'IA par itérations courtes

Une bonne démonstration ne demande pas "génère tout le composant final" en une seule fois. On réduit la surface de changement et on vérifie une chose à la fois.

1. Demander les types et la signature, sans implémentation.
2. Vérifier que toutes les entrées et sorties sont représentées.
3. Demander l'implémentation de la décision.
4. Comparer chaque branche à une règle du prompt.
5. Demander les tests dérivés des règles.
6. Ajouter le cas limite batterie = 20.
7. Compiler et exécuter localement.
8. Revoir les hypothèses et l'ordre des raisons de refus.

5.8 Pièges typiques à injecter dans la démonstration

Piège volontaire	Pourquoi il est intéressant	Correction attendue
battery > 20 au lieu de >= 20	Erreur de borne discrète.	Test battery = 20 la détecte.
Oublier frontSensorAvailable	Omission d'une exigence.	Traçabilité règle -> branche -> test.
mode == 0	Nombre magique et type peu explicite.	enum class RobotMode.
Renvoie seulement bool	Diagnostic faible.	MoveDecision avec raison.
Test uniquement cas nominal	La majorité des refus n'est jamais exercée.	Un test par condition + borne.

Piège volontaire	Pourquoi il est intéressant	Correction attendue
Inventer un seuil 30	Décision métier non fournie.	Refuser / demander clarification ou utiliser le seuil spécifié.

5.9 Atelier : corriger une sortie IA non conforme

CONSIGNE

Donner aux participants une version dans laquelle le capteur frontal est oublié et le test de batterie utilise > 20. Ils doivent : 1) identifier les écarts, 2) relier chaque écart au prompt, 3) corriger le code, 4) ajouter les tests qui auraient empêché la régression.

Correction attendue

- Remplacer la comparaison de batterie par batteryPercent < minBatteryPercent_ comme condition de refus, ce qui rend la borne égale autorisée.
- Ajouter la vérification frontSensorAvailable avant l'autorisation.
- Ajouter testBatteryBoundary et testFrontSensorUnavailable.
- Vérifier que le cas nominal passe toujours après les corrections.
- Expliquer que la correction vient de la spécification, pas d'une préférence de style.

5.10 Mini-synthèse 15:00-16:30

À RETENIR

On ne vérifie pas une sortie IA en demandant seulement si elle compile. On trace chaque contrainte vers le code et vers un test, puis on exécute réellement les contrôles.

16:30-17:30 - Revue IA + revue humaine

OBJECTIF DE LA SÉQUENCE

Savoir demander une revue à l'IA puis effectuer une seconde revue humaine indépendante.

La dernière heure donne une méthode de revue. L'IA peut jouer le rôle d'un lecteur supplémentaire, mais ses remarques doivent être triées. Le développeur doit être capable d'accepter, modifier ou refuser une suggestion en s'appuyant sur le besoin et sur des vérifications.

6.1 Pourquoi faire une revue IA si elle n'est pas l'autorité ?

Une seconde lecture automatique peut augmenter le nombre de pistes examinées : conditions oubliées, noms ambigus, tests manquants, branches non couvertes, hypothèses implicites. Sa valeur est de proposer des angles de revue, pas de rendre un verdict.

6.2 Un prompt de revue utile

Prompt de revue guidée

Relis cette unité C++ à partir des critères suivants.

1. Compare le code aux règles métier fournies.
2. Liste toute contrainte absente ou ambiguë.
3. Cherche les erreurs de borne et les cas négatifs oubliés.
4. Vérifie si les tests couvrent chaque règle.
5. Ne propose pas de refactoring hors périmètre sauf s'il corrige un risque identifié.
6. Pour chaque remarque, indique :
   - le risque,
   - l'élément de code concerné,
   - la manière de vérifier la remarque.
7. Termine par la liste de tes hypothèses et incertitudes.

6.3 La matrice "suggestion / vérification / décision"

Suggestion IA	Vérification indépendante	Décision humaine
"Le seuil devrait être 30."	Le besoin fixe 20 ; aucune source ne donne 30.	REFUSER : valeur inventée.
"Ajouter un test à la borne 20."	Le code utilise une comparaison de seuil ; la borne est un risque classique.	ACCEPTER : test directement lié au critère.
"Remplacer enum class par int pour simplifier."	Perte de sens et pas de besoin de compatibilité imposant int.	REFUSER : simplification nuisible.
"La raison de refus n'est pas testée dans le cas capteur absent."	Le test actuel ne vérifie que allowed == false.	MODIFIER / ACCEPTER : vérifier aussi la raison.
"Fusionner toutes les conditions dans une expression."	La lisibilité et le diagnostic diminuent, aucune performance exigée.	REFUSER dans ce contexte pédagogique.

6.4 Repérer un code dangereux sans se laisser distraire par le style

Dans une revue industrielle, l'ordre des priorités compte. Une indentation parfaite ne compense pas une contrainte manquante. On peut utiliser l'ordre de lecture suivant :

1. Conformité au besoin : les règles sont-elles toutes présentes ?
2. Comportements dangereux : peut-on autoriser alors qu'une condition est fausse ?
3. Cas limites : seuils, états inconnus, valeurs invalides.
4. Observabilité : peut-on comprendre pourquoi la décision a été prise ?
5. Tests : chaque règle et chaque cas négatif important est-il exercé ?
6. Lisibilité et maintenabilité : le code exprime-t-il clairement l'intention ?
7. Style mineur : seulement après les points précédents.

6.5 Code non optimal pour l'atelier final du J1

Code à analyser

```
bool canStart(const RobotStatus& s) {
    if (s.batteryPercent > 20 && s.brakesAvailable && !s.criticalError) {
        return true;
    }
    return false;
}
```

Questions à poser avant de montrer la correction

- Quelles contraintes du prompt sont présentes ?
- Quelles contraintes ont disparu ?
- Quel cas limite est probablement faux ?
- Le mode du robot est-il vérifié ?
- Le capteur frontal est-il vérifié ?
- Peut-on expliquer la cause d'un refus ?
- Quel test minimal détecterait chaque défaut ?

Correction raisonnée

Le code est insuffisant même s'il compile. Il oublie le capteur frontal et le mode Idle. Il utilise > 20 au lieu de la règle >= 20, donc refuse injustement la valeur limite. Il retourne seulement un booléen, ce qui réduit le diagnostic. La correction doit être justifiée par le prompt et les tests, pas seulement par une préférence de style.

6.6 Checklist de double revue

Contrôle	Revue IA	Revue humaine / preuve
Toutes les contraintes sont présentes	Peut signaler les omissions.	Comparer ligne par ligne au prompt / ticket.
Valeurs métier justifiées	Peut lister les constantes.	Vérifier la source officielle de chaque valeur.
Cas limites couverts	Peut suggérer des tests.	Choisir les limites pertinentes et exécuter les tests.
Comportement négatif sûr	Peut repérer des branches manquantes.	Vérifier l'effet réel de chaque condition absente.
Tests réellement exécutés	Peut générer du code de test.	Compiler et exécuter localement / CI.
Refactoring adapté au périmètre	Peut proposer des améliorations.	Accepter seulement ce qui sert le besoin et respecte l'architecture.
Décision finale	Ne doit pas être considérée comme autorité.	Accepté / modifié / refusé avec justification.

6.7 Atelier : accepter, modifier ou refuser

CONSIGNE

À partir du code non optimal et d'une liste de suggestions IA, remplissez trois colonnes : suggestion / vérification / décision. Pour chaque décision, écrivez une justification courte. Une suggestion "bonne" mais non vérifiée n'est pas encore acceptée.

6.8 Synthèse de fin de J1

LES CINQ COMPÉTENCES À FAIRE VERBALISER

1. Je distingue assistance, génération et validation.
2. Je sais augmenter les contrôles quand la criticité augmente.
3. Je transforme une demande vague en prompt testable.
4. Je trace les contraintes vers le code et les tests.
5. Je peux justifier l'acceptation ou le refus d'une suggestion IA.

Annexe A - Fiche formateur : questions de relance

Moment	Question de relance	But pédagogique
Introduction	“Si le code compile, qu’est-ce que cela prouve exactement ?”	Dissocier compilation et correction métier.
Criticité	“Si cette fonction se trompe, qui ou quoi subit la conséquence ?”	Raisonner par impact.
Charte IA	“Comment vérifier que votre règle a été appliquée ?”	Transformer les principes vagues en règles observables.
Prompt	“Quelle décision êtes-vous en train de laisser l’IA inventer ?”	Faire apparaître les trous de spécification.
Code	“Quelle ligne du prompt justifie cette branche ?”	Créer la traçabilité exigence -> code.
Tests	“Quel défaut ce test serait-il capable de détecter ?”	Éviter les tests décoratifs.
Revue	“Pouvez-vous vérifier cette suggestion sans demander à l’IA si elle a raison ?”	Renforcer l’indépendance de la revue.

Annexe B - Fiche participant : gabarit de prompt

CONTEXTE :

_____

MODULE / PÉRIMÈTRE :

_____

CRITICITÉ ET IMPACT D’UNE ERREUR :

_____

ENTRÉES :

_____

SORTIES :

_____

RÈGLES / CONTRAINTES :

_____

CAS D’ERREUR / LIMITES :

_____

CONTRAINTES C++ :

_____

TESTS ATTENDUS :

_____

CRITÈRES D'ACCEPTATION :

_____

HYPOTHÈSES À SIGNALER :

_____

Annexe C - Fiche participant : revue IA + revue humaine

Suggestion IA	Comment je la vérifie	Décision : accepter / modifier / refuser	Justification

Annexe D - Glossaire J1

Terme	Définition simple
IA - Intelligence artificielle	Dans ce support : assistant capable de générer ou transformer texte et code à partir d'instructions.
Prompt	Instruction donnée à l'assistant. Un prompt technique contient ici le contexte, les contraintes et les critères de contrôle.
Criticité	Niveau de conséquence potentiel d'une erreur et besoin de contrôle associé.
Hardware	Partie matérielle ou couche logicielle d'accès aux éléments physiques.
Safety	Ensemble de mécanismes et règles visant à empêcher des comportements dangereux. Le terme est utilisé ici au sens de responsabilité du système, sans prétendre remplacer une analyse de sûreté formelle.
Navigation	Fonctions liées à la localisation et au déplacement.
Mission	Logique qui décrit ce que le robot doit accomplir et dans quel ordre.
Supervision	Vue globale de l'état du système et interaction avec l'opérateur ou un système supérieur.
Diagnostic	Informations et outils permettant d'identifier et comprendre défauts et événements.
Traçabilité	Capacité à retrouver les éléments ayant conduit à une décision : besoin, contraintes, tests, revue, acceptation.
Test nominal	Test du scénario attendu lorsque toutes les conditions nécessaires sont satisfaites.
Test négatif	Test montrant qu'une condition invalide conduit bien au refus ou à l'erreur attendue.

Terme	Définition simple
Cas limite	Valeur située à une frontière de décision, par exemple exactement 20 % de batterie.
enum class	Énumération C++ fortement typée, utilisée ici pour nommer clairement modes et raisons de refus.
assert	Mécanisme C++ simple qui arrête le programme si une condition de test est fausse ; utilisé ici pour éviter une dépendance à un framework externe.
Revue	Lecture critique du code visant à détecter écarts, risques, erreurs ou améliorations nécessaires.

Annexe E - Checklist de préparation du formateur

- Préparer un exemple de prompt volontairement vague.
- Préparer la cartographie des sept couches à dessiner progressivement.
- Imprimer ou projeter les fonctions à classer vert / orange / rouge.
- Préparer le gabarit de prompt technique.
- Vérifier que g++ ou un compilateur C++17 est disponible si la démonstration est exécutée en direct.
- Copier le fichier main.cpp complet et vérifier sa compilation avant la session.
- Préparer une version volontairement défectueuse : oubli du capteur, borne batterie > 20, mode non vérifié.
- Préparer la matrice suggestion / vérification / décision.
- Rappeler au groupe que les exemples de safety sont pédagogiques et ne remplacent pas les processus et normes de l’entreprise.

Annexe F - Auto-évaluation de fin de J1

Je suis capable de…	Oui	À revoir
Expliquer la différence entre assistance, génération et validation.		
Expliquer pourquoi une sortie plausible peut être incorrecte.		
Raisonner sur la criticité d’une fonction avant d’utiliser l’IA.		
Rédiger une règle interne observable et contrôlable.		
Construire un prompt avec contexte, entrées, sorties, contraintes et tests.		
Tracer une contrainte vers une branche de code et vers un test.		
Identifier un cas limite sur un seuil.		
Utiliser une revue IA sans lui déléguer la décision finale.		
Justifier le refus d’une suggestion IA.		

J1 - ÉDITION ÉTENDUE

Comprendre, cadrer et utiliser l’IA correctement en C++ industriel

Approfondissements, variantes d’animation, exercices corrigés et démonstrations complémentaires

Objet de cette extension

Cette édition étendue approfondit les six séquences du J1 sans modifier le déroulé : responsabilité, criticité, règles d'usage, prompt technique, génération/test C++ et double revue. Elle ajoute des explications et ateliers destinés au formateur tout en conservant la règle centrale : l'IA propose, l'équipe vérifie et décide.

Principe de densification

Le contenu existant est conservé comme socle. Les pages suivantes approfondissent chaque séquence avec des explications supplémentaires, des variantes de démonstration, des contre-exemples, des questions de groupe, des exercices corrigés et des mises en situation. L'objectif n'est pas d'ajouter des pages pour elles-mêmes : chaque ajout doit aider le formateur à expliquer, faire pratiquer, faire verbaliser ou contrôler une compétence du déroulé pédagogique.

Mode d’utilisation conseillé

- Lire l'approfondissement correspondant avant la séquence, puis sélectionner les exemples adaptés au niveau réel du groupe.
- Ne pas tout réciter : utiliser les blocs comme réserve d'explications, de questions et de contre-exemples.
- Conserver le fil rouge du robot porte-palette pour relier les notions et éviter l'impression d'une collection de recettes.
- Faire distinguer systématiquement ce que l'IA propose, ce qu'un outil déterministe vérifie, et ce qu'un humain décide.
- Revenir aux productions attendues du déroulé : cartographies, checklists, fichiers d'instructions, contrats, tâches locales et dossier de ticket.

Approfondissement 1 - Proposition plausible, preuve et niveau de confiance

À DIRE TRÈS SIMPLEMENT
Une réponse IA peut être utile sans être vraie. On l'utilise comme point de départ, jamais comme certificat.

Explication approfondie. Dans un atelier, les participants ont souvent tendance à évaluer une réponse avec trois signaux trompeurs : elle est rapide, elle est bien écrite et elle ressemble à du code qu'ils connaissent. Aucun de ces signaux ne prouve que les exigences métier sont respectées. Il faut donc séparer la qualité de présentation de la qualité de preuve. Une proposition devient exploitable lorsqu'on peut la confronter à une spécification, à des tests et à des contraintes connues.

Le mécanisme pédagogique consiste à introduire quatre niveaux : proposition, vérification, décision, traçabilité. La proposition est produite par l'IA ou par un humain. La vérification rassemble des faits observables : compilation, tests, lecture d'interface, documentation, règles projet. La décision accepte, modifie ou refuse. La traçabilité permet de retrouver pourquoi cette décision a été prise. Cette chaîne est particulièrement importante lorsqu'un défaut peut affecter un mouvement physique.

Application au robot porte-palette. Demander à l'IA si le robot peut avancer avec 18 % de batterie n'a pas de sens tant que le seuil n'est pas connu. Si l'outil invente 20 %, la réponse peut sembler raisonnable mais reste une hypothèse. Le groupe doit apprendre à demander : d'où vient le seuil, quelle règle le fixe et quel test montre le comportement à la frontière ?

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
« L'IA a dit que la fonction est sûre et les tests qu'elle a générés passent. On peut fusionner. » Cette phrase mélange génération, auto-évaluation et validation.

VERSION MIEUX MAÎTRISÉE
« L'IA propose une implémentation. Nous comparons chaque règle au ticket, nous exécutons les tests définis par l'équipe, nous vérifions les cas limites et un reviewer humain décide de l'intégration. »

Pièges et nuances à anticiper

- Confondre compilation et conformité métier.
- Considérer qu'un test généré par la même hypothèse que le code constitue une preuve indépendante.
- Laisser l'IA inventer une valeur métier manquante.
- Utiliser des mots comme sûr, correct ou conforme sans critères associés.

Questions au groupe

- Qu'est-ce qui pourrait être faux même si le code compile ?
- Quelle preuve vous ferait changer d'avis sur cette proposition ?
- Qui possède la règle métier utilisée par la décision ?

Mini-activité guidée
Présenter trois réponses IA à une même demande, dont une syntaxiquement élégante mais avec un seuil inventé. Faire surligner en vert les faits fournis, en orange les hypothèses et en rouge les affirmations non vérifiées.

Correction et débrief
La meilleure réponse n'est pas nécessairement celle qui contient le plus de code. La réponse la plus exploitable est celle qui expose ses hypothèses, respecte les contraintes connues et facilite la vérification. Toute valeur métier absente doit être signalée, pas inventée.

À RETENIR : Une sortie plausible est une proposition ; la preuve vient de critères et de contrôles indépendants.

Approfondissement 2 - Criticité : raisonner par conséquence d'erreur

À DIRE TRÈS SIMPLEMENT
Plus une erreur peut produire une conséquence grave, plus on augmente les contrôles avant d'accepter le code.

Explication approfondie. Le classement vert, orange et rouge n'a d'intérêt que s'il modifie concrètement le processus de travail. Il ne s'agit pas de coller une couleur sur un fichier, mais de réfléchir aux conséquences d'une erreur, à la détectabilité du défaut et à la possibilité de revenir en arrière. Deux fonctions de taille identique peuvent exiger des niveaux

de revue très différents selon leur rôle dans le système.

On peut faire raisonner avec quatre questions : que se passe-t-il si la fonction se trompe, à quelle vitesse le défaut est-il visible, peut-on limiter son effet, et existe-t-il un mécanisme indépendant de protection ? Une zone rouge réclame davantage de spécification, de tests négatifs, de revue humaine et parfois l'interdiction d'une génération automatique non supervisée. Une zone verte n'est pas 'sans risque' ; elle autorise simplement un processus plus léger.

Application au robot porte-palette. Une fonction qui reformate un message de diagnostic peut être traitée plus légèrement qu'une fonction qui autorise un mouvement. Si la première se trompe, l'opérateur voit peut-être un texte imparfait. Si la seconde se trompe, l'action physique peut être inappropriée. La couleur vient de cette différence d'impact, pas du langage C++ utilisé.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Classer hardware en rouge, supervision en vert, puis appliquer la même checklist à toutes les fonctions sans ajuster la profondeur des contrôles.

VERSION MIEUX MAÎTRISÉE
Utiliser la couleur pour décider du niveau de prompt, du nombre de cas négatifs, du type de revue, des preuves obligatoires et de l'autorité qui accepte le changement.

Pièges et nuances à anticiper

- Transformer l'outil pédagogique en certification de sûreté.
- Classer une couche entière sans regarder la fonction réelle.
- Confondre probabilité d'erreur et gravité des conséquences.
- Supposer qu'une zone verte n'a besoin d'aucun test.

Questions au groupe

- Pourquoi deux fonctions de dix lignes peuvent-elles avoir des exigences de revue différentes ?
- Quel critère ferait passer une fonction orange en rouge ?
- Une fonction de diagnostic peut-elle devenir critique dans certains usages ?

Mini-activité guidée
Distribuer huit cartes de fonctions et demander pour chacune : conséquence, contrôles, niveau de validation. Le groupe doit justifier la couleur en une phrase et proposer un contrôle proportionné.

Correction et débrief
La couleur doit être reliée à une conséquence et à un dispositif de contrôle. Un classement sans justification n'est pas exploitable. Une même fonction peut changer de niveau si son rôle dans l'architecture change.

À RETENIR : La criticité ne décrit pas la complexité du code ; elle décrit le niveau de prudence imposé par les conséquences possibles.

Approfondissement 3 - Confidentialité : décider avant de copier dans un prompt

À DIRE TRÈS SIMPLEMENT
Avant d'envoyer du code ou des données à un assistant, on vérifie si on a le droit de les transmettre.

Explication approfondie. Dans un usage professionnel, la confidentialité ne peut pas être réduite à 'ne pas coller de secret'. Un extrait de code peut révéler une architecture, des noms de clients, des identifiants, une logique métier propriétaire ou des détails d'infrastructure. Le réflexe doit donc précéder la qualité du prompt : on qualifie d'abord l'information, puis on décide ce qui peut être fourni à l'outil autorisé.

Le formateur peut proposer un tri en quatre catégories pédagogiques : public, interne partageable dans l'outil validé, sensible à anonymiser, interdit à transmettre. L'important est d'insister sur la politique réelle de l'entreprise. La règle ne se déduit pas du niveau technique de l'étudiant. Quand le contexte nécessaire est sensible, on peut reformuler le problème avec des noms génériques, réduire l'extrait au minimum ou travailler hors assistant.

Application au robot porte-palette. Au lieu de copier un fichier contenant des adresses réseau, des identifiants de site et des journaux d'exploitation, on construit un exemple réduit : état du frein, disponibilité du capteur, niveau de batterie. On conserve le problème technique tout en retirant les informations inutiles à la tâche.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Copier une grande portion de dépôt 'pour donner plus de contexte' sans vérifier la classification des informations ni l'outil autorisé.

VERSION MIEUX MAÎTRISÉE
Donner uniquement le contexte nécessaire, anonymisé si besoin, dans l'environnement approuvé, et conserver une trace du type d'information partagé lorsque le processus l'exige.

Pièges et nuances à anticiper

- Croire qu'un petit extrait est automatiquement non sensible.
- Confondre anonymisation et suppression de tout risque.
- Inclure des logs contenant des jetons, URLs internes ou données opérationnelles.
- Utiliser un outil personnel lorsque la politique impose un outil d'entreprise.

Questions au groupe

- Quelle information du contexte est réellement nécessaire à la tâche ?
- Comment reproduire le problème sans transmettre le fichier complet ?
- Qui décide de la politique d'usage de l'assistant dans l'entreprise ?

Mini-activité guidée
Fournir un prompt volontairement trop riche contenant noms de machines, chemin interne, ticket et extrait de log. Faire barrer tout élément qui n'est pas nécessaire au raisonnement technique et demander une version minimisée.

Correction et débrief
La version corrigée conserve le contrat technique et les valeurs utiles au scénario, mais retire les identifiants et détails non nécessaires. La conclusion doit rappeler qu'une politique interne peut être plus restrictive que l'exercice.

À RETENIR : Le bon prompt commence par la bonne décision de partage de l'information.

Approfondissement 4 - Prompt technique : transformer un besoin en contrat vérifiable

À DIRE TRÈS SIMPLEMENT
Un bon prompt ne demande pas seulement du code : il donne le contexte, les contraintes et la manière de vérifier le résultat.

Explication approfondie. Une demande vague force l'assistant à combler les blancs. Plus le domaine est contraint, plus ces blancs deviennent dangereux. Le prompt technique doit donc réduire l'ambiguïté sans devenir un roman. Il décrit l'objectif, la zone de code concernée, les règles métier connues, les interdits, le format de sortie et les critères d'acceptation. Cette structure améliore aussi le travail humain : elle rend le besoin discutable avant la génération.

On peut enseigner le prompt comme un contrat temporaire. Le contexte explique où l'on travaille. La tâche dit ce qui doit changer. Les contraintes bornent les solutions. Les critères d'acceptation permettent de tester. Les hypothèses obligent l'IA à signaler ce qu'elle ne sait pas. Enfin, la demande de preuves distingue ce qui a été proposé de ce qui a réellement été exécuté.

Application au robot porte-palette. Pour `canStartMotion`, le prompt précise l'état Idle, la batterie, les freins, le capteur frontal et l'erreur critique. Il interdit d'inventer un seuil. Il demande des tests à la valeur frontière et des cas de refus. Le résultat devient révisable parce que le groupe sait exactement quelles règles rechercher.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
« Fais-moi une fonction C++ propre qui vérifie si le robot peut bouger. » L'assistant doit inventer les entrées, les seuils et le comportement d'erreur.

VERSION MIEUX MAÎTRISÉE
« À partir de `RobotState`, retourne un résultat explicite. N'invente aucun seuil. La batterie minimale est 20 inclus. Refuser si frein indisponible, capteur frontal indisponible, erreur critique ou état différent de Idle. Fournir les tests de frontière et négatifs. »

Pièges et nuances à anticiper

- Mettre trop de contexte non pertinent.
- Ne pas préciser ce qui est interdit.
- Demander 'du bon code' sans critères mesurables.
- Oublier de demander à l'outil de signaler les hypothèses manquantes.

Questions au groupe

- Quel bloc du prompt permet de juger la réponse ?
- Quelle information l'IA ne doit jamais inventer ici ?
- Comment réduire le prompt sans perdre les contraintes importantes ?

Mini-activité guidée
Partir d'une demande de deux lignes. Le groupe doit la réécrire en six blocs : contexte, tâche, entrées, contraintes, critères, preuves. Comparer ensuite deux sorties obtenues avec les deux prompts.

Correction et débrief
La correction valorise la précision utile plutôt que la longueur. Chaque contrainte doit pouvoir être reliée à un test ou à une vérification. Les éléments inconnus sont marqués comme questions ouvertes.

À RETENIR : Un prompt technique est utile lorsqu'il rend la sortie vérifiable, pas lorsqu'il paraît sophistiqué.

Approfondissement 5 - Tests : dériver les cas depuis les règles métier

À DIRE TRÈS SIMPLEMENT
On ne teste pas ce que le code fait ; on teste ce que la règle exige que le code fasse.

Explication approfondie. Lorsque l'IA génère à la fois le code et ses tests, elle peut répéter la même mauvaise hypothèse dans les deux. Pour éviter cette boucle, le groupe doit apprendre à dériver les tests avant ou indépendamment de l'implémentation. Une règle de seuil produit au minimum un cas sous le seuil, un cas à la frontière et un cas au-dessus. Une condition de refus produit un scénario où seule cette condition change.

La construction d'une table de décision est un excellent outil pédagogique. On liste les conditions, puis les résultats attendus. Cela rend visibles les combinaisons utiles et empêche de se contenter d'un cas nominal. Les tests doivent aussi couvrir le diagnostic : si plusieurs raisons de refus existent, on vérifie que la bonne raison est retournée selon l'ordre de priorité défini.

Application au robot porte-palette. Batterie à 19, 20 et 21 ; freins vrai/faux ; capteur vrai/faux ; Idle vrai/faux ; erreur critique active/inactive. Le groupe peut commencer avec cinq cas minimaux puis ajouter un cas de combinaison pour discuter la priorité des erreurs.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Tester uniquement `battery=80, brakes=true, sensor=true, idle=true` et conclure que la fonction est correcte.

VERSION MIEUX MAÎTRISÉE
Écrire une matrice de règles, identifier les frontières et cas négatifs, puis vérifier que chaque exigence du ticket est protégée par au moins un test explicite.

Pièges et nuances à anticiper

- Multiplier des tests redondants sans couvrir les frontières.
- Oublier que l'ordre des vérifications peut modifier le motif d'erreur.
- Tester les détails d'implémentation plutôt que le contrat observable.
- Accepter un test parce qu'il passe sans vérifier qu'il pourrait échouer si la règle est cassée.

Questions au groupe

- Quel test échouerait si le seuil devenait 19 par erreur ?
- Comment tester qu'un motif d'erreur est bien celui attendu ?
- Quel cas nominal est indispensable mais insuffisant ?

Mini-activité guidée
Donner uniquement les règles, sans code. Faire écrire une table de cinq à huit scénarios. Ensuite montrer une implémentation fautive et demander quels tests la détectent.

Correction et débrief
La correction doit montrer le lien une exigence -> un ou plusieurs tests. Le cas frontière à 20 est essentiel. Les conditions de refus doivent être isolées autant que possible pour faciliter le diagnostic.

À RETENIR : Des tests utiles sont une traduction des exigences, pas un commentaire automatique de l'implémentation.

Approfondissement 6 - Résultat explicite : sortir du booléen ambigu

À DIRE TRÈS SIMPLEMENT
`true` ou `false` dit ce qui s'est passé ; un résultat explicite aide à comprendre pourquoi.

Explication approfondie. Le booléen est parfois suffisant, mais il devient fragile lorsqu'un appelant doit distinguer plusieurs causes. Dans le fil rouge, la supervision a besoin de savoir si le mouvement est refusé à cause de la batterie, des freins ou d'un capteur. Un type de résultat explicite réduit les chaînes de `if` dispersées et améliore le diagnostic, les tests et la revue.

Une `enum class` décrit un ensemble fermé de motifs. Une structure peut regrouper l'autorisation et le motif. Le formateur doit insister sur le contrat : l'énumération n'est pas créée pour 'faire moderne', mais parce que plusieurs consommateurs ont besoin d'un vocabulaire commun. Le choix des noms doit refléter le métier et rester stable entre les modules.

Application au robot porte-palette. `MotionError::BatteryTooLow` est plus informatif que `false`. La supervision peut afficher un message, le diagnostic peut compter les occurrences et les tests peuvent vérifier précisément la cause. Le module Mission n'a plus à reconstituer la cause en relisant l'état du robot.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Retourner `false` puis demander à chaque appelant de deviner la cause à partir de l'état courant.

VERSION MIEUX MAÎTRISÉE
Retourner un `MotionResult` avec un motif explicite défini dans le contrat partagé et testé aux frontières.

Pièges et nuances à anticiper

- Créer une énumération immense qui mélange causes métier et détails hardware.
- Changer les valeurs ou le sens sans informer les consommateurs.
- Utiliser une chaîne libre lorsque l'ensemble des motifs est fermé et partagé.
- Ajouter un type sans définir l'ordre de priorité quand plusieurs conditions sont fausses.

Questions au groupe

- Qui consomme le motif d'erreur ?
- Pourquoi le motif doit-il faire partie du contrat ?
- Que se passe-t-il si deux conditions de refus sont vraies en même temps ?

Mini-activité guidée
Présenter une fonction booléenne et trois consommateurs. Faire identifier les duplications et les interprétations divergentes. Transformer collectivement le contrat.

Correction et débrief
La version améliorée centralise la décision et son explication. Les consommateurs utilisent le résultat, sans répéter la logique métier. L'équipe documente la priorité des motifs si nécessaire.

À RETENIR : Un contrat explicite réduit l'ambiguïté entre modules et rend la décision plus facile à tester.

Approfondissement 7 - Revue IA : utiliser l'outil comme générateur de questions

À DIRE TRÈS SIMPLEMENT
Une revue IA est utile quand elle attire notre attention sur des risques ; elle ne signe pas la validation.

Explication approfondie. Le principal intérêt d'une revue assistée est d'élargir le regard : cas limites, incohérences de contrat, tests manquants, hypothèses implicites. La faiblesse apparaît lorsque l'on demande une conclusion globale comme 'est-ce bon ?'. L'assistant peut alors produire une réponse rassurante sans avoir accès au contexte complet. On doit donc demander une revue structurée par catégories et exiger la séparation entre faits et hypothèses.

Le protocole peut suivre quatre colonnes : observation, risque, vérification proposée, décision humaine. L'IA remplit les trois premières comme aide. Le reviewer confirme les faits dans le code et la spécification, exécute les contrôles, puis décide. Une suggestion non vérifiable reste une piste, pas un défaut avéré.

Application au robot porte-palette. Pour une modification de mouvement, on demande de rechercher : seuils, unités, cas inconnus, gestion des erreurs, modification de contrat, contournement de Safety, dépendance nouvelle et tests manquants. Le reviewer choisit ensuite quelles alertes sont pertinentes.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
« Review this code and approve it if it looks safe. »

VERSION MIEUX MAÎTRISÉE
« Rechercher les écarts au contrat, les hypothèses non justifiées, les cas limites, les changements d'interface et les tests manquants. Pour chaque point, indiquer ce qui est certain et ce qui doit être vérifié. Ne pas approuver le code. »

Pièges et nuances à anticiper

- Prendre une alerte IA comme un fait sans reproduire le problème.
- Ignorer un risque parce que l'IA ne l'a pas mentionné.
- Laisser la revue dériver vers le style en oubliant le comportement.
- Demander une approbation finale à l'outil.

Questions au groupe

- Quelles catégories de risque doivent être nommées dans le prompt de revue ?
- Comment distinguer une observation factuelle d'une hypothèse ?
- Que faire d'une alerte que l'on ne peut pas reproduire ?

Mini-activité guidée
Donner une revue IA contenant cinq remarques, dont deux fausses et une superficielle. Les participants doivent classer : confirmé, à vérifier, non pertinent, puis justifier la décision.

Correction et débrief
La bonne pratique est de convertir chaque remarque utile en action de vérification. Une remarque non vérifiée ne doit pas être copiée dans la conclusion comme un fait.

À RETENIR : La revue IA augmente le nombre de questions utiles ; la validation reste fondée sur des preuves et une décision humaine.

Approfondissement 8 - Tracabilité : reconstruire le chemin vers la décision

À DIRE TRÈS SIMPLEMENT
Après le changement, on doit pouvoir expliquer ce qui a été demandé, ce qui a été vérifié et pourquoi on a accepté.

Explication approfondie. La traçabilité évite que la connaissance disparaisse après la conversation avec l'assistant. Dans un projet d'équipe, le prompt seul ne suffit pas : il faut conserver les exigences, les tests, les observations de revue et la décision dans les artefacts du projet. Cela facilite les audits internes, la maintenance, l'analyse d'incident et la compréhension par un collègue qui n'était pas présent.

Le support peut faire utiliser une chaîne simple : ticket -> prompt ou consigne -> diff -> tests -> revue -> décision. Chaque étape doit pointer vers des faits observables. L'objectif n'est pas d'archiver toute conversation, mais de conserver les éléments qui justifient le changement. Une trace utile est courte, structurée et liée au dépôt ou au processus de l'équipe.

Application au robot porte-palette. Pour un changement sur `canStartMotion`, la PR indique le ticket, les règles modifiées, les cas testés, les commandes exécutées et les points de revue. Si le seuil est 20, la source de cette valeur doit être identifiable.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
« Ça vient de la réponse Copilot que j'ai eue hier, mais je n'ai plus la conversation. »

VERSION MIEUX MAÎTRISÉE
La règle métier est dans le ticket ou la documentation, le diff montre l'implémentation, les tests protègent la frontière et la PR résume les preuves exécutées.

Pièges et nuances à anticiper

- Confondre traçabilité et accumulation de captures d'écran.
- Conserver une conclusion sans conserver le critère qui la justifie.
- Écrire 'tests OK' sans préciser lesquels.
- Laisser des décisions importantes uniquement dans une conversation privée.

Questions au groupe

- Quel artefact permettra à un collègue de comprendre la décision dans six mois ?
- Quelle information doit rester dans le dépôt plutôt que dans le chat ?
- Comment tracer qu'un test n'a pas été exécuté ?

Mini-activité guidée
À partir d'un changement fictif, demander au groupe de créer une mini-fiche de traçabilité en cinq lignes : besoin, règle, test, revue, décision.

Correction et débrief
Une bonne fiche ne recopie pas le code. Elle relie les éléments de preuve au besoin et indique les limites de la vérification. Si un contrôle n'a pas été exécuté, cela doit être visible.

À RETENIR : La traçabilité conserve la raison de la décision, pas seulement le résultat final.

Études de cas supplémentaires

Ces études de cas servent à faire transférer les règles vues dans le cours. Chaque cas est conçu pour obliger le groupe à justifier une décision plutôt qu'à réciter une définition.

Cas 1 - Seuil de batterie absent du ticket

Une demande précise de générer `canStartMotion()` mais ne fournit aucun seuil de batterie. L’assistant choisit 15 % et produit des tests cohérents avec cette valeur.

Questions d’analyse

- Qu’est-ce qui est connu ?
- Qu’est-ce qui a été inventé ?
- Peut-on accepter le code si tous les tests générés passent ?
- Quelle action demander avant de poursuivre ?

Correction argumentée
Le seuil est une donnée métier manquante. Les tests ne prouvent rien sur la bonne valeur puisqu’ils reprennent la même hypothèse. Il faut arrêter la décision, signaler l’information manquante et obtenir la règle depuis la source autorisée.

Cas 2 - Capteur frontal indisponible mais diagnostic vert

Le module de diagnostic indique « OK » alors que la donnée du capteur frontal est périmée depuis plusieurs secondes. La fonction de mouvement ne vérifie que la valeur booléenne.

Questions d’analyse

- Quelle différence entre valeur et fraîcheur de la donnée ?
- Quel contrat manque entre diagnostic et décision ?
- Quel test négatif ajouter ?

Correction argumentée
Le problème est un contrat incomplet : une valeur vraie ne garantit pas qu’elle soit valide ou récente. L’exercice ne demande pas d’inventer une solution temps réel ; il faut identifier l’information manquante et faire expliciter le contrat de disponibilité.

Cas 3 - Prompt trop large sur un module rouge

Le développeur demande à l'IA de « refactorer toute la couche Safety pour la rendre plus propre » sans ticket détaillé ni critères d'acceptation.

Questions d'analyse

- Pourquoi le périmètre est-il dangereux ?
- Comment réduire le changement ?
- Quelles preuves seraient obligatoires ?

Correction argumentée
Sur une zone critique, on réduit le scope, on rattache chaque modification à une exigence et on augmente les contrôles. Le besoin de « propreté » ne justifie pas une refonte globale.

Cas 4 - Revue rassurante mais incomplète

L'IA conclut que le code est « safe and production ready » après avoir vérifié le style et la présence de trois tests.

Questions d'analyse

- Quelle affirmation dépasse les preuves ?
- Quelles catégories de risques n'ont pas été vérifiées ?
- Quelle formulation demander à la place ?

Correction argumentée
La conclusion est trop forte. Une revue utile doit lister les faits, les limites et les points à vérifier. Le reviewer humain demeure responsable de la décision selon le processus.

Cas 5 - Copier un log de production

Pour expliquer une erreur, un participant veut coller un log complet avec noms de machines, chemins, identifiants et contexte client.

Questions d’analyse

- Que peut-on réduire ?
- Quelles données sont nécessaires au diagnostic ?
- Quelle politique doit être consultée ?

Correction argumentée
Le bon réflexe est de minimiser et anonymiser selon les règles internes, ou de ne pas utiliser l’assistant si le partage n’est pas autorisé. Le contenu utile peut souvent être reproduit par un exemple synthétique.

Cas 6 - Tests tous verts après une mauvaise priorité d’erreur

Deux conditions sont fausses à la fois. Le code renvoie BatteryTooLow alors que l’architecture veut prioriser CriticalError, mais aucun test de combinaison n’existe.

Questions d’analyse

- Pourquoi les tests unitaires isolés n’ont-ils pas suffi ?
- Où documenter la priorité ?
- Quel test ajouter ?

Correction argumentée
Il faut expliciter la politique de priorité dans le contrat et ajouter au moins un cas de combinaison significatif. La règle ne doit pas rester implicite dans l’ordre des `if`.

Ateliers longs et corrections

Atelier 1 - Construire une charte IA réellement opérationnelle

Contexte
L'équipe utilise déjà un assistant mais les pratiques diffèrent selon les développeurs.

Consigne
Rédiger six règles internes qui couvrent confidentialité, criticité, prompts, validation, traçabilité et revue. Chaque règle doit indiquer une action observable, pas seulement une intention.

Déroulement formateur

- Former des binômes et attribuer un thème à chacun.
- Faire écrire une première règle en langage naturel.
- Demander : comment prouver que cette règle est suivie ?
- Réécrire toute règle non vérifiable.
- Assembler les six règles et tester leur cohérence sur deux scénarios.

Correction détaillée
Une règle utile possède un déclencheur et une action. Exemple : « Avant de transmettre du code à l'assistant, vérifier la classification de l'information et utiliser uniquement l'outil autorisé ». À éviter : « Être prudent avec les données ». La correction doit conserver la prudence sans prétendre définir une politique d'entreprise réelle.

Critères de réussite

- Les six thèmes sont couverts.
- Chaque règle est formulée en action observable.
- Aucune règle ne délègue la validation finale à l'IA.
- Le niveau de contrôle varie avec la criticité.
- La traçabilité est explicitement mentionnée.

Atelier 2 - Du besoin vague au code testable

Contexte
Le ticket dit uniquement : « empêcher le robot d’avancer quand ce n’est pas sûr ».

Consigne
Produire une liste de questions, un prompt technique, une table de décision et une série de tests avant d’écrire l’implémentation.

Déroulement formateur

- Interdire de commencer par le code pendant dix minutes.
- Collecter les questions de clarification.
- Fournir ensuite les règles connues : Idle, batterie >=20, freins, capteur, absence d’erreur critique.
- Faire construire une table de décision.
- Seulement ensuite proposer le prompt et le code.

Correction détaillée
Le point clé est de montrer que la qualité du code dépend d’abord de la qualité du contrat. Les tests doivent couvrir la frontière 19/20/21, chaque refus isolé et au moins un cas combiné discutant la priorité.

Critères de réussite

- Les hypothèses sont distinguées des faits.
- Le seuil n’est pas inventé.
- Les critères d’acceptation sont vérifiables.
- Les tests sont dérivés des règles.
- La sortie d’IA est présentée comme proposition, pas comme preuve.

Tableaux de référence complémentaires

Matrice de profondeur de contrôle selon la criticité

Niveau	Type de changement	Contrôles minimum	Revue attendue
Vert	Documentation, formatage, outil local non décisionnel	Lecture + test simple si pertinent	Auteur ou pair selon politique
Orange	Logique métier, contrat partagé, diagnostic	Tests nominaux et négatifs, revue d'interface	Pair technique
Rouge	Décision de mouvement, Safety, interaction physique	Spécification explicite, frontières, tests renforcés, revue dédiée	Validation humaine renforcée selon processus

Checklist de prompt technique

Bloc	Question à poser
Contexte	Dans quel module, version et scénario travaillons-nous ?
Tâche	Quel changement précis est demandé ?
Entrées	Quelles données sont disponibles et dans quelles unités ?
Contraintes	Qu'est-ce qui est interdit ou imposé ?
Critères	Comment saurons-nous que le résultat est correct ?
Hypothèses	Quelles informations manquent et doivent être signalées ?
Preuves	Quelles commandes/tests pourront réellement être exécutés ?

Démonstrations et code complémentaires

Démonstration - table de décision avant le code

Ce fragment montre comment rendre la priorité de décision lisible sans dépendre d'une bibliothèque externe.

```
#include <cassert>
#include <iostream>

enum class MotionError { None, CriticalError, NotIdle, BatteryTooLow, BrakesUnavailable, SensorUnavailable };
struct RobotState { bool criticalError; bool idle; int battery; bool brakes; bool sensor; };
struct MotionResult { bool allowed; MotionError error; };

MotionResult evaluate(const RobotState& s) {
    if (s.criticalError) return {false, MotionError::CriticalError};
    if (!s.idle) return {false, MotionError::NotIdle};
    if (s.battery < 20) return {false, MotionError::BatteryTooLow};
    if (!s.brakes) return {false, MotionError::BrakesUnavailable};
    if (!s.sensor) return {false, MotionError::SensorUnavailable};
    return {true, MotionError::None};
}

int main() {
    assert(evaluate({false,true,20,true,true}).allowed);
    assert(evaluate({false,true,19,true,true}).error == MotionError::BatteryTooLow);
    assert(evaluate({true,true,80,true,true}).error == MotionError::CriticalError);
    std::cout << "decision tests passed\n";
}
```

Points à commenter

- La valeur frontière 20 doit être expliquée avant l'exécution.
- L'ordre des `if` matérialise une priorité : elle doit venir du contrat, pas du hasard.
- Les assertions vérifient des comportements observables.
- Le code reste pédagogique ; il ne prétend pas modéliser tout le système robotique.

Banque de questions de vérification

1. Qu'est-ce qui pourrait être faux même si le code compile ?

Réponse attendue : Une sortie plausible est une proposition ; la preuve vient de critères et de contrôles indépendants. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

2. Quelle preuve vous ferait changer d'avis sur cette proposition ?

Réponse attendue : Une sortie plausible est une proposition ; la preuve vient de critères et de contrôles indépendants. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

3. Pourquoi deux fonctions de dix lignes peuvent-elles avoir des exigences de revue différentes ?

Réponse attendue : La criticité ne décrit pas la complexité du code ; elle décrit le niveau de prudence imposé par les conséquences possibles. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

4. Quel critère ferait passer une fonction orange en rouge ?

Réponse attendue : La criticité ne décrit pas la complexité du code ; elle décrit le niveau de prudence imposé par les conséquences possibles. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

5. Quelle information du contexte est réellement nécessaire à la tâche ?

Réponse attendue : Le bon prompt commence par la bonne décision de partage de l'information. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

6. Comment reproduire le problème sans transmettre le fichier complet ?

Réponse attendue : Le bon prompt commence par la bonne décision de partage de l'information. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

7. Quel bloc du prompt permet de juger la réponse ?

Réponse attendue : Un prompt technique est utile lorsqu'il rend la sortie vérifiable, pas lorsqu'il paraît sophistiqué. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

8. Quelle information l'IA ne doit jamais inventer ici ?

Réponse attendue : Un prompt technique est utile lorsqu'il rend la sortie vérifiable, pas lorsqu'il paraît sophistiqué. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

9. Quel test échouerait si le seuil devenait 19 par erreur ?

Réponse attendue : Des tests utiles sont une traduction des exigences, pas un commentaire automatique de l'implémentation. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

10. Comment tester qu'un motif d'erreur est bien celui attendu ?

Réponse attendue : Des tests utiles sont une traduction des exigences, pas un commentaire automatique de l'implémentation. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

11. Qui consomme le motif d'erreur ?

Réponse attendue : Un contrat explicite réduit l'ambiguïté entre modules et rend la décision plus facile à tester. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

12. Pourquoi le motif doit-il faire partie du contrat ?

Réponse attendue : Un contrat explicite réduit l'ambiguïté entre modules et rend la décision plus facile à tester. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

13. Quelles catégories de risque doivent être nommées dans le prompt de revue ?

Réponse attendue : La revue IA augmente le nombre de questions utiles ; la validation reste fondée sur des preuves et une décision humaine. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

14. Comment distinguer une observation factuelle d'une hypothèse ?

Réponse attendue : La revue IA augmente le nombre de questions utiles ; la validation reste fondée sur des preuves et une décision humaine. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

15. Quel artefact permettra à un collègue de comprendre la décision dans six mois ?

Réponse attendue : La traçabilité conserve la raison de la décision, pas seulement le résultat final. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

16. Quelle information doit rester dans le dépôt plutôt que dans le chat ?

Réponse attendue : La traçabilité conserve la raison de la décision, pas seulement le résultat final. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

Synthèse générale de la journée - version approfondie

La compétence recherchée n'est pas de connaître une liste de règles par cœur. Le participant doit être capable de regarder une situation nouvelle et de reconstruire le raisonnement : identifier le risque, expliciter le contrat, cadrer la demande, obtenir une proposition, organiser les preuves, puis décider. Le fil rouge du robot permet de rendre ce raisonnement concret, mais la méthode doit pouvoir être transférée à d'autres systèmes C++ industriels.

- Une sortie plausible est une proposition ; la preuve vient de critères et de contrôles indépendants.
- La criticité ne décrit pas la complexité du code ; elle décrit le niveau de prudence imposé par les conséquences possibles.
- Le bon prompt commence par la bonne décision de partage de l'information.
- Un prompt technique est utile lorsqu'il rend la sortie vérifiable, pas lorsqu'il paraît sophistiqué.
- Des tests utiles sont une traduction des exigences, pas un commentaire automatique de l'implémentation.
- Un contrat explicite réduit l'ambiguïté entre modules et rend la décision plus facile à tester.
- La revue IA augmente le nombre de questions utiles ; la validation reste fondée sur des preuves et une décision humaine.
- La traçabilité conserve la raison de la décision, pas seulement le résultat final.

Annexe approfondie 1 - Guide d’animation par séquence

Ce guide aide le formateur à transformer le support en rythme de salle. Pour chaque séquence, la logique reste la même : faire émerger le problème, introduire la notion seulement ensuite, faire produire une décision par les participants, puis terminer par une preuve ou un artefact réutilisable.

Séquence	Ouverture conseillée	Activité centrale	Trace de fin
09:00-10:00 responsabilité et IA	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
10:00-11:15 criticité	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
11:15-12:30 règles internes	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
13:30-15:00 prompt technique	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
15:00-16:30 code et tests	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
16:30-17:30 double revue	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite

Conseil d’animation : ne pas donner immédiatement la réponse. Laisser d’abord le groupe formuler ce qu’il faudrait vérifier. Lorsque plusieurs réponses sont possibles, demander quel fait permettrait de trancher. Cette habitude construit le réflexe central de la formation : ne pas remplacer une incertitude par une affirmation plausible.

- Faire reformuler la règle à la fin de chaque bloc.
- Conserver au tableau les décisions qui seront réutilisées plus tard dans la journée.
- Faire expliciter la différence entre une hypothèse, un fait observé et une décision.
- Demander régulièrement : « quelle preuve avons-nous réellement ? »
- Si le groupe va trop vite vers le code, revenir au contrat et aux critères d’acceptation.

Annexe approfondie 2 - FAQ formateur et réponses de secours

Question 1. Qu'est-ce qui pourrait être faux même si le code compile ?

Réponse courte : Une sortie plausible est une proposition ; la preuve vient de critères et de contrôles indépendants.

Réponse développée : Dans un atelier, les participants ont souvent tendance à évaluer une réponse avec trois signaux trompeurs : elle est rapide, elle est bien écrite et elle ressemble à du code qu'ils connaissent. Aucun de ces signaux ne prouve que les exigences métier sont respectées. Il faut donc séparer la qualité de présentation de la qualité de preuve. Une proposition devient exploitable lorsqu'on peut la confronter à une spécification, à des tests et à des contraintes connues. Le formateur peut ensuite revenir sur le fil rouge : Demander à l'IA si le robot peut avancer avec 18 % de batterie n'a pas de sens tant que le seuil n'est pas connu. Si l'outil invente 20 %, la réponse peut sembler raisonnable mais reste une hypothèse. Le groupe doit apprendre à demander : d'où vient le seuil, quelle règle le fixe et quel test montre le comportement à la frontière ?

Question 2. Pourquoi deux fonctions de dix lignes peuvent-elles avoir des exigences de revue différentes ?

Réponse courte : La criticité ne décrit pas la complexité du code ; elle décrit le niveau de prudence imposé par les conséquences possibles.

Réponse développée : Le classement vert, orange et rouge n'a d'intérêt que s'il modifie concrètement le processus de travail. Il ne s'agit pas de coller une couleur sur un fichier, mais de réfléchir aux conséquences d'une erreur, à la détectabilité du défaut et à la possibilité de revenir en arrière. Deux fonctions de taille identique peuvent exiger des niveaux de revue très différents selon leur rôle dans le système. Le formateur peut ensuite revenir sur le fil rouge : Une fonction qui reformate un message de diagnostic peut être traitée plus légèrement qu'une fonction qui autorise un mouvement. Si la première se trompe, l'opérateur voit peut-être un texte imparfait. Si la seconde se trompe, l'action physique peut être inappropriée. La couleur vient de cette différence d'impact, pas du langage C++ utilisé.

Question 3. Quelle information du contexte est réellement nécessaire à la tâche ?

Réponse courte : Le bon prompt commence par la bonne décision de partage de l'information.

Réponse développée : Dans un usage professionnel, la confidentialité ne peut pas être réduite à 'ne pas coller de secret'. Un extrait de code peut révéler une architecture, des noms de clients, des identifiants, une logique métier propriétaire ou des détails d'infrastructure. Le réflexe doit donc précéder la qualité du prompt : on qualifie d'abord l'information, puis on décide ce qui peut être fourni à l'outil autorisé. Le formateur peut ensuite revenir sur le fil rouge : Au lieu de copier un fichier contenant des adresses réseau, des identifiants de site et des journaux d'exploitation, on construit un exemple réduit : état du frein, disponibilité du capteur, niveau de batterie. On conserve le problème technique tout en retirant les informations inutiles à la tâche.

Question 4. Quel bloc du prompt permet de juger la réponse ?

Réponse courte : Un prompt technique est utile lorsqu'il rend la sortie vérifiable, pas lorsqu'il paraît sophistiqué.

Réponse développée : Une demande vague force l'assistant à combler les blancs. Plus le domaine est contraint, plus ces blancs deviennent dangereux. Le prompt technique doit donc réduire l'ambiguïté sans devenir un roman. Il décrit l'objectif, la zone de code concernée, les règles métier connues, les interdits, le format de sortie et les critères d'acceptation. Cette structure améliore aussi le travail humain : elle rend le besoin discutable avant la génération. Le formateur peut ensuite revenir sur le fil rouge : Pour `canStartMotion`, le prompt précise l'état Idle, la batterie, les freins, le capteur frontal et l'erreur critique. Il interdit d'inventer un seuil. Il demande des tests à la valeur frontière et des cas de refus. Le résultat devient révisable parce que le groupe sait exactement quelles règles rechercher.

Question 5. Quel test échouerait si le seuil devenait 19 par erreur ?

Réponse courte : Des tests utiles sont une traduction des exigences, pas un commentaire automatique de l'implémentation.

Réponse développée : Lorsque l'IA génère à la fois le code et ses tests, elle peut répéter la même mauvaise hypothèse dans les deux. Pour éviter cette boucle, le groupe doit apprendre à dériver les tests avant ou indépendamment de l'implémentation. Une règle de seuil produit au minimum un cas sous le seuil, un cas à la frontière et un cas au-dessus. Une condition de refus produit un scénario où seule cette condition change. Le formateur peut ensuite revenir sur le fil rouge : Batterie à 19, 20 et 21 ; freins vrai/faux ; capteur vrai/faux ; Idle vrai/faux ; erreur critique active/inactive. Le groupe peut commencer avec cinq cas minimaux puis ajouter un cas de combinaison pour discuter la priorité des erreurs.

Question 6. Qui consomme le motif d'erreur ?

Réponse courte : Un contrat explicite réduit l'ambiguïté entre modules et rend la décision plus facile à tester.

Réponse développée : Le booléen est parfois suffisant, mais il devient fragile lorsqu'un appelant doit distinguer plusieurs causes. Dans le fil rouge, la supervision a besoin de savoir si le mouvement est refusé à cause de la batterie, des freins ou d'un capteur. Un type de résultat explicite réduit les chaînes de `if` dispersées et améliore le diagnostic, les tests et la revue. Le formateur peut ensuite revenir sur le fil rouge : `MotionError::BatteryTooLow` est plus informatif que `false`. La

supervision peut afficher un message, le diagnostic peut compter les occurrences et les tests peuvent vérifier précisément la cause. Le module Mission n'a plus à reconstituer la cause en relisant l'état du robot.

Si une question dépasse le périmètre de la journée, ne pas improviser une architecture ou une règle future. La bonne réponse pédagogique consiste à reconnaître le périmètre, reformuler ce qui est déjà établi et indiquer clairement ce qui doit être confirmé par la politique, l’architecture ou les spécialistes de l’entreprise.

Annexe approfondie 3 - Grille de diagnostic d’une production participant

Dimension	Question de contrôle	Signal satisfaisant	Signal d’alerte
Besoin	Le participant sait-il expliquer le problème avant la solution ?	Objectif et contraintes reformulés	Code produit sans besoin explicite
Hypothèses	Les inconnues sont-elles signalées ?	Questions ouvertes visibles	Valeurs inventées ou implicites
Contrat	Entrées, sorties et erreurs sont-elles compréhensibles ?	Comportement testable	Booléens / nombres ambigus
Preuves	Les tests ou contrôles sont-ils reliés aux exigences ?	Frontières et cas négatifs couverts	Seulement un cas nominal
IA	Le rôle de l’assistant est-il correctement limité ?	Proposition / aide à la revue	Validation déléguée à l’outil
Décision	La conclusion est-elle argumentée ?	Acceptation, modification ou refus justifié	« ça marche » sans critères
Traçabilité	Peut-on retrouver les éléments importants ?	Ticket, tests, revue, décision reliés	Connaissance uniquement dans le chat

Cette grille n’est pas un barème obligatoire. Elle sert à diagnostiquer rapidement où se situe la difficulté : technique, contrat, preuve ou raisonnement. Elle permet aussi d’éviter de sur-noter une solution qui compile mais dont le processus reste fragile.

Annexe approfondie 4 - Variantes selon le niveau du groupe

Situation du groupe	Adaptation	Ce qu'il ne faut pas supprimer
Groupe peu expérimenté	Réduire la taille du code, garder un seul changement à la fois, verbaliser chaque terme	La distinction proposition / preuve / décision
Groupe expérimenté	Ajouter des ambiguïtés de contrat et demander plusieurs solutions	La justification des choix et les preuves
Groupe très orienté outils	Commencer par le problème avant d'ouvrir Copilot ou VS Code	Le pourquoi métier et le rôle humain
Pas d'accès réseau	Utiliser des sorties IA préparées à l'avance et simuler la revue	L'analyse critique et les contrôles déterministes
Temps réduit	Supprimer un cas secondaire, pas les étapes de raisonnement	Une activité pratique + correction + synthèse
Groupe hétérogène	Binômes mélangeant niveaux, rôles alternés auteur/reviewer	La participation de chacun et la verbalisation

Une adaptation de niveau ne doit pas transformer la formation en démonstration passive. Même avec des débutants, les participants doivent prendre des décisions simples et expliquer leur raisonnement. Avec des experts, on augmente surtout l'ambiguïté des cas et la profondeur de justification plutôt que d'ajouter des outils hors programme.

- Version courte : une seule erreur injectée à la fois.
- Version avancée : deux contraintes contradictoires à détecter.
- Version atelier : un binôme produit, un autre révise avec la checklist.
- Version paperboard : aucune machine pendant dix minutes, uniquement contrat et tests attendus.

Annexe approfondie 5 - Plan B et scénarios de secours pour le formateur

Une formation centrée sur des outils peut être fragilisée par un problème de compte, de réseau, de licence, d'extension ou de configuration. Le contenu pédagogique doit rester exécutable même si l'assistant ou la plateforme n'est pas disponible. Les scénarios ci-dessous permettent de continuer sans changer les objectifs de la journée.

Incident	Plan B pédagogique	Objectif conservé
Copilot indisponible	Projeter deux réponses IA préparées, dont une volontairement imparfaite	Évaluer et revoir une proposition
GitHub indisponible	Utiliser un diff local et une fiche PR papier	Traçabilité, impacts et décision
CI indisponible	Lire un log de workflow préparé avec un check vert et un check rouge	Interpréter un contrôle automatisé
Compilateur indisponible	Faire prédire les résultats et analyser les tests sur papier	Relier règles, code et preuves
Temps de build trop long	Utiliser la mini-démo autonome du support	Conserver la logique de contrôle
Question hors politique interne	Revenir au principe générique et signaler la validation nécessaire	Ne pas inventer une règle d'entreprise

Préparer ces éléments avant le cours évite de transformer un incident d'outillage en perte pédagogique. Le fil conducteur doit rester le même : une proposition est analysée, les contraintes sont explicitées, des preuves sont recherchées et la décision finale reste attribuée au bon rôle.

CHECKLIST AVANT DE QUITTER LA SALLE
Le groupe peut expliquer au moins trois règles de la journée avec ses propres mots ; il peut citer une erreur typique ; il sait donner un exemple de preuve ; il sait dire ce qui reste une décision humaine ; et il a produit au moins un artefact réutilisable dans la suite de la formation.

SUPPORT FORMATEUR ENRICHI

J2

Synchronisation d’équipe, maintenabilité et réutilisation du code

AI Technical Sync, contrats d’interface, règles projet Copilot et Reusable Candidate

FIL ROUGE

Robot porte-palette : le J2 suit plusieurs développeurs travaillant sur des modules interdépendants. L’objectif est de réduire le travail en silo, stabiliser les contrats, améliorer la maintenabilité et décider rationnellement ce qui mérite d’être réutilisé.

DURÉE

7 h 30 de formation - 09:00-12:30 et 13:30-17:30.

Document de préparation et d’animation - version dense et détaillée

0. Comment utiliser ce support

Ce document est le support formateur enrichi du J2. Il développe le déroulé validé sans le transformer en simple liste de bonnes pratiques. Chaque séquence part d'un problème concret d'équipe, explique pourquoi il apparaît, montre ses conséquences dans un projet C++ industriel puis propose une méthode de travail vérifiable.

IDÉE DIRECTRICE DU J2

Une équipe n'est pas maintenable parce que chaque développeur écrit du bon code dans son coin. Elle devient maintenable lorsque les contrats sont explicites, les changements sont visibles, les dépendances sont comprises et les composants réellement réutilisables sont extraits sans créer de couplage inutile.

0.1 Progression pédagogique du J2

Horaire	Séquence	Question centrale	Production
09:00-10:00	Le problème du travail en silo	Pourquoi de bons modules peuvent-ils produire une mauvaise intégration ?	Carte des risques de synchronisation
10:00-11:30	Process "AI Technical Sync"	Comment rendre visibles les changements et leurs impacts avant l'intégration ?	Plan de synchronisation + fiche d'impact
11:30-12:30	Standardiser les interfaces	Comment rendre un contrat stable, lisible et testable ?	Contrat entrée / sortie / erreur
13:30-14:30	Règles de maintenabilité	Quelles règles diminuent le coût du changement ?	Checklist de maintenabilité
14:30-15:30	Configurer Copilot avec les règles du projet	Comment donner à l'assistant un contexte commun au dépôt ?	.github/copilot-instructions.md
15:30-16:45	Réutilisation du code	Quand un doublon doit-il devenir une politique ou une librairie interne ?	Refactoring guidé
16:45-17:30	Process "Reusable Candidate"	Comment décider d'extraire, nommer, documenter, versionner et tester ?	Fiche de décision réutilisable

0.2 Vocabulaire du jour à introduire progressivement

Silo - Situation dans laquelle une personne ou une équipe avance avec peu de visibilité sur les changements des autres modules.

Contrat - Ce qu'un composant promet à ses consommateurs : entrées, sorties, erreurs, comportement et hypothèses utiles.

Couplage - Degré de dépendance entre deux éléments. Plus une modification locale force des modifications ailleurs, plus le couplage est fort.

Cohésion - Degré avec lequel les éléments d'un module contribuent à une même responsabilité.

Interface stable - Contrat qui évolue de manière maîtrisée. Stable ne signifie pas immuable.

Reusable Candidate - Élément repéré comme candidat à une extraction réutilisable, mais qui doit encore passer des critères de décision.

0.3 Point de cohérence à signaler au formateur

NOM DU FICHIER COPILOT

Le déroulé indique explicitement `.github/copilot-instructions.md` dans le contenu de la séquence, puis mentionne `githuib-copilot.md` dans la ligne d'atelier. Le présent support conserve `.github/copilot-instructions.md`, qui est le nom donné dans le contenu principal, et signale ici cette divergence au lieu de la corriger silencieusement.

09:00-10:00 - Le problème du travail en silo

OBJECTIF DE LA SÉQUENCE

Faire comprendre que les défauts d'intégration apparaissent souvent aux frontières entre modules, même lorsque chaque module semble correct isolément.

Le déroulé ouvre J2 par cinq symptômes : chacun avance sur sa partie, les interfaces changent sans coordination, les doublons apparaissent, l'intégration arrive trop tard et les bugs transverses sont découverts en fin de sprint. Il faut transformer cette liste en chaîne de causes et de conséquences.

1.1 Qu'est-ce qu'un silo dans une équipe de développement ?

Un silo n'est pas simplement le fait que les développeurs travaillent sur des fichiers différents. La division du travail est normale. Le silo apparaît lorsque l'information nécessaire aux autres n'est pas transmise au bon moment : modification d'une signature, changement de sémantique, nouveau code d'erreur, hypothèse sur une unité, dépendance ajoutée, délai modifié, ou règle métier dupliquée.

À DIRE TRÈS SIMPLEMENT

Le problème n'est pas : chacun a son module. Le problème est : chacun change son module comme si les autres n'en dépendaient pas.

1.2 Exemple fil rouge : quatre développeurs, quatre visions partielles

Développeur / module	Décision locale	Pourquoi elle semble raisonnable	Risque transversal
Navigation	Renomme startMove() en requestMotion().	Nom plus précis localement.	Mission ne compile plus ou adapte trop tard.
Mission	Considère une batterie > 20 comme suffisante.	Règle recopiée depuis un ancien ticket.	Divergence avec la politique commune >= 20.
Hardware	Retourne false si le bus ne répond pas.	API minimale et simple.	Impossible de distinguer timeout, refus matériel et indisponibilité.

Développeur / module	Décision locale	Pourquoi elle semble raisonnable	Risque transversal
Supervision	Traduit plusieurs erreurs en “MOTION_FAILED”.	Affichage simplifié.	Perte du diagnostic nécessaire à l’intégration.

1.3 Les cinq symptômes du déroulé transformés en chaîne causale

Symptôme	Cause fréquente	Conséquence visible	Question de prévention
Chaque développeur avance sur sa partie	Vision locale du besoin et peu de points de synchronisation.	Décisions incompatibles prises en parallèle.	Qui consomme ce que je change ?
Interfaces changent sans coordination	Le changement est vu comme un détail interne.	Compilation cassée ou pire : comportement subtilement différent.	Quels consommateurs et quels tests sont impactés ?
Doublons apparaissent	La même règle est réécrite faute de composant commun identifié.	Règles qui divergent dans le temps.	Cette logique représente-t-elle une politique commune ?
Intégration arrive trop tard	On attend la fin pour assembler.	Beaucoup de problèmes simultanés et diagnostic difficile.	Quel est le plus petit point d’intégration testable cette semaine ?
Bugs transverses en fin de sprint	Tests principalement locaux.	Les frontières entre modules restent peu exercées.	Quel scénario traverse plusieurs composants ?

1.4 Le bug transverse : ni “le bug de A” ni “le bug de B”

Un bug transverse apparaît lorsque deux composants respectent chacun leur compréhension locale mais ne partagent pas la même interprétation du contrat. Exemple : Navigation interprète un timeout à 500 ms comme un échec définitif ; Mission considère qu’un timeout signifie “réessayer”. Les deux codes peuvent être propres, testés et cohérents localement. L’assemblage produit pourtant un comportement indésirable.

Schéma à dessiner au tableau

Étape	À dessiner	Question au groupe
1	Trois boîtes : Mission -> Navigation -> Hardware.	Où se trouve le code de chaque équipe ?
2	Sur chaque flèche : entrée / sortie / erreur.	Qu’est-ce qui doit être partagé explicitement ?
3	Ajouter une modification rouge sur Navigation.	Qui doit être informé avant le merge ?
4	Ajouter Supervision qui observe le résultat.	Une modification d’erreur touche-t-elle seulement l’appelant direct ?

1.5 Exercice diagnostic : “ça marche chez moi”

CONSIGNE

Par groupes de deux, analysez le scénario suivant : Mission envoie une commande avec un identifiant de mission ; Navigation accepte la commande ; Hardware signale parfois un timeout ; Supervision n’affiche qu’un booléen succès/échec. Identifiez au moins quatre informations manquantes qui rendent l’intégration fragile.

Correction attendue

- Sémantique du timeout : erreur définitive, état indéterminé ou possibilité de retry ?
- Contrat d’erreur : booléen trop pauvre pour expliquer la cause.
- Traçabilité de l’identifiant de mission à travers les modules.
- Règle sur le nombre ou la stratégie de tentatives éventuelles.
- Responsable de la traduction technique -> métier -> supervision.
- Tests d’intégration qui traversent Mission, Navigation et Hardware.

1.6 À retenir

RÈGLE

Le risque d’équipe se concentre souvent aux frontières : changement d’interface, changement de sens, duplication d’une règle et intégration tardive. La solution du J2 sera de rendre ces frontières explicites et visibles.

10:00-11:30 - Process “AI Technical Sync”

OBJECTIF DE LA SÉQUENCE

Construire un rituel léger de synchronisation où l’IA aide à résumer et comparer, sans devenir une autorité technique.

Le process du déroulé contient six briques : fiche d’impact module, résumé IA des changements, détection des conflits, points d’intégration, documentation automatique et synthèse hebdomadaire. L’objectif n’est pas de créer une réunion supplémentaire, mais de raccourcir le délai entre un changement et sa visibilité pour les autres.

2.1 Pourquoi ajouter un process au lieu de “mieux communiquer” ?

“Communiquez davantage” est une intention, pas un mécanisme. Un process utile doit préciser ce qui déclenche l’échange, quelles informations sont attendues, qui les lit, quelle décision en sort et où la trace est conservée. L’IA peut accélérer la synthèse, mais les faits de départ doivent venir du dépôt, du ticket et des développeurs.

2.2 Vue d'ensemble du process

Étape	Entrée	Usage possible de l'IA	Sortie vérifiable
1. Fiche d'impact	PR, ticket, fichiers modifiés, contrats touchés.	Aider à reformuler et structurer.	Impacts déclarés par le développeur.
2. Résumé des changements	Diff + fiche d'impact.	Produire un résumé court orienté consommateurs.	Résumé relu et corrigé.
3. Détection des conflits	Résumés de plusieurs modules.	Comparer noms, types, erreurs, hypothèses et versions.	Liste de conflits potentiels à confirmer.
4. Points d'intégration	Dépendances et flux.	Suggérer les scénarios transverses à vérifier.	Plan d'intégration avec responsables.
5. Documentation	Contrat validé et décisions.	Mettre à jour un brouillon de documentation.	Documentation relue.
6. Synthèse hebdomadaire	Fiches et décisions de la semaine.	Condensation et regroupement.	Liste d'actions, risques, décisions ouvertes.

2.3 La fiche d'impact module

La fiche d'impact est courte. Son rôle est d'obliger l'auteur d'un changement à regarder au-delà du fichier qu'il modifie. Elle doit pouvoir être remplie en quelques minutes et servir de matière première à la synchronisation.

Champ	Question à laquelle répondre	Exemple
Module	Où se situe le changement ?	Navigation
But	Pourquoi change-t-on ?	Distinguer timeout et refus matériel.
Interface touchée	Quel contrat public évolue ?	startMove() retourne maintenant MotionResult.
Consommateurs	Qui dépend directement ou indirectement ?	Mission, Supervision, tests d'intégration.
Compatibilité	Ancien code continue-t-il à fonctionner ?	Non : type de retour modifié.
Erreurs / états	Nouveaux codes ou nouvelles significations ?	Timeout, HardwareRejected.
Tests	Quels scénarios prouvent la modification ?	Succès, timeout, refus.
Migration	Que faut-il adapter ailleurs ?	Mapper les nouveaux codes dans Mission et Supervision.
Responsable	Qui coordonne le changement ?	Développeur Navigation.

2.4 Un résumé IA utile n'est pas un résumé de code ligne par ligne

FRAGILE	MIEUX
"Résume ce diff." Résultat : liste de lignes modifiées, peu de contexte sur les consommateurs et les ruptures possibles.	"À partir du diff et de la fiche d'impact, résume les changements de contrat, les consommateurs affectés, les nouvelles erreurs et les actions d'intégration à prévoir. N'invente aucune dépendance non visible."

Le résumé doit servir à une autre équipe. Il est donc orienté impact : ce qui change, pour qui, comment vérifier et ce qui reste incertain. Une belle description de l'algorithme interne peut être inutile si elle ne dit pas que le type de retour a changé.

2.5 Catégories de conflits à rechercher

Conflit potentiel	Exemple	Pourquoi l'IA peut aider	Pourquoi l'humain doit confirmer
Nom / signature	requestMotion(int) vs requestMotion(MissionId).	Comparer rapidement les déclarations.	Le choix dépend du contrat voulu.
Type / unité	mm vs m, ms vs s.	Repérer des noms ou conversions incohérents.	L'unité réelle doit venir de la spécification.
Sémantique d'erreur	false = timeout dans un module, refus dans un autre.	Mettre en évidence des descriptions incompatibles.	L'interprétation métier doit être décidée.
Règle dupliquée	Seuil batterie 20 dans deux fichiers avec comparateurs différents.	Détecter des motifs proches.	Une similarité n'implique pas automatiquement une abstraction.
Ordre / protocole	init -> configure -> start attendu par Hardware.	Comparer des séquences d'appel documentées.	Le protocole réel doit être testé.

2.6 Les points d'intégration

Un point d'intégration est un endroit précis où deux ou plusieurs modules doivent être assemblés et testés. Le but est de transformer "on intégrera à la fin" en petits rendez-vous techniques observables.

1. Lister les frontières modifiées.
2. Nommer un scénario minimal de bout en bout pour chaque frontière.
3. Choisir le responsable de la préparation des données ou du mock nécessaire.
4. Définir le résultat attendu et les erreurs attendues.
5. Exécuter le test avant la fin du sprint, pas seulement au moment du merge final.

2.7 Documentation automatique : brouillon, pas vérité

L'IA peut produire un premier brouillon de notes de changement, de documentation d'interface ou de synthèse. La règle à conserver est la même que J1 : la documentation générée doit être relue à partir du contrat réel. Une documentation fausse est parfois plus dangereuse qu'une documentation absente, car elle donne une confiance injustifiée.

2.8 Synthèse hebdomadaire

Rubrique	Contenu attendu
Interfaces modifiées	Liste courte + statut : compatible / rupture / en discussion.
Conflits détectés	Conflit, modules concernés, décision ou propriétaire.
Points d'intégration	Scénarios exécutés et scénarios encore à faire.
Règles dupliquées	Candidats éventuels à une extraction, sans décider trop tôt.
Risques ouverts	Hypothèses non vérifiées, dépendances ou timeouts incertains.

Rubrique	Contenu attendu
Décisions	Ce qui a été tranché, par qui, avec lien vers la preuve.

2.9 Atelier : produire un plan de synchronisation

CAS

Navigation modifie son résultat de mouvement ; Mission ajoute une logique de nouvelle tentative ; Supervision veut afficher une cause précise ; Hardware change son timeout de 500 ms à 800 ms. Produisez un plan AI Technical Sync : fiches d'impact, conflits à vérifier, points d'intégration et synthèse finale.

Correction guidée

1. Créer une fiche d'impact par changement, avec consommateurs et compatibilité.
2. Comparer la sémantique du timeout avec la nouvelle logique de retry : le changement de 500 à 800 ms peut modifier le comportement de Mission.
3. Vérifier le mapping des nouvelles erreurs vers Supervision.
4. Créer au minimum un scénario succès, un timeout et un refus matériel traversant plusieurs modules.
5. Documenter la décision finale sur la signification du timeout et la responsabilité du retry.
6. Utiliser l'IA pour synthétiser ou comparer, mais valider les valeurs et la sémantique avec les sources du projet.

11:30-12:30 - Standardiser les interfaces

OBJECTIF DE LA SÉQUENCE

Rendre les échanges entre modules explicites : entrées, sorties, erreurs et responsabilités.

La standardisation n'a pas pour objectif de rendre toutes les interfaces identiques. Elle fournit une manière commune de penser et de lire les contrats afin que les équipes détectent plus tôt les incompatibilités.

3.1 Une interface stable n'est pas une interface figée

Stable signifie que son évolution est prévisible et maîtrisée : les consommateurs savent ce qui change, les types expriment le sens, les erreurs sont définies et les ruptures sont traitées explicitement. Une interface peut évoluer ; elle ne doit pas surprendre silencieusement.

3.2 Le contrat entrée / sortie / erreur

Dimension	Questions minimales	Exemple robot
Entrées	Quels types ? Quelles unités ? Valeurs valides ? Préconditions ?	RobotState ; batterie 0..100 ; mode connu.
Sortie	Que signifie le résultat ? Est-il total ou partiel ?	MotionResult indique allowed + reason.

Dimension	Questions minimales	Exemple robot
Erreur	Quelles erreurs sont distinguées ? Comment sont-elles propagées ?	NotIdle, BatteryTooLow, HardwareUnavailable.
Effet	Le composant modifie-t-il l'état ou déclenche-t-il une action ?	requestStart peut commander Hardware uniquement après validation.
Timing	Y a-t-il un timeout ou une limite de durée ?	Timeout explicite au niveau de l'accès matériel.

3.3 Pourquoi préférer enum class à des nombres magiques

Exemple de contrat d'erreur

```
// Fragile : que signifient 0, 1, 2 ?
int reason = 2;

// Mieux : le type porte le vocabulaire du contrat.
enum class MotionError {
    None,
    NotIdle,
    BatteryTooLow,
    BrakesUnavailable,
    FrontSensorUnavailable,
    HardwareUnavailable
};
```

`enum class` ne rend pas le programme correct à lui seul. Son bénéfice principal ici est de supprimer une convention cachée : les valeurs ont un nom, appartiennent à un type précis et peuvent être documentées dans le contrat.

3.4 Pourquoi une struct de résultat peut être plus utile qu'un bool

FRAGILE	MIEUX
bool startMotion(); false peut signifier batterie faible, mode incorrect, timeout matériel ou autre erreur.	struct MotionResult { bool allowed; MotionError error; }; Le résultat transporte la décision et une cause structurée.

3.5 Séparer hardware / métier / supervision

Zone	Responsabilité	À éviter
Hardware	Accéder au matériel et traduire les retours techniques en informations maîtrisées.	Y recopier les règles métier de batterie ou de mission.
Métier	Décider si une opération est autorisée selon les règles du domaine.	Dépendre directement de détails de protocole du bus.
Supervision	Présenter / journaliser l'état et les événements utiles à l'opérateur ou au système supérieur.	Réinventer une règle d'autorisation.

Schéma de lecture

À DESSINER

SUPERVISION <- résultat / événements <- MÉTIER -> commande abstraite -> HARDWARE -> robot physique

Faire remarquer que la règle métier n'a pas besoin de connaître chaque détail matériel, et que la supervision n'a pas à recalculer la décision.

3.6 Exemple C++ : contrat explicite et responsabilités séparées

Extrait à retaper progressivement

```
#include <string>

enum class MotionError {
    None, NotIdle, BatteryTooLow, BrakesUnavailable,
    FrontSensorUnavailable, HardwareUnavailable
};

struct RobotState {
    int batteryPercent;
    bool brakesAvailable;
    bool frontSensorAvailable;
    bool idle;
};

struct MotionResult {
    bool allowed;
    MotionError error;
};

class MotionPolicy {
public:
    MotionResult evaluate(const RobotState& s) const {
        if (!s.idle) return {false, MotionError::NotIdle};
        if (s.batteryPercent < 20) return {false, MotionError::BatteryTooLow};
        if (!s.brakesAvailable) return {false, MotionError::BrakesUnavailable};
        if (!s.frontSensorAvailable) return {false, MotionError::FrontSensorUnavailable};
        return {true, MotionError::None};
    }
};
```

3.7 Atelier minute : lire un contrat avant de lire l'implémentation

QUESTION

À partir uniquement des types RobotState, MotionError et MotionResult, demandez au groupe ce qu'il peut déjà comprendre sans lire MotionPolicy. Puis demandez ce qui reste impossible à savoir tant que le comportement n'est pas documenté ou testé.

Réponse attendue

On comprend le vocabulaire des entrées et erreurs, mais pas encore l'ordre exact des règles, le seuil batterie, ni la politique face aux valeurs invalides. Les types réduisent l'ambiguïté ; ils ne remplacent ni la spécification ni les tests.

13:30-14:30 - Définir les règles de maintenabilité

OBJECTIF DE LA SÉQUENCE

Donner au groupe une grille simple pour juger si un code restera compréhensible et modifiable quand le projet évolue.

Le déroulé liste huit thèmes : responsabilité unique, faible couplage, forte cohésion, interfaces simples, documentation courte, tests autour des règles métier, logs utiles et timeouts visibles. Ils doivent être reliés au coût du changement, pas appris comme des slogans.

4.1 Maintenabilité : la question n'est pas "ce code est-il beau ?"

Un code maintenable permet de comprendre une intention, localiser une règle, modifier une partie avec un impact raisonnable, détecter une régression et expliquer un problème. La maintenabilité se mesure surtout quand quelque chose change.

4.2 Les huit règles du déroulé

Règle	Explication simple	Signal d'alerte	Exemple MyRobot
Responsabilité unique	Un module a une raison principale de changer.	Une classe décide, journalise, parle au bus et formate l'UI.	Séparer décision métier et accès hardware.
Faible couplage	Limiter les connaissances inutiles entre modules.	Mission inclut des détails de driver matériel.	Mission dépend d'un contrat abstrait.
Forte cohésion	Regrouper ce qui sert la même responsabilité.	Des règles d'autorisation dispersées dans cinq fichiers.	MotionPolicy regroupe les règles de départ.
Interfaces simples	Exposer seulement ce dont le consommateur a besoin.	Dix paramètres booléens et conventions implicites.	RobotState + MotionResult.
Documentation courte	Expliquer le contrat, les invariants et les décisions non évidentes.	Commentaire paraphrasant chaque ligne.	Documenter l'unité, le sens des erreurs et la limite.
Tests métier	Protéger les règles qui comptent.	Tests uniquement getters / setters.	Borne batterie, mode, capteurs, freins.
Logs utiles	Donner le contexte nécessaire au diagnostic.	"error" sans identifiant ni cause.	missionId + module + cause + état pertinent.
Timeouts visibles	Les délais font partie du comportement.	Valeur 500 cachée dans une implémentation.	Constante nommée / configuration documentée.

4.3 Responsabilité unique : ce que cela ne veut pas dire

La responsabilité unique ne signifie pas "une classe = une méthode" ni "un fichier minuscule". La bonne question est : quelles raisons indépendantes pourraient me forcer à modifier ce composant ? Si une classe change à la fois lorsque le protocole du bus évolue, lorsque la règle batterie change et lorsque le format des logs change, elle mélange plusieurs responsabilités.

4.4 Couplage faible et cohésion forte : deux idées complémentaires

Question	Couplage	Cohésion
On regarde quoi ?	Les dépendances ENTRE composants.	La logique À L'INTÉRIEUR d'un composant.
On cherche quoi ?	Moins de connaissances inutiles.	Une responsabilité claire et regroupée.
Mauvais signe	Une modification locale casse de nombreux consommateurs.	Des fonctions sans rapport réunies ou une règle dispersée.
But	Pouvoir remplacer ou modifier plus localement.	Pouvoir comprendre le rôle du module rapidement.

4.5 Interfaces simples : supprimer les booléens ambigus

Comparer la charge cognitive des deux contrats

```
// Fragile : que signifient true, false, true ?
bool canMove(int battery, bool b1, bool b2, bool b3);

// Mieux : un objet d'entrée avec des noms.
MotionResult evaluate(const RobotState& state);
```

4.6 Documentation courte mais utile

FRAGILE	MIEUX

// Incrémente i
++i;
 	
Commentaire qui répète le code.
	// Timeout de communication avec le contrôleur moteur.
	// Une expiration signifie état matériel indéterminé ; ne pas relancer automatiquement ici.
	constexpr auto kMotionTimeout = ...;

La documentation doit prioritairement expliquer le pourquoi, le sens métier, les unités, les invariants et les décisions que le code seul ne rend pas évidentes.

4.7 Tests autour des règles métier

Les tests les plus rentables du jour sont ceux qui protègent une décision observable : autorisé / refusé, raison de refus, comportement à la borne, traduction d'une erreur. Ils servent de filet lors des refactorings et des extractions de code commun.

4.8 Logs utiles

Log faible	Problème	Log plus exploitable
"Motion failed"	Pas de contexte ni de cause.	mission=481 module=Navigation event=MotionRejected reason=BatteryTooLow battery=17
"Timeout"	On ne sait pas quelle opération ni la durée.	module=Hardware op=StartMotion timeoutMs=800 requestId=abc123

Log faible	Problème	Log plus exploitable
“Invalid state”	État réel absent.	module=Mission expected=Idle actual=Charging mission=481

4.9 Timeouts visibles

Un timeout caché est une règle de comportement cachée. Il influence le délai de réponse, les retries et parfois la perception d’un défaut. Le rendre visible signifie au minimum lui donner un nom, documenter son sens et savoir où il est configuré.

4.10 Mini-revue de maintenabilité

CONSIGNE

Choisissez un extrait de code du projet pédagogique. Pour chacune des huit règles, notez : OK / risque / non applicable, puis justifiez avec un fait observable. Interdiction d’écrire seulement “le code est propre” ou “le code est sale”.

14:30-15:30 - Configurer Copilot avec les règles du projet

OBJECTIF DE LA SÉQUENCE

Centraliser dans le dépôt les consignes récurrentes que l’assistant doit prendre en compte lorsqu’il propose du code ou une revue.

Le déroulé prévoit un fichier `.github/copilot-instructions.md` contenant des instructions C++, sécurité, architecture, tests et licences. L’intérêt pédagogique est de passer du prompt individuel à un contexte de projet partagé.

5.1 Pourquoi des instructions de dépôt ?

Si chaque développeur réécrit ses règles dans chaque prompt, les consignes deviennent variables et certaines sont oubliées. Un fichier d’instructions de dépôt sert de référence commune pour rappeler le style de code attendu, les frontières architecturales et les contrôles à proposer.

LIMITE IMPORTANTE

Une instruction donnée à l’assistant n’est pas un mécanisme d’enforcement. Elle oriente ses propositions, mais ne remplace pas le compilateur, les tests, la revue de code, la CI ni les règles de merge.

5.2 Structure conseillée pour le fichier du J2

Section	But	Exemples de règles
C++	Aligner les propositions sur les conventions du projet.	C++17 ; enum class ; types explicites ; éviter nombres magiques.

Section	But	Exemples de règles
Sécurité / criticité	Empêcher les suggestions qui contournent les garde-fous.	Ne jamais supprimer une vérification de frein ou capteur pour simplifier.
Architecture	Respecter les frontières du système.	Hardware ne contient pas de règles métier ; supervision ne décide pas l'autorisation.
Tests	Exiger des scénarios vérifiables.	Nominal + négatifs + bornes pour les règles métier modifiées.
Licences	Éviter l'introduction non contrôlée de dépendances.	Ne proposer aucune nouvelle dépendance sans signaler sa licence et sans validation du projet.

5.3 Exemple complet de `.github/copilot-instructions.md`

Exemple pédagogique à adapter au dépôt réel

```
# Project instructions - industrial C++

## Scope
- Target: C++17.
- Prefer clear, explicit code over clever abstractions.
- Do not invent requirements, thresholds, units, protocols or dependencies.

## Safety and critical rules
- Never remove or bypass an existing safety-related check to simplify code.
- If a requirement is missing or ambiguous, state the assumption instead of inventing it.
- Keep negative paths explicit when a motion must be refused.

## Architecture
- Keep hardware access, business rules and supervision concerns separated.
- Business code must not depend on low-level bus details unless the existing architecture requires it.
- Do not duplicate a business rule when an approved shared policy already exists.

## C++ conventions
- Use enum class for closed sets of states or errors when appropriate.
- Prefer named structs to long lists of boolean parameters.
- Avoid magic numbers: use named constants or project configuration.
- Keep interfaces small and explicit.

## Tests
- For each changed business rule, propose nominal, negative and boundary tests.
- Do not claim that tests pass unless they were actually executed by the developer or CI.

## Dependencies and licences
- Do not add a third-party dependency without explicitly flagging it.
- When suggesting a dependency, state that its licence and project approval must be checked.

## Review behaviour
- Distinguish facts visible in the code from assumptions.
- During review, prioritise requirement gaps and unsafe behaviour before style remarks.
```

5.4 Faire des règles testables plutôt que décoratives

Règle vague	Pourquoi elle est faible	Règle plus opérationnelle
“Écris du code sûr.”	Aucun critère observable.	“Ne supprime pas une vérification safety existante ; signale toute hypothèse absente du ticket.”
“Fais des tests.”	Ne dit ni quoi ni quand.	“Pour une règle métier modifiée, proposer nominal, négatifs et bornes.”
“Respecte l’architecture.”	L’assistant ne sait pas nécessairement quelle frontière compte.	“Hardware / métier / supervision restent séparés ; ne déplacer une responsabilité qu’avec justification.”
“Pas de mauvaises licences.”	Catégorie juridique implicite.	“Toute nouvelle dépendance doit être signalée pour validation de licence avant adoption.”

5.5 Atelier : première version du fichier

CONSIGNE

En binôme, rédigez une première version de `.github/copilot-instructions.md` avec exactement cinq sections : C++, sécurité, architecture, tests, licences. Chaque section doit contenir au moins deux règles observables. Pour chaque règle, demandez : comment verrait-on qu’elle n’a pas été respectée ?

Critères de correction

- Les règles ne supposent pas que l’IA connaît des contraintes absentes du dépôt.
- Les frontières hardware / métier / supervision sont exprimées clairement.
- Les règles de tests mentionnent des comportements, pas seulement un pourcentage de couverture.
- La section licences interdit au minimum l’ajout silencieux d’une dépendance non validée.
- Le fichier ne promet pas que Copilot garantit la conformité : il guide les suggestions.

15:30-16:45 - Réutilisation du code

OBJECTIF DE LA SÉQUENCE

Savoir repérer une logique commune, l’extraire proprement et éviter les dépendances trop fortes.

La réutilisation du code n’est pas “mettre tout ce qui se ressemble dans utils”. Il faut distinguer duplication accidentelle, règle réellement commune et similarité superficielle. Le déroulé prévoit : repérer les doublons, extraire une politique, créer une librairie interne, documenter, générer des exemples et des tests, puis éviter les dépendances trop fortes.

6.1 Tous les doublons ne méritent pas une abstraction

Situation	Décision probable	Pourquoi
Même règle métier copiée dans Mission et Navigation	Candidat fort à extraction.	Une divergence future donnerait deux décisions différentes pour la même règle.

Situation	Décision probable	Pourquoi
Deux boucles qui ont seulement une forme similaire	Attendre.	La ressemblance syntaxique ne prouve pas une responsabilité commune.
Même conversion d'unité officielle utilisée partout	Candidat utile.	Sémantique commune et risque d'erreur de conversion.
Deux morceaux de code encore instables et en exploration	Ne pas extraire trop tôt.	L'abstraction figerait prématurément une forme encore inconnue.

6.2 Exemple : règle batterie dupliquée

Deux copies qui ont déjà divergé

```
// Dans Mission
bool canStartMission(const RobotState& s) {
    return s.batteryPercent >= 20 && s.brakesAvailable;
}

// Dans Navigation - six mois plus tard
bool canStartMotion(const RobotState& s) {
    return s.batteryPercent > 20 && s.brakesAvailable;
}
```

La différence `>= 20` / `> 20` est minuscule visuellement mais change la décision à la borne. Si les deux fonctions représentent réellement la même politique d'autorisation, c'est un signal d'extraction.

6.3 Extraire une politique

Une source de vérité pour une politique commune

```
class MotionEligibilityPolicy {
public:
    MotionResult evaluate(const RobotState& s) const {
        if (!s.idle) return {false, MotionError::NotIdle};
        if (s.batteryPercent < 20) return {false, MotionError::BatteryTooLow};
        if (!s.brakesAvailable) return {false, MotionError::BrakesUnavailable};
        if (!s.frontSensorAvailable) return {false, MotionError::FrontSensorUnavailable};
        return {true, MotionError::None};
    }
};
```

Le mot "Policy" est utilisé ici parce que le composant porte une règle de décision. Le nom doit refléter le rôle métier, pas le mécanisme technique. Une extraction réussie réduit la duplication sans devenir un composant qui connaît toute l'application.

6.4 Quand créer une librairie interne ?

1. Le besoin de réutilisation existe dans plusieurs consommateurs réels, pas uniquement hypothétiques.
2. Le contrat est suffisamment stable pour être documenté.
3. La responsabilité est identifiable et relativement autonome.
4. Les dépendances nécessaires restent limitées.
5. Le composant peut être testé sans démarrer tout le produit.
6. L'équipe accepte d'en assurer la maintenance et les versions.

6.5 Documentation minimale d'un composant réutilisable

Élément	Question
But	Quel problème précis ce composant résout-il ?
Non-but	Que ne doit-il pas prendre en charge ?
Entrées / sorties	Quels types, unités et erreurs ?
Exemple minimal	Comment l'utiliser correctement en quelques lignes ?
Dépendances	De quoi dépend-il réellement ?
Tests	Quels comportements garantissent son contrat ?
Version / compatibilité	Comment signaler un changement de contrat ?

6.6 Générer des exemples avec l'IA sans transformer l'exemple en spécification

L'IA peut accélérer la production d'exemples d'utilisation, mais l'exemple doit être dérivé du contrat. Si l'assistant ajoute un paramètre, un comportement de retry ou une dépendance non prévus, l'exemple devient une invention. Revenir alors au contrat et corriger l'exemple.

6.7 Générer des tests pour protéger l'extraction

Avant un refactoring de duplication vers un composant commun, les tests servent à prouver que les comportements importants restent inchangés. Après l'extraction, ils protègent le composant réutilisable contre des changements incompatibles.

6.8 Éviter les dépendances trop fortes

FRAGILE	MIEUX
Une petite policy commune inclut le driver du moteur, le logger concret, la configuration globale et la supervision. Pour la réutiliser, il faut importer la moitié du produit.	La policy dépend surtout de types métier simples. Les accès hardware et la supervision restent à l'extérieur. Elle peut être testée avec des données ordinaires.

6.9 Atelier : faut-il extraire ?

CONSIGNE

Pour quatre doublons proposés par le formateur, classez : EXTRAIRE MAINTENANT / OBSERVER ENCORE / NE PAS EXTRAIRE. Justifiez par la sémantique commune, la stabilité, le nombre de consommateurs et le coût de dépendance.

16:45-17:30 - Process “Reusable Candidate”

OBJECTIF DE LA SÉQUENCE

Donner à l’équipe une méthode de décision avant de transformer une logique locale en composant partagé.

Le déroulé demande explicitement : quand extraire, quand ne pas extraire, comment documenter, nommer, versionner et tester. Cette séquence synthétise J2 sous forme de décision reproductible.

7.1 Pourquoi un statut “candidate” ?

Le mot candidate évite une décision binaire trop rapide. Repérer un doublon ne signifie pas qu’il faut immédiatement créer une bibliothèque. On collecte les preuves : répétition réelle, même sémantique, stabilité, consommateurs, dépendances et capacité de test.

7.2 Matrice de décision

Critère	Signal favorable à l’extraction	Signal pour attendre
Sémantique	Même règle / même responsabilité.	Simple ressemblance de code.
Consommateurs	Au moins plusieurs usages réels.	Un seul usage et réutilisation hypothétique.
Stabilité	Contrat relativement compris.	Règle encore changeante ou exploratoire.
Dépendances	Peu nombreuses et explicites.	Nécessite de tirer de nombreuses couches.
Tests	Comportement isolable et testable.	Impossible à vérifier sans tout le système.
Ownership	Une équipe accepte la maintenance.	Aucun responsable clair.

7.3 Quand ne pas extraire

- Quand le code est seulement similaire visuellement.
- Quand on ne comprend pas encore la règle commune.
- Quand l’abstraction obligerait à ajouter beaucoup d’options pour satisfaire des cas différents.
- Quand elle introduirait une dépendance centrale sur des détails locaux.
- Quand personne ne peut définir le contrat ou maintenir le composant.
- Quand le coût de migration dépasse le bénéfice réel à court terme.

7.4 Comment nommer

Le nom doit exprimer la responsabilité et le vocabulaire du domaine. Préférer `MotionEligibilityPolicy` à `CommonUtils`, `RobotHelper` ou `SharedStuff`. Un nom générique masque souvent un regroupement sans responsabilité claire.

7.5 Comment versionner

Le déroulé demande d’aborder le versionnement, sans imposer une stratégie précise. Le point pédagogique est simple : dès qu’un composant partagé a plusieurs consommateurs, une modification de son contrat a un coût collectif. Il faut donc identifier les changements compatibles et les changements qui obligent les consommateurs à s’adapter, puis conserver une trace de cette évolution selon les pratiques du projet.

7.6 Comment tester

Niveau	But	Exemple
Contrat nominal	Prouver le comportement attendu.	Robot Idle, batterie 50, freins et capteur OK -> autorisé.
Cas négatifs	Prouver chaque refus important.	Freins absents -> BrakesUnavailable.
Bornes	Protéger les seuils.	Batterie 19 / 20 / 21.
Compatibilité consommateur	Prouver qu'un consommateur typique utilise encore correctement le composant.	Mission interprète MotionError sans convention numérique.

7.7 Process "Reusable Candidate" en sept étapes

1. REPÉRER : relever une duplication ou une logique commune potentielle.
2. QUALIFIER : écrire en une phrase la responsabilité commune supposée.
3. COMPARER : vérifier que les cas sont sémantiquement identiques, pas seulement similaires.
4. ÉVALUER LES DÉPENDANCES : mesurer ce que le composant devrait connaître.
5. DÉFINIR LE CONTRAT : entrées, sorties, erreurs, non-buts.
6. PROTÉGER : écrire exemples et tests représentatifs.
7. DÉCIDER : extraire maintenant, observer encore ou abandonner la candidature.

7.8 Synthèse de fin de J2

LES SIX COMPÉTENCES À FAIRE VERBALISER

1. Je sais identifier un risque de silo et un bug transverse.
2. Je sais produire une fiche d'impact et organiser un AI Technical Sync.
3. Je sais décrire une interface par entrées, sorties et erreurs.
4. Je sais expliquer faible couplage, forte cohésion et responsabilité unique avec des exemples.
5. Je sais écrire des instructions projet Copilot sans les confondre avec un contrôle automatique.
6. Je sais décider si un doublon est un vrai Reusable Candidate.

8. Démonstration C++ complète - standardiser puis réutiliser

Cette démonstration rassemble les notions de J2 dans un exemple autonome C++17 : types explicites, `enum class`, `struct` de résultat, politique métier réutilisable et tests simples. Elle ne simule pas un vrai driver robotique ; son but est de montrer la structure des responsabilités.

8.1 Arborescence

```
j2-demo/
├── include/
│   ├── motion_types.hpp
│   └── motion_policy.hpp
└── main.cpp
```

8.2 `include/motion_types.hpp`

```
#pragma once

enum class MotionError {
    None,
    NotIdle,
    BatteryTooLow,
    BrakesUnavailable,
    FrontSensorUnavailable
};

struct RobotState {
    int batteryPercent;
    bool brakesAvailable;
    bool frontSensorAvailable;
    bool idle;
};

struct MotionResult {
    bool allowed;
    MotionError error;
};
```

8.3 `include/motion_policy.hpp`

```
#pragma once
#include "motion_types.hpp"

class MotionEligibilityPolicy {
public:
    MotionResult evaluate(const RobotState& s) const {
        if (!s.idle) {
            return {false, MotionError::NotIdle};
        }
        if (s.batteryPercent < 20) {
            return {false, MotionError::BatteryTooLow};
        }
        if (!s.brakesAvailable) {
            return {false, MotionError::BrakesUnavailable};
        }
        if (!s.frontSensorAvailable) {
```

```
            return {false, MotionError::FrontSensorUnavailable};
        }
        return {true, MotionError::None};
    }
};
```

8.4 `main.cpp`

```
#include <cassert>
#include <iostream>
#include "include/motion_policy.hpp"

int main() {
    MotionEligibilityPolicy policy;

    auto ok = policy.evaluate({50, true, true, true});
    assert(ok.allowed);
    assert(ok.error == MotionError::None);

    auto boundary = policy.evaluate({20, true, true, true});
    assert(boundary.allowed);

    auto lowBattery = policy.evaluate({19, true, true, true});
    assert(!lowBattery.allowed);
    assert(lowBattery.error == MotionError::BatteryTooLow);

    auto noBrakes = policy.evaluate({80, false, true, true});
    assert(!noBrakes.allowed);
    assert(noBrakes.error == MotionError::BrakesUnavailable);

    auto noSensor = policy.evaluate({80, true, false, true});
    assert(!noSensor.allowed);
    assert(noSensor.error == MotionError::FrontSensorUnavailable);

    auto notIdle = policy.evaluate({80, true, true, false});
    assert(!notIdle.allowed);
    assert(notIdle.error == MotionError::NotIdle);

    std::cout << "J2 demo: all tests passed\n";
    return 0;
}
```

8.5 Compilation et résultat attendu

```
g++ -std=c++17 -Wall -Wextra -pedantic main.cpp -o j2_demo
./j2_demo

# Résultat attendu :
J2 demo: all tests passed
```

QUESTIONS À POSER

Pourquoi la policy est-elle plus facile à réutiliser qu'une fonction qui parlerait directement au matériel ? Quel consommateur pourrait l'utiliser ? Quel changement de règle nécessiterait de modifier un seul endroit ? Quelles informations restent volontairement hors de cette policy ?

Annexe A - Fiche AI Technical Sync

MODULE / OWNER :

_____

BUT DU CHANGEMENT :

_____

INTERFACE(S) TOUCHÉE(S) :

_____

CONSOMMATEURS IMPACTÉS :

_____

COMPATIBILITÉ :

_____

NOUVELLES ERREURS / ÉTATS :

_____

TESTS À EXÉCUTER :

_____

POINTS D'INTÉGRATION :

_____

RISQUES / HYPOTHÈSES :

_____

DÉCISION / ACTION :

_____

Annexe B - Fiche de contrat d'interface

Rubrique	Questions
But	Quelle responsabilité porte cette interface ?
Entrées	Types, unités, valeurs valides, préconditions ?
Sorties	Que signifie chaque résultat ?
Erreurs	Quelles catégories et quelle sémantique ?
Effets	Quelles actions ou modifications d'état ?
Timing	Timeouts ou contraintes temporelles visibles ?
Compatibilité	Quels consommateurs seraient touchés par une évolution ?

Rubrique	Questions
Tests	Quels cas prouvent le contrat ?

Annexe C - Canevas `.github/copilot-instructions.md`

```
# Project instructions

## C++
- ...
- ...

## Safety / critical rules
- ...
- ...

## Architecture
- ...
- ...

## Tests
- ...
- ...

## Licences / dependencies
- ...
- ...
```

Annexe D - Fiche Reusable Candidate

Critère	Réponse / preuve
Responsabilité commune	
Usages réels	
Même sémantique ?	
Contrat assez stable ?	
Dépendances nécessaires	
Tests possibles	
Nom proposé	
Documentation minimale	
Owner / maintenance	
Décision : extraire / observer / abandonner	

Annexe E - Glossaire J2

Terme	Définition simple
Silo	Travail isolé où les changements utiles aux autres modules sont visibles trop tard.
Interface	Frontière par laquelle un composant est utilisé ; dans ce support, on insiste sur son contrat.
Contrat	Entrées, sorties, erreurs, effets et hypothèses que les consommateurs doivent pouvoir connaître.
Bug transverse	Défaut qui apparaît dans l'interaction entre plusieurs modules.
Couplage	Dépendance entre composants ; un fort couplage augmente l'impact d'un changement.
Cohésion	Degré avec lequel un composant regroupe des éléments qui servent une même responsabilité.
Responsabilité unique	Principe pratique consistant à éviter de mélanger plusieurs raisons indépendantes de changement.
enum class	Énumération C++ fortement typée qui donne des noms explicites à un ensemble fermé de valeurs.
struct	Type C++ permettant de regrouper plusieurs informations sous un nom commun.
Timeout	Durée limite après laquelle une opération est considérée comme n'ayant pas répondu dans le délai prévu.
AI Technical Sync	Process pédagogique du cours : fiche d'impact, résumé IA, conflits, points d'intégration, documentation et synthèse.
Reusable Candidate	Logique repérée comme potentiellement réutilisable et soumise à des critères avant extraction.
Librairie interne	Composant partagé produit et maintenu à l'intérieur de l'organisation ou du projet.
Dépendance	Composant, bibliothèque, type ou service dont un autre élément a besoin pour fonctionner.
PR - Pull Request	Proposition de changement soumise à revue avant intégration dans une branche cible.
CI - Continuous Integration	Intégration continue : automatisation des contrôles de build, tests et autres validations lors des changements.

Annexe F - Questions de relance formateur

Moment	Question	But
Silos	"Quel changement local pourrait casser un autre module sans casser le vôtre ?"	Faire penser frontières et consommateurs.
Technical Sync	"Quelle information de cette fiche doit absolument être factuelle ?"	Distinguer résumé IA et source réelle.

Moment	Question	But
Interface	“Que signifie false ici ?”	Faire émerger le besoin d’erreurs explicites.
Maintenabilité	“Quelle raison indépendante pourrait faire changer cette classe ?”	Tester la responsabilité.
Copilot	“Comment prouver que cette instruction est respectée ?”	Distinguer guidance et enforcement.
Réutilisation	“Même code ou même règle ?”	Éviter l’abstraction sur similarité syntaxique.
Reusable Candidate	“Qui maintiendra ce composant si trois équipes l’utilisent ?”	Introduire ownership et coût collectif.

Annexe G - Checklist de préparation du formateur

- Préparer un scénario de changement d’interface entre Navigation et Mission.
- Imprimer ou projeter une fiche d’impact vierge.
- Préparer deux résumés de changements volontairement contradictoires pour l’exercice de conflits.
- Préparer un exemple bool / int ambigu puis la version enum class + struct.
- Préparer un extrait avec règles métier et accès hardware mélangés pour la revue de maintenabilité.
- Créer dans le dépôt pédagogique le chemin `.github/copilot-instructions.md`.
- Préparer trois doublons : un vrai candidat, un cas à observer, une fausse similarité.
- Vérifier la compilation C++17 de la démonstration avant la session.
- Préparer la fiche Reusable Candidate pour la synthèse de fin de journée.

Annexe H - Auto-évaluation de fin de J2

Je suis capable de…	Oui	À revoir
Expliquer pourquoi une intégration tardive favorise les bugs transverses.		
Remplir une fiche d’impact module.		
Utiliser l’IA pour résumer un changement sans confondre résumé et validation.		
Décrire une interface par entrées, sorties et erreurs.		
Expliquer la différence entre couplage et cohésion.		
Repérer une responsabilité trop large.		
Écrire des règles projet Copilot observables.		
Expliquer pourquoi ces règles ne remplacent pas la CI ou la revue.		
Distinguer un vrai doublon métier d’une simple ressemblance de code.		
Évaluer un Reusable Candidate avant extraction.		

J2 - ÉDITION ÉTENDUE

Synchronisation d’équipe, maintenabilité et réutilisation du code

Approfondissements sur les frontières, contrats, règles projet et décisions de réutilisation

Objet de cette extension

Cette édition étendue approfondit le J2 en conservant sa progression : travail en silo, AI Technical Sync, contrats d'interface, maintenabilité, règles projet Copilot, réutilisation et Reusable Candidate. Les ajouts visent à rendre les notions plus facilement enseignables par des exemples, contre-exemples, matrices et ateliers.

Principe de densification

Le contenu existant est conservé comme socle. Les pages suivantes approfondissent chaque séquence avec des explications supplémentaires, des variantes de démonstration, des contre-exemples, des questions de groupe, des exercices corrigés et des mises en situation. L'objectif n'est pas d'ajouter des pages pour elles-mêmes : chaque ajout doit aider le formateur à expliquer, faire pratiquer, faire verbaliser ou contrôler une compétence du déroulé pédagogique.

Mode d’utilisation conseillé

- Lire l'approfondissement correspondant avant la séquence, puis sélectionner les exemples adaptés au niveau réel du groupe.
- Ne pas tout réciter : utiliser les blocs comme réserve d'explications, de questions et de contre-exemples.
- Conserver le fil rouge du robot porte-palette pour relier les notions et éviter l'impression d'une collection de recettes.
- Faire distinguer systématiquement ce que l'IA propose, ce qu'un outil déterministe vérifie, et ce qu'un humain décide.
- Revenir aux productions attendues du déroulé : cartographies, checklists, fichiers d'instructions, contrats, tâches locales et dossier de ticket.

Approfondissement 1 - Le silo comme perte de visibilité, pas comme séparation de fichiers

À DIRE TRÈS SIMPLEMENT
Travailler sur des modules différents est normal ; le silo commence quand les décisions utiles aux autres restent invisibles.

Explication approfondie. Une architecture modulaire répartit volontairement le travail. Le problème apparaît lorsque chaque équipe traite les frontières comme des détails internes. Une modification de signature, de sémantique d'erreur, d'unité, de timeout ou de politique métier peut être localement correcte et néanmoins casser l'intégration. Le but de J2 est donc de rendre visibles les informations qui traversent les frontières.

Le formateur peut faire distinguer trois niveaux de changement : interne sans impact externe, contrat visible mais compatible, contrat visible et incompatible. Pour chaque changement, on demande qui consomme, quel comportement change, quels tests traversent la frontière et quand les autres équipes doivent être informées. Cette grille transforme 'communiquer plus' en questions concrètes.

Application au robot porte-palette. Navigation remplace un booléen par un résultat détaillé. Mission peut continuer à compiler si une adaptation implicite existe, mais la supervision attend peut-être encore un booléen. Le risque n'est pas seulement la compilation : c'est la divergence de sens entre consommateurs.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Chaque développeur écrit un résumé en fin de sprint, quand toutes les décisions incompatibles ont déjà été prises.

VERSION MIEUX MAÎTRISÉE
Chaque changement de contrat déclenche une fiche d'impact courte, une identification des consommateurs et un point d'intégration avant la fin du travail.

Pièges et nuances à anticiper

- Confondre autonomie et absence de coordination.
- Ne surveiller que les erreurs de compilation.
- Oublier les changements de sémantique sans changement de type.
- Supposer qu'un nom de fonction suffit à décrire son contrat.

Questions au groupe

- Quel changement peut casser un consommateur sans casser la compilation ?
- Qui doit être informé quand un code d'erreur change de sens ?
- Quel test traverse réellement plusieurs modules ?

Mini-activité guidée
Donner quatre changements locaux et demander au groupe de tracer les consommateurs directs puis indirects. Faire classer l'impact faible, moyen ou fort avec justification.

Correction et débrief
Les changements d'unité, de sémantique et d'erreur sont des exemples classiques d'impact non visible par le compilateur. La fiche d'impact doit pointer les consommateurs et les tests transverses.

À RETENIR : Le risque d'intégration naît souvent à la frontière entre deux modules qui sont corrects séparément.

Approfondissement 2 - AI Technical Sync : du rituel à l'artefact vérifiable

À DIRE TRÈS SIMPLEMENT
Un bon point de synchronisation ne sert pas à raconter sa journée ; il rend visibles les changements qui peuvent affecter les autres.

Explication approfondie. Les réunions longues et générales peuvent coexister avec une mauvaise synchronisation technique. Le processus proposé vise l'inverse : peu d'informations, mais structurées autour des interfaces, hypothèses, erreurs, dépendances et points d'intégration. L'IA peut condenser les informations, comparer deux fiches ou suggérer des conflits potentiels, mais le contenu factuel doit être confirmé par les développeurs.

La fiche d'impact devient l'entrée du process. Le résumé orienté consommateurs évite le détail ligne par ligne. La détection de conflits compare noms, types, unités, erreurs et versions. Le plan d'intégration assigne ensuite un scénario, un responsable et une date. Enfin, la documentation et la synthèse hebdomadaire conservent les décisions utiles.

Application au robot porte-palette. Mission change le timeout attendu pour Navigation. Le Technical Sync doit faire apparaître : valeur ancienne, valeur nouvelle, raison, consommateurs, tests impactés et date d'intégration. L'IA peut résumer ce changement mais ne décide pas qu'il est acceptable.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
« J'ai modifié Navigation, rien de spécial. » Cette phrase ne permet pas de savoir si un contrat a changé.

VERSION MIEUX MAÎTRISÉE
« `requestMotion` garde la même signature mais le timeout passe de 500 à 800 ms ; Mission et les tests d'intégration sont consommateurs ; la décision doit être validée avant merge. »

Pièges et nuances à anticiper

- Transformer le sync en compte rendu exhaustif.
- Accepter le résumé IA sans relire les faits.
- Oublier les changements de sens quand la signature reste identique.
- Ne pas assigner les actions issues du sync.

Questions au groupe

- Quelle information doit absolument être dans la fiche d'impact ?
- Qu'est-ce qu'un résumé orienté consommateur ?
- Quand l'IA peut-elle aider sans être source de vérité ?

Mini-activité guidée
Faire remplir une fiche à partir d'un diff fictif, puis donner la fiche à un autre binôme qui n'a pas vu le diff. Le second binôme doit identifier les impacts. S'il n'y arrive pas, la fiche est à améliorer.

Correction et débrief
Une fiche réussie permet à un consommateur de savoir s'il est concerné, ce qu'il doit vérifier et quand. Les détails internes non pertinents peuvent rester hors de la synthèse.

À RETENIR : Le Technical Sync rend le changement visible avant qu'il ne devienne un problème d'intégration.

Approfondissement 3 - Contrat d'interface : entrée, sortie, erreur et sémantique

À DIRE TRÈS SIMPLEMENT
Une interface n'est pas seulement une signature C++ ; c'est une promesse sur le comportement observable.

Explication approfondie. Deux fonctions peuvent avoir la même signature tout en n'ayant pas le même contrat. Les unités, les valeurs limites, les codes d'erreur, la gestion d'un état inconnu et les effets de bord font partie de la compréhension partagée. Standardiser une interface signifie donc rendre ces éléments explicites et testables, pas seulement appliquer une convention de nommage.

On peut écrire le contrat en quatre colonnes : entrée et unité, résultat nominal, erreurs, hypothèses. Pour chaque élément, demander comment un consommateur peut le tester. Cette méthode met en évidence les booléens ambigus et les nombres magiques. `enum class` et structures de résultat deviennent alors des outils de clarté plutôt que des objectifs en soi.

Application au robot porte-palette. Un retour `false` de Hardware peut signifier refus, timeout ou indisponibilité. Mission n'a pas besoin du détail du driver, mais elle doit recevoir une sémantique suffisante pour décider ou informer la supervision.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
`bool sendMotion(int value)` sans unité, plage valide ni description de `false`.

VERSION MIEUX MAÎTRISÉE
Un contrat nomme l'unité, documente les préconditions et retourne un type qui distingue les erreurs utiles aux consommateurs.

Pièges et nuances à anticiper

- Confondre interface stable et interface immuable.
- Exposer trop de détails hardware aux couches métier.
- Créer des codes d'erreur sans définir leur sens.
- Utiliser des valeurs numériques non nommées dans le contrat.

Questions au groupe

- Qu'est-ce qu'un consommateur doit savoir sans lire l'implémentation ?
- Pourquoi `false` peut-il être un contrat insuffisant ?
- Quelle information doit rester cachée derrière la couche hardware ?

Mini-activité guidée
Présenter trois signatures et demander de rédiger le contrat manquant. Faire ensuite comparer deux contrats qui compilent mais donnent des sens différents au timeout.

Correction et débrief
La correction valorise la précision sémantique : unités, frontières, erreurs et hypothèses. Le contrat doit être assez riche pour permettre les tests sans exposer l'implémentation interne.

À RETENIR : Une interface stable est un contrat compris et maîtrisé, pas une signature figée pour toujours.

Approfondissement 4 - Maintenabilité : diminuer le coût du changement

À DIRE TRÈS SIMPLEMENT
Un code maintenable est un code qu'on peut modifier avec un risque et un effort maîtrisés.

Explication approfondie. La maintenabilité ne se résume ni à la beauté du code ni à un score automatique. Elle se juge lorsque le besoin change : combien de fichiers faut-il comprendre, combien de modules sont impactés, quels tests protègent la règle, et peut-on localiser l'erreur ? Les règles du J2 - responsabilité, couplage, cohésion, interfaces simples, documentation, tests, logs et timeouts visibles - réduisent l'incertitude liée au changement.

Pour chaque règle, le formateur doit associer un symptôme et une conséquence. Une classe à responsabilités multiples rend les modifications imprévisibles. Un couplage fort propage les changements. Une documentation trop longue devient obsolète ; trop courte, elle laisse les hypothèses implicites. Des logs sans identifiant empêchent de corréler un scénario. Les timeouts cachés empêchent de comprendre le comportement temporel.

Application au robot porte-palette. Si la politique de batterie est copiée dans Mission, Navigation et Supervision, un nouveau seuil exige trois changements coordonnés. Une policy cohésive, testée et appelée par les consommateurs réduit le coût du changement, à condition de ne pas créer une dépendance globale vers tout le système.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Refactorer parce que 'le code est moche' sans identifier le coût concret du changement.

VERSION MIEUX MAÎTRISÉE
Identifier une douleur mesurable - duplication, contrat ambigu, tests difficiles, logs insuffisants - puis améliorer la structure avec un objectif vérifiable.

Pièges et nuances à anticiper

- Créer des abstractions prématurées.
- Confondre responsabilité unique avec classe minuscule.
- Documenter chaque ligne au lieu du contrat et des décisions.
- Masquer les timeouts dans des constantes dispersées.

Questions au groupe

- Quel changement métier révèle le manque de maintenabilité ?
- Comment mesurer qu'un refactoring a réduit le couplage ?
- Quel test protège l'extraction d'une règle commune ?

Mini-activité guidée
Montrer une petite base avec une règle dupliquée et demander le coût d'un changement de seuil. Puis comparer avec une policy commune et discuter le nouveau couplage introduit.

Correction et débrief
Une extraction est utile si elle centralise une responsabilité stable et simplifie les consommateurs. Elle est nuisible si elle devient un module fourre-tout dépendant de tout le système.

À RETENIR : La maintenabilité se vérifie au moment du changement : moins de propagation, plus de clarté et des preuves plus faciles à exécuter.

Approfondissement 5 - Instructions de dépôt : rendre les règles partagées visibles à Copilot

À DIRE TRÈS SIMPLEMENT
Les instructions de dépôt rappellent à l'assistant les règles communes ; elles ne remplacent pas les outils de contrôle.

Explication approfondie. Quand chaque développeur réécrit le contexte à sa manière, les réponses deviennent incohérentes et certaines contraintes sont oubliées. Un fichier partagé permet de définir la version C++, les frontières architecturales, les commandes de test, les règles de dépendance et la manière de signaler l'incertitude. Le bénéfice est la cohérence de guidance, pas une garantie de conformité.

Une bonne instruction est spécifique au projet et vérifiable. 'Écris du bon C++' est décoratif. 'Ne pas accéder aux drivers depuis Mission' décrit une frontière. 'Exécuter les tests du module modifié' décrit une preuve attendue. Le J2 prépare ainsi

la matière qui sera réellement outillée au J3.

Application au robot porte-palette. Le fichier peut rappeler que Mission ne contourne pas Safety, que les motifs d'erreur sont explicites et qu'aucune dépendance externe n'est ajoutée sans validation. Ces règles sont ensuite utilisées dans les prompts et revues.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Un fichier de deux pages rempli de conseils génériques, jamais relu ni relié au processus.

VERSION MIEUX MAÎTRISÉE
Une liste courte de décisions du projet, maintenue avec le dépôt et formulée de manière observable.

Pièges et nuances à anticiper

- Croire que l'assistant suivra toujours parfaitement les instructions.
- Mettre des règles contradictoires.
- Copier des conseils génériques sans lien avec le dépôt.
- Ne pas mettre à jour les commandes lorsque le build change.

Questions au groupe

- Quelle instruction mérite d'être partagée à tout le dépôt ?
- Comment rendre une règle testable ?
- Quelle règle doit être automatisée plutôt que seulement écrite ?

Mini-activité guidée
Donner dix phrases candidates. Le groupe doit retenir celles qui décrivent une décision projet, réécrire les formulations vagues et classer celles qui devraient devenir des contrôles automatisés.

Correction et débrief
Le fichier final doit être court, spécifique et cohérent avec les commandes réelles. Les règles critiques seront renforcées par les tests, tâches et CI au J3.

À RETENIR : Une instruction de dépôt est une guidance commune ; les garanties viennent des contrôles et de la revue.

Approfondissement 6 - Réutilisation : distinguer duplication et politique commune

À DIRE TRÈS SIMPLEMENT
Deux morceaux de code semblables ne justifient pas forcément une abstraction ; il faut d'abord vérifier qu'ils représentent la même règle.

Explication approfondie. L'extraction trop précoce produit des composants génériques difficiles à comprendre. L'absence d'extraction produit des règles qui divergent. Le process Reusable Candidate aide à temporiser : on repère une répétition, on cherche la raison commune, on identifie les consommateurs, on vérifie la stabilité du contrat et on décide si l'extraction diminue réellement le couplage.

On peut poser cinq questions : la règle est-elle conceptuellement la même, a-t-elle au moins deux consommateurs réels, change-t-elle pour les mêmes raisons, peut-elle être testée sans dépendre du hardware, et son nom reste-t-il compréhensible hors du premier module ? Si plusieurs réponses sont non, garder la duplication temporaire peut être plus sain.

Application au robot porte-palette. Le seuil de batterie répété dans Mission et Navigation peut représenter une politique commune. Mais deux fonctions qui comparent une batterie à 20 pour des raisons différentes ne doivent pas forcément être fusionnées. Il faut comprendre la sémantique avant la syntaxe.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Extraire `CommonUtils` dès que deux lignes se ressemblent.

VERSION MIEUX MAÎTRISÉE
Créer une policy nommée d'après la responsabilité métier, avec contrat, tests et consommateurs identifiés.

Pièges et nuances à anticiper

- Nommer un composant `Utils` ou `Helper` sans responsabilité claire.
- Créer une librairie interne pour un seul appelant hypothétique.
- Faire dépendre la policy d'objets hardware lourds.
- Versionner sans définir les consommateurs et la compatibilité.

Questions au groupe

- Ces deux duplications changent-elles pour la même raison ?
- Quel nom métier décrit l'abstraction ?
- Peut-on tester le composant sans le robot complet ?

Mini-activité guidée
Fournir cinq paires de duplications : certaines sémantiques, d'autres accidentelles. Les binômes classent : extraire maintenant, candidat à observer, ne pas extraire.

Correction et débrief
La décision doit être argumentée par la responsabilité et les consommateurs, jamais par le nombre de lignes identiques seul.

À RETENIR : Réutiliser consiste à partager une responsabilité stable, pas à éliminer mécaniquement tout doublon.

Approfondissement 7 - Reusable Candidate : décider avant de créer une librairie

À DIRE TRÈS SIMPLEMENT
Un composant candidat n'est pas encore une librairie : il doit passer des critères avant l'extraction.

Explication approfondie. Le statut intermédiaire évite deux extrêmes : la duplication éternelle et l'abstraction précipitée. Il permet d'accumuler des preuves : plusieurs consommateurs, contrat commun, tests stables, dépendances maîtrisées, nom clair et responsable identifié. La décision peut être revue lorsque le contexte évolue.

Le processus en sept étapes du support peut être animé comme un entonnoir : repérer, caractériser, comparer, concevoir le contrat, tester, documenter, décider. À chaque étape, une raison de ne pas extraire est acceptable. Cette idée est importante : refuser une abstraction peut être une décision de qualité.

Application au robot porte-palette. Une politique d'éligibilité au mouvement peut être candidate si Mission et Navigation partagent réellement le même calcul. Elle n'est pas candidate si chaque module doit appliquer une règle différente ou si l'extraction impose une dépendance vers la couche Hardware.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Créer `robot_common` comme dépendance de tous les modules et y déplacer progressivement toute logique partagée.

VERSION MIEUX MAÎTRISÉE
Extraire un composant petit, cohésif, avec une API minimale, des tests et une ownership claire.

Pièges et nuances à anticiper

- Faire d'une librairie interne un point de couplage global.
- Oublier la politique de version et de compatibilité.
- Ne pas documenter les cas non couverts.
- Confondre réutilisable et universel.

Questions au groupe

- Quel critère justifie réellement l'extraction aujourd'hui ?
- Qui maintient ce composant ?
- Comment savoir si l'API devient trop générique ?

Mini-activité guidée
Remplir une fiche Reusable Candidate sur `MotionEligibilityPolicy`, puis changer une hypothèse : Navigation a désormais une règle différente. Refaire la décision.

Correction et débrief
Avec des règles divergentes, l'abstraction commune peut ne plus être juste. Le processus doit accepter ce résultat et éviter de forcer une généralisation.

À RETENIR : Le statut candidat protège l'équipe contre les abstractions prématurées tout en gardant visibles les opportunités réelles.

Approfondissement 8 - Logs, timeouts et diagnostic : rendre le comportement observable

À DIRE TRÈS SIMPLEMENT
Un bug qu'on ne peut pas observer devient beaucoup plus coûteux à comprendre.

Explication approfondie. Dans un système distribué entre modules, une erreur peut traverser plusieurs couches. Les logs et timeouts font partie du contrat opérationnel : ils permettent de savoir quelle demande a été faite, combien de temps elle a attendu et quel composant a refusé. Le J2 n'a pas pour but d'enseigner une plateforme de logs, mais de faire comprendre pourquoi une information utile doit être structurée et cohérente.

Un log utile répond à des questions : quelle mission, quel module, quel événement, quel résultat, quel motif. Un timeout visible est nommé, documenté et testé ; il ne doit pas être un nombre magique perdu dans une fonction. La supervision peut traduire le résultat, mais ne doit pas supprimer l'information nécessaire au diagnostic.

Application au robot porte-palette. Si Hardware répond timeout et Supervision transforme tout en `MOTION_FAILED`, l'équipe perd la cause. Un identifiant de mission et un motif structuré permettent de corréler la chaîne sans exposer tous les détails internes à l'utilisateur final.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
`std::cout << "error"` dans plusieurs modules et timeout `500` copié dans trois fichiers.

VERSION MIEUX MAÎTRISÉE
Événements nommés, contexte minimal de corrélation, motifs explicites et constantes de timeout visibles dans le contrat approprié.

Pièges et nuances à anticiper

- Logger des données sensibles.
- Confondre volume de logs et qualité du diagnostic.
- Modifier un timeout sans tester les consommateurs.
- Traduire trop tôt une erreur technique en message générique.

Questions au groupe

- Quelle information faut-il conserver pour relier un échec à une mission ?
- Pourquoi un timeout est-il une partie du comportement observable ?
- Quelle couche doit traduire le message pour l'opérateur ?

Mini-activité guidée
Donner une trace de cinq lignes ambiguës et demander quelles informations manquent. Puis proposer une version structurée minimale.

Correction et débrief
La version corrigée ajoute seulement les éléments nécessaires : identifiant, module, événement, motif. Le message utilisateur peut rester simple tandis que la cause détaillée reste disponible pour le diagnostic.

À RETENIR : Observabilité et maintenabilité se renforcent : un comportement visible se diagnostique et se modifie plus facilement.

Études de cas supplémentaires

Ces études de cas servent à faire transférer les règles vues dans le cours. Chaque cas est conçu pour obliger le groupe à justifier une décision plutôt qu'à réciter une définition.

Cas 1 - Renommage de méthode sans changement de comportement

Navigation renomme `startMove()` en `requestMotion()` parce que le nouveau nom paraît meilleur. Mission et des tests d’intégration utilisent encore l’ancien nom.

Questions d’analyse

- Quel est l’impact immédiat ?
- Quelle fiche d’impact remplir ?
- Faut-il synchroniser avant ou après le merge ?

Correction argumentée
Le changement est visible au contrat et doit être signalé avant intégration. La compilation détectera une partie de l’impact, mais la synchronisation évite de transformer le merge en découverte tardive.

Cas 2 - Même signature, nouvelle unité

Une fonction conserve `setSpeed(int value)` mais le développeur passe de cm/s à mm/s sans changer le nom ni le type.

Questions d’analyse

- Pourquoi le compilateur ne protège-t-il pas ?
- Quelle information fait partie du contrat ?
- Quel test transverse ajouter ?

Correction argumentée
L’unité est sémantique et doit être explicitée. La signature identique masque un changement incompatible. Le contrat, les consommateurs et les tests doivent être mis à jour ensemble.

Cas 3 - Doublon qui n’est pas la même règle

Mission et Supervision comparent toutes deux la batterie à 20, mais Mission décide du mouvement tandis que Supervision choisit seulement un niveau d’alerte.

Questions d’analyse

- Doit-on extraire ?
- Quelles raisons de changement sont différentes ?
- Quel nom commun serait trompeur ?

Correction argumentée
La similarité syntaxique ne prouve pas une responsabilité commune. Les règles peuvent évoluer séparément ; une extraction commune risquerait de coupler deux décisions distinctes.

Cas 4 - Librairie interne sans propriétaire

Une policy est extraite et utilisée par quatre modules, mais personne n’est responsable de la compatibilité ou de la version.

Questions d’analyse

- Quel risque organisationnel apparaît ?
- Que doit contenir la fiche Reusable Candidate ?
- Comment décider d’une modification incompatible ?

Correction argumentée
La réutilisation crée une responsabilité de maintenance. Il faut identifier owner, version ou stratégie de compatibilité, tests et consommateurs.

Cas 5 - Résumé IA qui invente un impact

Un assistant résume une PR et affirme que Supervision n’est pas affectée, alors qu’il n’a pas analysé ce module.

Questions d’analyse

- Fait ou hypothèse ?
- Comment reformuler le résumé ?
- Qui confirme les consommateurs ?

Correction argumentée
Le résumé doit signaler les limites. Les consommateurs sont confirmés à partir du dépôt, de la fiche d’impact et de la connaissance de l’équipe ; l’IA peut proposer, pas certifier.

Cas 6 - Erreur technique écrasée

Hardware distingue Timeout et Rejected, mais Mission convertit les deux en false et Supervision affiche le même message.

Questions d’analyse

- Quelle information est perdue ?
- Quel contrat intermédiaire proposer ?
- Faut-il exposer le code hardware brut ?

Correction argumentée
Il faut préserver la distinction utile sans nécessairement exposer le détail brut. Un type d’erreur métier partagé peut jouer ce rôle.

Ateliers longs et corrections

Atelier 1 - Simuler un AI Technical Sync sur trois modules

Contexte
Trois équipes ont modifié Mission, Navigation et Hardware en parallèle.

Consigne
Produire trois fiches d’impact, un résumé orienté consommateurs, une liste de conflits potentiels et un plan d’intégration minimal.

Déroulement formateur

- Distribuer un mini-diff différent à chaque groupe.
- Interdire l’accès aux diff des autres groupes au début.
- Faire échanger uniquement les fiches d’impact.
- Identifier les zones encore ambiguës.
- Autoriser ensuite la consultation croisée pour valider les faits.

Correction détaillée
La correction doit faire apparaître au moins un conflit de sémantique, un consommateur indirect et un test d’intégration. L’IA peut être utilisée pour condenser les fiches, mais toute conclusion doit être relue.

Critères de réussite

- Chaque fiche nomme les contrats touchés.
- Les consommateurs sont identifiés.
- Les conflits sont formulés comme hypothèses à confirmer.
- Le plan d’intégration a un scénario et un responsable.

Atelier 2 - Décider d’une extraction réutilisable

Contexte
Deux modules contiennent une logique proche de décision batterie.

Consigne
Remplir la matrice Reusable Candidate et décider : extraire, observer ou ne pas extraire. Puis proposer un contrat minimal si l’extraction est retenue.

Déroulement formateur

- Faire identifier la responsabilité de chaque duplication.
- Comparer les raisons de changement.
- Lister les consommateurs réels, pas hypothétiques.
- Concevoir un test indépendant du hardware.
- Discuter le nom et l’ownership.

Correction détaillée
Une extraction est retenue seulement si la règle est vraiment commune. Le contrat doit rester petit et ne pas dépendre de couches inutiles. Si les règles diffèrent, la bonne correction peut être de ne pas extraire.

Critères de réussite

- La décision est argumentée.
- Le nom décrit une responsabilité métier.
- Les dépendances sont minimales.
- Les tests protègent le contrat.

Tableaux de référence complémentaires

Fiche d’impact module - version enrichie

Champ	Question
Changement	Qu’est-ce qui change exactement ?
Contrat touché	Signature, type, unité, erreur, timeout ou sémantique ?
Consommateurs	Qui appelle, observe ou dépend du résultat ?
Compatibilité	Le changement est-il compatible ? Pourquoi ?
Tests	Quels tests locaux et transverses sont impactés ?
Action	Qui doit être informé ou adapter son code ?

Matrice Reusable Candidate

Critère	Oui	Non / risque
Même responsabilité métier	Même raison de changement	Similarité seulement syntaxique
Consommateurs réels	Au moins deux cas actuels	Usage futur hypothétique
Contrat stable	Entrées/sorties claires	Besoins encore divergents
Testabilité	Test sans infrastructure lourde	Dépend du robot complet
Couplage	Réduit les dépendances	Crée un module central global

Démonstrations et code complémentaires

Démonstration - contrat explicite et policy cohésive

Exemple C++17 autonome pour discuter responsabilité, réutilisation et testabilité.

```
#include <cassert>
#include <iostream>

enum class MotionError { None, NotIdle, BatteryTooLow, BrakesUnavailable, FrontSensorUnavailable };
struct RobotState { int batteryPercent; bool brakesAvailable; bool frontSensorAvailable; bool idle; };
struct MotionResult { bool allowed; MotionError error; };

class MotionEligibilityPolicy {
public:
    MotionResult evaluate(const RobotState& s) const {
        if (!s.idle) return {false, MotionError::NotIdle};
        if (s.batteryPercent < 20) return {false, MotionError::BatteryTooLow};
        if (!s.brakesAvailable) return {false, MotionError::BrakesUnavailable};
        if (!s.frontSensorAvailable) return {false, MotionError::FrontSensorUnavailable};
        return {true, MotionError::None};
    }
};

int main() {
    MotionEligibilityPolicy policy;
    assert(policy.evaluate({20,true,true,true}).allowed);
    assert(policy.evaluate({19,true,true,true}).error == MotionError::BatteryTooLow);
    std::cout << "contract demo passed\n";
}
```

Points à commenter

- La policy ne parle pas directement au matériel.
- Le contrat est lisible sans connaître l’implémentation du driver.
- La valeur frontière peut être protégée par un test.
- L’extraction n’est justifiée que si plusieurs consommateurs partagent réellement cette règle.

Banque de questions de vérification

1. Quel changement peut casser un consommateur sans casser la compilation ?

Réponse attendue : Le risque d'intégration naît souvent à la frontière entre deux modules qui sont corrects séparément. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

2. Qui doit être informé quand un code d'erreur change de sens ?

Réponse attendue : Le risque d'intégration naît souvent à la frontière entre deux modules qui sont corrects séparément. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

3. Quelle information doit absolument être dans la fiche d'impact ?

Réponse attendue : Le Technical Sync rend le changement visible avant qu'il ne devienne un problème d'intégration. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

4. Qu'est-ce qu'un résumé orienté consommateur ?

Réponse attendue : Le Technical Sync rend le changement visible avant qu'il ne devienne un problème d'intégration. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

5. Qu'est-ce qu'un consommateur doit savoir sans lire l'implémentation ?

Réponse attendue : Une interface stable est un contrat compris et maîtrisé, pas une signature figée pour toujours. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

6. Pourquoi `false` peut-il être un contrat insuffisant ?

Réponse attendue : Une interface stable est un contrat compris et maîtrisé, pas une signature figée pour toujours. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

7. Quel changement métier révèle le manque de maintenabilité ?

Réponse attendue : La maintenabilité se vérifie au moment du changement : moins de propagation, plus de clarté et des preuves plus faciles à exécuter. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

8. Comment mesurer qu'un refactoring a réduit le couplage ?

Réponse attendue : La maintenabilité se vérifie au moment du changement : moins de propagation, plus de clarté et des preuves plus faciles à exécuter. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

9. Quelle instruction mérite d'être partagée à tout le dépôt ?

Réponse attendue : Une instruction de dépôt est une guidance commune ; les garanties viennent des contrôles et de la revue. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

10. Comment rendre une règle testable ?

Réponse attendue : Une instruction de dépôt est une guidance commune ; les garanties viennent des contrôles et de la revue. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

11. Ces deux duplications changent-elles pour la même raison ?

Réponse attendue : Réutiliser consiste à partager une responsabilité stable, pas à éliminer mécaniquement tout doublon. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

12. Quel nom métier décrit l'abstraction ?

Réponse attendue : Réutiliser consiste à partager une responsabilité stable, pas à éliminer mécaniquement tout doublon. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

13. Quel critère justifie réellement l'extraction aujourd'hui ?

Réponse attendue : Le statut candidat protège l'équipe contre les abstractions prématurées tout en gardant visibles les opportunités réelles. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

14. Qui maintient ce composant ?

Réponse attendue : Le statut candidat protège l'équipe contre les abstractions prématurées tout en gardant visibles les opportunités réelles. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

15. Quelle information faut-il conserver pour relier un échec à une mission ?

Réponse attendue : Observabilité et maintenabilité se renforcent : un comportement visible se diagnostique et se modifie plus facilement. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

16. Pourquoi un timeout est-il une partie du comportement observable ?

Réponse attendue : Observabilité et maintenabilité se renforcent : un comportement visible se diagnostique et se modifie plus facilement. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

Synthèse générale de la journée - version approfondie

La compétence recherchée n'est pas de connaître une liste de règles par cœur. Le participant doit être capable de regarder une situation nouvelle et de reconstruire le raisonnement : identifier le risque, expliciter le contrat, cadrer la demande, obtenir une proposition, organiser les preuves, puis décider. Le fil rouge du robot permet de rendre ce raisonnement concret, mais la méthode doit pouvoir être transférée à d'autres systèmes C++ industriels.

- Le risque d'intégration naît souvent à la frontière entre deux modules qui sont corrects séparément.
- Le Technical Sync rend le changement visible avant qu'il ne devienne un problème d'intégration.
- Une interface stable est un contrat compris et maîtrisé, pas une signature figée pour toujours.
- La maintenabilité se vérifie au moment du changement : moins de propagation, plus de clarté et des preuves plus faciles à exécuter.
- Une instruction de dépôt est une guidance commune ; les garanties viennent des contrôles et de la revue.
- Réutiliser consiste à partager une responsabilité stable, pas à éliminer mécaniquement tout doublon.
- Le statut candidat protège l'équipe contre les abstractions prématurées tout en gardant visibles les opportunités réelles.
- Observabilité et maintenabilité se renforcent : un comportement visible se diagnostique et se modifie plus facilement.

Annexe approfondie 1 - Guide d’animation par séquence

Ce guide aide le formateur à transformer le support en rythme de salle. Pour chaque séquence, la logique reste la même : faire émerger le problème, introduire la notion seulement ensuite, faire produire une décision par les participants, puis terminer par une preuve ou un artefact réutilisable.

Séquence	Ouverture conseillée	Activité centrale	Trace de fin
09:00-10:00 travail en silo	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
10:00-11:30 AI Technical Sync	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
11:30-12:30 contrats d’interface	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
13:30-14:30 maintenabilité	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
14:30-15:30 règles Copilot	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
15:30-16:45 réutilisation	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
16:45-17:30 Reusable Candidate	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite

Conseil d’animation : ne pas donner immédiatement la réponse. Laisser d’abord le groupe formuler ce qu’il faudrait vérifier. Lorsque plusieurs réponses sont possibles, demander quel fait permettrait de trancher. Cette habitude construit le réflexe central de la formation : ne pas remplacer une incertitude par une affirmation plausible.

- Faire reformuler la règle à la fin de chaque bloc.
- Conserver au tableau les décisions qui seront réutilisées plus tard dans la journée.
- Faire expliciter la différence entre une hypothèse, un fait observé et une décision.
- Demander régulièrement : « quelle preuve avons-nous réellement ? »
- Si le groupe va trop vite vers le code, revenir au contrat et aux critères d’acceptation.

Annexe approfondie 2 - FAQ formateur et réponses de secours

Question 1. Quel changement peut casser un consommateur sans casser la compilation ?

Réponse courte : Le risque d'intégration naît souvent à la frontière entre deux modules qui sont corrects séparément.

Réponse développée : Une architecture modulaire répartit volontairement le travail. Le problème apparaît lorsque chaque équipe traite les frontières comme des détails internes. Une modification de signature, de sémantique d'erreur, d'unité, de timeout ou de politique métier peut être localement correcte et néanmoins casser l'intégration. Le but de J2 est donc de rendre visibles les informations qui traversent les frontières. Le formateur peut ensuite revenir sur le fil rouge : Navigation remplace un booléen par un résultat détaillé. Mission peut continuer à compiler si une adaptation implicite existe, mais la supervision attend peut-être encore un booléen. Le risque n'est pas seulement la compilation : c'est la divergence de sens entre consommateurs.

Question 2. Quelle information doit absolument être dans la fiche d'impact ?

Réponse courte : Le Technical Sync rend le changement visible avant qu'il ne devienne un problème d'intégration.

Réponse développée : Les réunions longues et générales peuvent coexister avec une mauvaise synchronisation technique. Le processus proposé vise l'inverse : peu d'informations, mais structurées autour des interfaces, hypothèses, erreurs, dépendances et points d'intégration. L'IA peut condenser les informations, comparer deux fiches ou suggérer des conflits potentiels, mais le contenu factuel doit être confirmé par les développeurs. Le formateur peut ensuite revenir sur le fil rouge : Mission change le timeout attendu pour Navigation. Le Technical Sync doit faire apparaître : valeur ancienne, valeur nouvelle, raison, consommateurs, tests impactés et date d'intégration. L'IA peut résumer ce changement mais ne décide pas qu'il est acceptable.

Question 3. Qu'est-ce qu'un consommateur doit savoir sans lire l'implémentation ?

Réponse courte : Une interface stable est un contrat compris et maîtrisé, pas une signature figée pour toujours.

Réponse développée : Deux fonctions peuvent avoir la même signature tout en n'ayant pas le même contrat. Les unités, les valeurs limites, les codes d'erreur, la gestion d'un état inconnu et les effets de bord font partie de la compréhension partagée. Standardiser une interface signifie donc rendre ces éléments explicites et testables, pas seulement appliquer une convention de nommage. Le formateur peut ensuite revenir sur le fil rouge : Un retour `false` de Hardware peut signifier refus, timeout ou indisponibilité. Mission n'a pas besoin du détail du driver, mais elle doit recevoir une sémantique suffisante pour décider ou informer la supervision.

Question 4. Quel changement métier révèle le manque de maintenabilité ?

Réponse courte : La maintenabilité se vérifie au moment du changement : moins de propagation, plus de clarté et des preuves plus faciles à exécuter.

Réponse développée : La maintenabilité ne se résume ni à la beauté du code ni à un score automatique. Elle se juge lorsque le besoin change : combien de fichiers faut-il comprendre, combien de modules sont impactés, quels tests protègent la règle, et peut-on localiser l'erreur ? Les règles du J2 - responsabilité, couplage, cohésion, interfaces simples, documentation, tests, logs et timeouts visibles - réduisent l'incertitude liée au changement. Le formateur peut ensuite revenir sur le fil rouge : Si la politique de batterie est copiée dans Mission, Navigation et Supervision, un nouveau seuil exige trois changements coordonnés. Une policy cohésive, testée et appelée par les consommateurs réduit le coût du changement, à condition de ne pas créer une dépendance globale vers tout le système.

Question 5. Quelle instruction mérite d'être partagée à tout le dépôt ?

Réponse courte : Une instruction de dépôt est une guidance commune ; les garanties viennent des contrôles et de la revue.

Réponse développée : Quand chaque développeur réécrit le contexte à sa manière, les réponses deviennent incohérentes et certaines contraintes sont oubliées. Un fichier partagé permet de définir la version C++, les frontières architecturales, les commandes de test, les règles de dépendance et la manière de signaler l'incertitude. Le bénéfice est la cohérence de guidance, pas une garantie de conformité. Le formateur peut ensuite revenir sur le fil rouge : Le fichier peut rappeler que Mission ne contourne pas Safety, que les motifs d'erreur sont explicites et qu'aucune dépendance externe n'est ajoutée sans validation. Ces règles sont ensuite utilisées dans les prompts et revues.

Question 6. Ces deux duplications changent-elles pour la même raison ?

Réponse courte : Réutiliser consiste à partager une responsabilité stable, pas à éliminer mécaniquement tout doublon.

Réponse développée : L'extraction trop précoce produit des composants génériques difficiles à comprendre. L'absence d'extraction produit des règles qui divergent. Le process Reusable Candidate aide à temporiser : on repère une répétition, on cherche la raison commune, on identifie les consommateurs, on vérifie la stabilité du contrat et on décide si l'extraction diminue réellement le couplage. Le formateur peut ensuite revenir sur le fil rouge : Le seuil de batterie répété dans Mission et Navigation peut représenter une politique commune. Mais deux fonctions qui comparent une batterie à 20 pour des raisons différentes ne doivent pas forcément être fusionnées. Il faut comprendre la sémantique avant la syntaxe.

Si une question dépasse le périmètre de la journée, ne pas improviser une architecture ou une règle future. La bonne réponse pédagogique consiste à reconnaître le périmètre, reformuler ce qui est déjà établi et indiquer clairement ce qui doit être confirmé par la politique, l'architecture ou les spécialistes de l'entreprise.

Annexe approfondie 3 - Grille de diagnostic d’une production participant

Dimension	Question de contrôle	Signal satisfaisant	Signal d’alerte
Besoin	Le participant sait-il expliquer le problème avant la solution ?	Objectif et contraintes reformulés	Code produit sans besoin explicite
Hypothèses	Les inconnues sont-elles signalées ?	Questions ouvertes visibles	Valeurs inventées ou implicites
Contrat	Entrées, sorties et erreurs sont-elles compréhensibles ?	Comportement testable	Booléens / nombres ambigus
Preuves	Les tests ou contrôles sont-ils reliés aux exigences ?	Frontières et cas négatifs couverts	Seulement un cas nominal
IA	Le rôle de l’assistant est-il correctement limité ?	Proposition / aide à la revue	Validation déléguée à l’outil
Décision	La conclusion est-elle argumentée ?	Acceptation, modification ou refus justifié	« ça marche » sans critères
Traçabilité	Peut-on retrouver les éléments importants ?	Ticket, tests, revue, décision reliés	Connaissance uniquement dans le chat

Cette grille n’est pas un barème obligatoire. Elle sert à diagnostiquer rapidement où se situe la difficulté : technique, contrat, preuve ou raisonnement. Elle permet aussi d’éviter de sur-noter une solution qui compile mais dont le processus reste fragile.

Annexe approfondie 4 - Variantes selon le niveau du groupe

Situation du groupe	Adaptation	Ce qu'il ne faut pas supprimer
Groupe peu expérimenté	Réduire la taille du code, garder un seul changement à la fois, verbaliser chaque terme	La distinction proposition / preuve / décision
Groupe expérimenté	Ajouter des ambiguïtés de contrat et demander plusieurs solutions	La justification des choix et les preuves
Groupe très orienté outils	Commencer par le problème avant d'ouvrir Copilot ou VS Code	Le pourquoi métier et le rôle humain
Pas d'accès réseau	Utiliser des sorties IA préparées à l'avance et simuler la revue	L'analyse critique et les contrôles déterministes
Temps réduit	Supprimer un cas secondaire, pas les étapes de raisonnement	Une activité pratique + correction + synthèse
Groupe hétérogène	Binômes mélangeant niveaux, rôles alternés auteur/reviewer	La participation de chacun et la verbalisation

Une adaptation de niveau ne doit pas transformer la formation en démonstration passive. Même avec des débutants, les participants doivent prendre des décisions simples et expliquer leur raisonnement. Avec des experts, on augmente surtout l'ambiguïté des cas et la profondeur de justification plutôt que d'ajouter des outils hors programme.

- Version courte : une seule erreur injectée à la fois.
- Version avancée : deux contraintes contradictoires à détecter.
- Version atelier : un binôme produit, un autre révise avec la checklist.
- Version paperboard : aucune machine pendant dix minutes, uniquement contrat et tests attendus.

Annexe approfondie 5 - Plan B et scénarios de secours pour le formateur

Une formation centrée sur des outils peut être fragilisée par un problème de compte, de réseau, de licence, d'extension ou de configuration. Le contenu pédagogique doit rester exécutable même si l'assistant ou la plateforme n'est pas disponible. Les scénarios ci-dessous permettent de continuer sans changer les objectifs de la journée.

Incident	Plan B pédagogique	Objectif conservé
Copilot indisponible	Projeter deux réponses IA préparées, dont une volontairement imparfaite	Évaluer et revoir une proposition
GitHub indisponible	Utiliser un diff local et une fiche PR papier	Traçabilité, impacts et décision
CI indisponible	Lire un log de workflow préparé avec un check vert et un check rouge	Interpréter un contrôle automatisé
Compilateur indisponible	Faire prédire les résultats et analyser les tests sur papier	Relier règles, code et preuves
Temps de build trop long	Utiliser la mini-démo autonome du support	Conserver la logique de contrôle
Question hors politique interne	Revenir au principe générique et signaler la validation nécessaire	Ne pas inventer une règle d'entreprise

Préparer ces éléments avant le cours évite de transformer un incident d'outillage en perte pédagogique. Le fil conducteur doit rester le même : une proposition est analysée, les contraintes sont explicitées, des preuves sont recherchées et la décision finale reste attribuée au bon rôle.

CHECKLIST AVANT DE QUITTER LA SALLE
Le groupe peut expliquer au moins trois règles de la journée avec ses propres mots ; il peut citer une erreur typique ; il sait donner un exemple de preuve ; il sait dire ce qui reste une décision humaine ; et il a produit au moins un artefact réutilisable dans la suite de la formation.

SUPPORT FORMATEUR ENRICHI

J3

Copilot, VS Code, licences, CI et mise en pratique complète

Environnement IA contrôlé, tâches locales, politique de dépendances, Dependency Review et simulation de PR

FIL ROUGE

Robot porte-palette : le J3 transforme les règles vues en J1 et J2 en environnement de travail concret. L'objectif n'est plus seulement de savoir demander ou relire : il faut intégrer les règles dans le dépôt, exécuter les contrôles localement et empêcher qu'une pull request contourne les validations attendues.

DURÉE

7 h 30 de formation - 09:00-12:30 et 13:30-17:30.

Document de préparation et d'animation - version dense et détaillée

0. Comment utiliser ce support

Ce document est le support formateur enrichi du J3. Il suit le déroulé fourni : paramétrage Copilot, paramétrage VS Code, politique de licences, GitHub Dependency Review, puis atelier final sur un ticket complet. Les exemples techniques ajoutés servent à rendre ces notions démontrables ; ils sont identifiés comme exemples pédagogiques et doivent être adaptés à la vraie chaîne de build de l'entreprise.

IDÉE DIRECTRICE DU J3

Une règle écrite dans un prompt ou dans un fichier d'instructions aide l'équipe ; une règle automatisée dans les tâches locales et la CI protège réellement le flux de livraison. Le J3 fait donc passer le groupe de la guidance vers la vérification, puis de la vérification vers le blocage contrôlé.

0.1 Progression pédagogique du J3

Horaire	Séquence	Question centrale	Production
09:00-10:15	Paramétrage Copilot	Comment donner à l'IA des règles stables sans répéter tout le contexte ?	Instructions dépôt + instructions ciblées + protocole de revue
10:15-11:30	Paramétrage VS Code	Comment rendre build, tests et contrôle licence faciles à lancer avant une PR ?	.vscode/settings.json + .vscode/tasks.json
11:30-12:30	Politique de licences	Comment décider si une dépendance peut entrer dans le produit ?	Matrice de décision + classification de dépendances fictives
13:30-15:30	GitHub Dependency Review	Comment automatiser les contrôles vulnérabilités/licences dans une pull request ?	Workflow de revue des dépendances + analyse guidée
15:30-17:30	Atelier final	Comment appliquer tout le processus à un ticket réel de bout en bout ?	Ticket traité, contrôles locaux, PR simulée, revue et décision

0.2 Les trois niveaux de contrôle à faire distinguer

Niveau	Exemple	Force	Limite
Guidance	Prompt, `.github/copilot-instructions.md`, instructions ciblées.	Réduit les écarts et rappelle le contexte.	Une IA peut mal interpréter ou ne pas suivre parfaitement une instruction.
Vérification locale	Tâche VS Code de build, test, scan licence.	Feedback rapide avant partage du changement.	Le développeur peut oublier de la lancer.
Enforcement CI	Workflow de PR, revue de dépendances, checks requis.	Contrôle reproductible et partagé.	La politique doit être bien configurée et maintenue.

PHRASE À FAIRE RETENIR

Copilot conseille. Les tests vérifient. La CI impose les contrôles décidés par l'équipe.

0.3 Vocabulaire du jour

Dépôt / repository - Ensemble versionné contenant le code, la configuration, les tests et les fichiers de travail du projet.

Instruction de dépôt - Règle persistante donnée à l'assistant IA pour l'ensemble du projet.

Instruction ciblée - Règle appliquée seulement à certains fichiers ou chemins du projet.

Task VS Code - Commande de build, test ou contrôle décrite dans `.vscode/tasks.json` et lançable depuis l'éditeur.

Licence logicielle - Texte juridique qui définit les droits et obligations liés à l'utilisation, la modification et la redistribution d'un logiciel.

SPDX - Standard qui fournit notamment des identifiants courts et normalisés pour désigner des licences.

PR - Pull Request - Proposition de changement soumise à revue avant fusion dans une branche cible.

CI - Continuous Integration - Intégration continue : exécution automatisée de contrôles sur les changements.

Dependency Review - Revue des dépendances introduites ou modifiées par une pull request, notamment vis-à-vis des vulnérabilités et licences.

Merge - Fusion d'une branche ou d'une pull request dans la branche cible.

0.4 Note d'actualisation technique - septembre 2026

POUR LE FORMATEUR

Le déroulé fixe les notions à enseigner. Les exemples de configuration ont été alignés sur les documentations officielles disponibles en septembre 2026. Deux nuances sont importantes : VS Code distingue les instructions toujours actives et les instructions ciblées ; et l'option `deny-licenses` de dependency-review-action est annoncée comme dépréciée pour une possible suppression dans une future version majeure. Le cours la montre parce qu'elle figure explicitement dans le déroulé, tout en présentant l'approche allow-list comme alternative plus durable.

09:00-10:15 - Paramétrage Copilot en environnement contrôlé

OBJECTIF DE LA SÉQUENCE

Transformer des règles d'équipe en contexte persistant pour Copilot, distinguer règles globales, règles ciblées et demandes ponctuelles, et comprendre que des instructions ne constituent pas une barrière de sécurité.

Le déroulé demande les instructions de dépôt, les instructions par type de fichier, les instructions de revue, les limites de l'IA, les bonnes pratiques de prompts et l'usage en pair programming. La séquence doit relier ces éléments au J2 : une règle d'équipe utile est explicite, observable et partagée.

1.1 Pourquoi paramétrer Copilot ?

Sans configuration projet, chaque conversation recommence presque au même point : version de C++, structure du dépôt, commandes de build, règles de sécurité, conventions de nommage, restrictions de dépendances. Le développeur répète le contexte, oublie parfois une règle et obtient des propositions moins cohérentes entre deux personnes.

À DIRE TRÈS SIMPLEMENT

Le paramétrage ne rend pas Copilot plus intelligent. Il lui donne le même panneau de règles avant de commencer à travailler.

1.2 Instructions de dépôt : le socle commun

Pour GitHub Copilot dans VS Code, un fichier `.github/copilot-instructions.md` peut fournir des instructions applicables à l'ensemble du dépôt. On y place les éléments que tout changement doit connaître : architecture, version du langage, commandes de build et de test, exigences de sécurité, règles de gestion des erreurs, conventions de documentation et politique de dépendances.

Bonne information à mettre	Pourquoi	Exemple
Contexte architectural	Évite les propositions qui traversent les mauvaises frontières.	Le module Mission ne doit pas accéder directement au driver moteur.
Version / contraintes C++	Évite une syntaxe ou une bibliothèque non supportée.	Compiler en C++17 ; ne pas supposer C++20.
Commandes de build et test	Permet à l'agent de proposer une validation cohérente.	Build et tests doivent réussir avant toute PR.
Règles safety	Rappelle que certaines décisions nécessitent validation humaine.	Ne pas modifier une règle safety sans ticket et revue dédiée.
Gestion des erreurs	Stabilise les contrats transverses.	Préférer des erreurs explicites aux booléens ambigus.
Dépendances / licences	Empêche les ajouts improvisés dans les suggestions.	Aucune dépendance externe sans validation.

1.3 Exemple pédagogique : `.github/copilot-instructions.md`

Exemple de base à projeter puis commenter ligne par ligne

```
# Project instructions - pallet robot

## Scope and architecture
- The project targets C++17.
- Keep hardware access inside the hardware layer.
- Mission code must not call motor drivers directly.
- Safety decisions require explicit human review.

## C++ rules
- Prefer explicit types at module boundaries.
- Use enum class for closed sets of states or errors.
- Avoid hidden global mutable state.
- Do not introduce a new external dependency without approval.

## Validation
- Build the project before proposing a change as complete.
- Run the unit tests relevant to the modified module.
- State which tests were run and which were not run.
- Never claim a test passed unless you have actual evidence.

## Review
- Highlight changes to public interfaces, units, timeouts and error semantics.
- Call out any uncertainty instead of inventing missing requirements.

## Licences
- Use SPDX identifiers when discussing dependency licences.
- Do not add an unapproved dependency.
```

Pourquoi ces instructions sont meilleures que “écris du bon C++” ?

- Elles décrivent des décisions propres au projet, pas des conseils génériques.
- Elles donnent des éléments contrôlables : version C++, frontières, tests, dépendances.
- Elles demandent explicitement de signaler l’incertitude au lieu de la masquer.
- Elles ne prétendent pas remplacer le compilateur, les tests, la revue ou la CI.

1.4 Instructions par type de fichier ou par chemin

Le déroulé demande des instructions par type de fichier. Dans VS Code, on peut compléter les instructions globales avec des fichiers `*.instructions.md` appliqués à des chemins correspondant à un motif. L’objectif est de ne pas surcharger les règles globales avec des détails qui ne concernent que certains fichiers.

Exemple : `.github/instructions/cpp.instructions.md`

```
---
name: C++ module rules
description: Rules for production C++ source files
applyTo: "src/**/*.cpp,include/**/*.hpp"
---

- Keep module boundaries explicit.
- Do not bypass the safety layer.
- When changing a public type, list impacted consumers.
- Add or adapt tests for changed business rules.
```

Exemple : `.github/instructions/tests.instructions.md`

```
---
name: Test rules
description: Rules for tests
applyTo: "tests/**/*"
---

- Name the behavior under test explicitly.
- Cover boundary values for thresholds.
- Include at least one negative case for each rejection rule.
- Keep tests deterministic.
```

1.5 Instructions de revue : orienter le regard, pas décider à la place du reviewer

Une revue IA est utile si la demande précise les catégories de risque : changement de contrat, unité, timeout, erreur, dépendance, comportement safety, test manquant. Une consigne vague comme “review this code” produit facilement une liste superficielle de style ou de micro-optimisations.

FRAGILE	MIEUX
“Review this PR and tell me if it is good.” Le mot “good” n’a aucun critère ; la réponse peut ignorer les frontières industrielles importantes.	“Review the changes for interface compatibility, unit/timeout semantics, error handling, safety boundary violations, new dependencies, and missing tests. Separate facts from hypotheses. Do not approve the change.”

1.6 Limites : instructions != garantie

Les instructions personnalisées influencent les réponses, mais elles restent du contexte fourni à un modèle non déterministe. Elles ne doivent donc pas être utilisées comme mécanisme d’enforcement. Une règle de sécurité ou de conformité importante doit être vérifiée par du code, des tests, des outils ou une étape humaine obligatoire.

ERREUR À ÉVITER

“C’est écrit dans copilot-instructions.md, donc c’est sécurisé.” Non. Le fichier améliore la cohérence de l’assistance ; il ne bloque pas physiquement une modification interdite.

1.7 Bonnes pratiques de prompts dans un dépôt déjà configuré

Dans le prompt	Pourquoi	Exemple
Objectif précis	Évite une réponse trop large.	Adapter MotionResult pour distinguer timeout et refus.
Fichiers / module concernés	Cadre la zone de changement.	Navigation uniquement ; ne pas modifier Safety.
Critères d’acceptation	Rend le résultat vérifiable.	Les anciens cas restent verts + deux nouveaux tests.
Interdits / contraintes	Évite les raccourcis.	Pas de nouvelle dépendance.
Demande de preuve	Force la séparation entre proposition et validation.	Indique les commandes à lancer et les tests réellement exécutés.

1.8 Pair programming : un rôle pour l'humain, un rôle pour l'IA

En pair programming assisté par IA, le développeur garde la responsabilité du problème, des choix et de la validation. L'IA peut produire une hypothèse, un brouillon de code, une liste de cas de test ou une revue. Le développeur doit rendre chaque étape observable : intention, proposition, vérification, décision.

Étape	Humain	IA
Cadrer	Décrit le besoin, le contexte et la criticité.	Pose éventuellement des questions ou reformule.
Proposer	Choisit le niveau de délégation.	Produit une solution ou des variantes.
Vérifier	Compile, teste, compare au contrat, vérifie les impacts.	Peut aider à chercher des cas manquants.
Décider	Accepte, modifie ou rejette avec justification.	N'est pas l'autorité finale.

1.9 Mini-atelier : améliorer les instructions

CONSIGNE

Le dépôt contient seulement : "Write clean modern C++ and be safe." En groupes, remplacez cette phrase par 8 à 12 instructions spécifiques au robot porte-palette : architecture, C++, safety, tests, erreurs, dépendances et revue. Chaque règle doit être assez précise pour qu'un reviewer puisse demander : "comment vérifie-t-on qu'elle a été respectée ?"

Correction / critères attendus

- Au moins une règle de frontière architecturale.
- Une version C++ explicitement déclarée.
- Une règle sur la vérification et la preuve des tests.
- Une règle interdisant l'ajout silencieux d'une dépendance.
- Une règle sur les changements de contrats, unités, timeouts ou erreurs.
- Une règle demandant de signaler l'incertitude.
- Aucune formulation qui prétend déléguer l'approbation safety à l'IA.

1.10 À retenir

RÈGLE

Les instructions Copilot donnent le contexte durable. Le prompt donne l'objectif du travail courant. La revue, les tests et la CI donnent la preuve.

10:15-11:30 - Paramétrage VS Code : rendre les contrôles faciles à lancer

OBJECTIF DE LA SÉQUENCE

Comprendre le rôle de `.vscode/settings.json` et `.vscode/tasks.json`, créer des tâches claires pour build, tests et contrôle licence, puis définir un parcours local avant PR.

Le déroulé demande settings.json, tasks.json, tâches de build, de test, de scan licence et lancement local avant PR. Le support doit montrer que l'éditeur ne remplace pas la chaîne de build : il offre un point d'entrée commun vers les commandes officielles du projet.

2.1 Pourquoi versionner une partie de la configuration VS Code ?

Une équipe perd du temps si chacun doit mémoriser les mêmes commandes, chemins et options. Une configuration de workspace peut rendre les opérations récurrentes visibles et reproductibles. Le but n'est pas de forcer tous les développeurs à utiliser VS Code : le but est que, pour ceux qui l'utilisent, le bouton ou la tâche appelle les mêmes commandes que la documentation et la CI.

2.2 `.vscode/settings.json` : préférences du workspace

`settings.json` contient des réglages applicables au workspace. Dans un cours industriel, il faut être sélectif : versionner les réglages qui améliorent la cohérence du projet, pas les préférences personnelles de thème ou de taille de police.

Exemple pédagogique minimal - à adapter aux extensions réellement installées

```
{
  "files.insertFinalNewline": true,
  "files.trimTrailingWhitespace": true,
  "editor.formatOnSave": false,
  "C_Cpp.default.cppStandard": "c++17"
}
```

NUANCE

Les paramètres disponibles dépendent du VS Code et des extensions installées. Une règle critique ne doit pas dépendre uniquement d'un réglage local d'éditeur.

2.3 `.vscode/tasks.json` : une façade vers les commandes de projet

Une task VS Code décrit une commande lançable depuis `Terminal > Run Task`. Pour les tâches personnalisées, les types `shell` et `process` sont les formes de base. La configuration `version` courante du schéma est `2.0.0`. Une tâche possède notamment un label, une commande, des arguments, éventuellement un groupe et un problem matcher.

Champ	Rôle	Question formateur
label	Nom visible de la tâche.	Un nouvel arrivant comprend-il quoi lancer ?

Champ	Rôle	Question formateur
type	Mode d'exécution, par exemple shell ou process.	A-t-on besoin des fonctionnalités du shell ?
command	Programme ou commande exécutée.	Est-ce la même commande que dans la CI/documentation ?
args	Arguments séparés.	Les chemins avec espaces restent-ils robustes ?
group	Classe la tâche, par exemple build ou test.	Peut-on déclencher facilement le build par défaut ?
problemMatcher	Interprète la sortie pour afficher des erreurs dans VS Code.	Existe-t-il un matcher pertinent ?

2.4 Exemple de tâches build / test / licence

Le déroulé ne précise pas l'outil de build ni le scanner de licences. L'exemple ci-dessous est donc un canevas pédagogique : les commandes `./scripts/build.sh`, `./scripts/test.sh` et `./scripts/check_licenses.sh` représentent les commandes officielles de l'entreprise. Le point à enseigner est la structure, pas ces noms particuliers.

```
{
  "version": "2.0.0",
  "tasks": [
    {
      "label": "robot: build",
      "type": "shell",
      "command": "./scripts/build.sh",
      "group": { "kind": "build", "isDefault": true },
      "problemMatcher": ["$gcc"]
    },
    {
      "label": "robot: test",
      "type": "shell",
      "command": "./scripts/test.sh",
      "group": "test",
      "problemMatcher": []
    },
    {
      "label": "robot: licence check",
      "type": "shell",
      "command": "./scripts/check_licenses.sh",
      "problemMatcher": []
    },
    {
      "label": "robot: pre-PR",
      "dependsOrder": "sequence",
      "dependsOn": [
        "robot: build",
        "robot: test",
        "robot: licence check"
      ],
      "problemMatcher": []
    }
  ]
}
```

2.5 Pourquoi une tâche `pre-PR` ?

Une tâche composée transforme une checklist mentale en séquence visible. Le développeur n'a plus à se souvenir des trois commandes dans le bon ordre. Cela ne remplace toujours pas la CI, mais réduit les aller-retours : une erreur détectée localement coûte moins cher qu'un échec après ouverture de la pull request.

Chaîne à dessiner au tableau

Étape	Action	Si échec
1	Build local	Corriger avant de poursuivre.
2	Tests locaux	Identifier le scénario cassé ; ne pas ouvrir la PR comme "prête".
3	Contrôle licence	Faire valider ou retirer la dépendance.
4	Ouverture de PR	Décrire les changements et preuves locales.
5	CI	Rejouer les contrôles dans un environnement partagé.

2.6 Attention à la divergence local / CI

Le piège classique est de créer une tâche locale qui ne correspond plus à la CI. Par exemple, la tâche locale compile en mode Debug tandis que la CI compile en Release avec des warnings traités en erreurs. Le développeur croit avoir "fait le build", mais il n'a pas reproduit le contrôle qui bloque la PR.

RÈGLE DE CONCEPTION

Les tâches VS Code doivent appeler autant que possible les scripts ou commandes canoniques du dépôt. Éviter de dupliquer la logique de build dans trois endroits différents.

2.7 Atelier : construire le parcours local

CONSIGNE

À partir des trois contrôles demandés par le déroulé - build, tests, scan licence - dessinez un `tasks.json` contenant trois tâches indépendantes et une tâche `pre-PR`. Ajoutez une réponse à la question : quelles commandes réelles de votre entreprise remplaceraient les scripts fictifs de l'exemple ?

Correction attendue

- Quatre labels clairs.
- Build, test et licence exécutables séparément.
- Une tâche composée avec ordre défini.
- Aucune logique métier cachée dans VS Code si elle existe déjà dans un script de dépôt.
- Identification explicite des commandes réelles à brancher dans le canevas.

11:30-12:30 - Politique de licences logicielles

OBJECTIF DE LA SÉQUENCE

Donner aux participants un vocabulaire minimal pour classer des licences, comprendre le rôle de SPDX et appliquer une règle interne : aucune nouvelle dépendance non validée.

Cette séquence ne transforme pas les développeurs en juristes. Elle doit leur apprendre à reconnaître qu'une dépendance est aussi un engagement juridique, à identifier correctement sa licence et à déclencher la validation interne lorsque la politique l'exige.

3.1 Une dépendance n'est jamais "gratuite" au sens organisationnel

Même quand une bibliothèque ne coûte rien à télécharger, son adoption crée des obligations : compatibilité technique, maintenance, vulnérabilités, support, contraintes de licence, documentation et parfois obligations de redistribution. La politique de licence est donc une partie de la politique de dépendances.

3.2 Trois grandes familles pédagogiques

Famille	Idée simple	Conséquence pratique à enseigner	Exemples typiques à vérifier
Permissive	Autorise généralement une réutilisation large avec obligations relativement limitées.	Souvent plus simple à intégrer, mais il faut respecter notices, attribution ou conditions précises.	MIT, BSD, Apache - la licence exacte doit être vérifiée.
Copyleft	Attache des obligations de partage à certaines redistributions ou œuvres dérivées selon la licence.	Peut être incompatible avec la politique d'un produit propriétaire ; validation juridique nécessaire.	Famille GPL - ne pas généraliser sans lire la version et le contexte.
Faible copyleft	Copyleft avec portée souvent plus ciblée que le copyleft fort.	Peut être acceptable dans certains modes d'intégration et non dans d'autres ; décision juridique/organisationnelle.	Famille LGPL/MPL selon le cas ; toujours vérifier.

IMPORTANT

Ces familles servent à enseigner une logique de tri. Elles ne remplacent jamais l'analyse du texte de licence, de sa version, du mode de distribution et de la politique juridique de l'entreprise.

3.3 SPDX : nommer les licences sans ambiguïté

SPDX fournit une liste de licences avec des identifiants courts normalisés. Au lieu d'écrire "licence Apache" ou "GPL", on utilise un identifiant précis tel que `Apache-2.0` ou `GPL-3.0-only` lorsque c'est celui qui correspond réellement au composant. Cette précision permet aux outils d'automatiser une partie des contrôles.

```
// Exemple d'identifiant SPDX dans un fichier source
// SPDX-License-Identifier: MIT
```

3.4 Pourquoi “licence permissive” ne veut pas dire “aucune obligation” ?

Le mot permissif décrit un niveau général de liberté, pas l’absence de conditions. Une licence peut exiger de conserver un copyright, un texte de licence ou une notice. L’équipe doit donc distinguer : “notre politique autorise cette licence” et “nous avons satisfait toutes ses obligations”.

3.5 La règle interne du déroulé : aucune dépendance non validée

Cette règle est volontairement simple : le développeur ne décide pas seul qu’une dépendance externe est acceptable. Il peut proposer la dépendance et fournir les informations nécessaires ; la validation suit le processus de l’entreprise.

Information à fournir	Pourquoi
Nom + version	Identifier exactement le composant évalué.
Source officielle	Éviter les paquets homonymes ou non officiels.
Licence SPDX	Permet le contrôle automatique et la lecture de la politique.
Usage prévu	Une dépendance de build, de test ou runtime n’a pas toujours le même impact.
Alternative sans dépendance	Vérifie si le coût est justifié.
Owner / mainteneur interne	Quelqu’un doit assumer son suivi.
Validation juridique ou sécurité si nécessaire	Sépare la décision de développement de la décision de conformité.

3.6 Atelier : classer des dépendances fictives

CONSIGNE

Pour chaque dépendance fictive, classez-la : A = potentiellement acceptable selon politique, B = nécessite validation juridique explicite, C = informations insuffisantes / ne pas intégrer. Vous devez justifier la décision par les données disponibles, pas par la réputation de la licence.

Dépendance fictive	Licence déclarée	Usage	Données disponibles
TelemetryLite 2.1	MIT	Runtime	Source, version et notice disponibles.
PathPlannerPro 5.0	GPL-3.0-only	Runtime lié au produit	Licence identifiée, usage propriétaire prévu.
TestHelpers 1.4	Apache-2.0	Tests uniquement	Source et licence identifiées.
FastMathX	“open source”	Runtime	Aucun identifiant SPDX ni fichier de licence trouvé.
LegacyComms 3.8	LGPL-2.1-only	Runtime	Mode de liaison et distribution non documentés dans le ticket.

Correction pédagogique

- TelemetryLite : peut entrer dans la catégorie “potentiellement acceptable”, sous réserve de la politique réelle et du respect des notices.
- PathPlannerPro : ne pas intégrer automatiquement ; déclencher la validation juridique/politique.
- TestHelpers : usage limité aux tests ne supprime pas toute obligation ; vérifier la politique mais le contexte est mieux cadré.
- FastMathX : classer “informations insuffisantes”. “Open source” n’est pas un identifiant de licence.
- LegacyComms : informations insuffisantes pour trancher ; le mode d’intégration et de redistribution doit être clarifié.

3.7 À retenir

RÈGLE

Identifier n’est pas autoriser. Un identifiant SPDX aide à nommer la licence ; la politique interne et, si nécessaire, la validation juridique déterminent si la dépendance peut être utilisée.

13:30-15:30 - GitHub Dependency Review : contrôler les dépendances dans la PR

OBJECTIF DE LA SÉQUENCE

Comprendre le rôle d'un contrôle automatisé sur les dépendances ajoutées ou modifiées par une pull request, interpréter les contrôles de vulnérabilité et de licence, puis étudier un workflow.

Le déroulé demande le scan automatique en pull request, le blocage des licences interdites, le blocage des vulnérabilités, la règle deny-licenses et l'obligation de validation avant merge. Cette séquence doit relier la politique humaine du matin à un mécanisme reproductible de CI.

4.1 Qu'est-ce que la Dependency Review ?

La revue de dépendances compare les dépendances entre l'état de référence et le changement proposé par une pull request. Le but pédagogique est de répondre à une question simple : "qu'est-ce que cette PR fait entrer dans notre chaîne logicielle ?" Le contrôle peut ensuite échouer si le changement introduit une vulnérabilité au-dessus du seuil configuré ou une licence non conforme à la politique.

4.2 Ce que ce contrôle ne fait pas

- Il ne remplace pas l'évaluation d'architecture : une dépendance "autorisée" peut être inutile ou trop lourde.
- Il ne remplace pas la validation juridique pour les cas ambigus.
- Il ne garantit pas qu'une bibliothèque est sans vulnérabilité future.
- Il ne prouve pas qu'une licence a été correctement identifiée par toutes les sources possibles.
- Il ne valide pas le code métier du robot.

4.3 Anatomie d'un workflow pédagogique

Exemple correspondant au déroulé - à adapter à la politique réelle

```
name: Dependency Review

on:
  pull_request:

permissions:
  contents: read

jobs:
  dependency-review:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout repository
        uses: actions/checkout@v6

      - name: Dependency Review
        uses: actions/dependency-review-action@v4
        with:
          fail-on-severity: high
          deny-licenses: GPL-3.0-only, AGPL-3.0-only
```

Lecture ligne par ligne

Élément	Sens pédagogique
on: pull_request	Le contrôle se déclenche sur une proposition de changement.
permissions: contents: read	Le job reçoit le minimum de permission montré dans ce canevas.
checkout	Rend le dépôt disponible au job lorsque nécessaire.
dependency-review-action@v4	Action officielle chargée d'analyser les modifications de dépendances.
fail-on-severity: high	Le job échoue pour des vulnérabilités au niveau configuré ou supérieur.
deny-licenses	Liste des licences que le workflow refuse selon la politique du cours.

4.4 Nuance 2026 : `deny-licenses` est déprécié

Le déroulé demande explicitement d'enseigner `deny-licenses`, donc il est présenté. La documentation actuelle du dépôt `actions/dependency-review-action` indique cependant que cette option est dépréciée et peut être supprimée dans une future version majeure. Il faut donc enseigner la notion de politique de licence indépendamment de cette clé particulière.

> **MESSAGE FORMATEUR**
> Ne faites pas mémoriser `deny-licenses` comme une vérité éternelle. Faites mémoriser : "la CI compare les licences aux règles de l'organisation". La syntaxe de l'outil peut évoluer.

4.5 Approche allow-list : autoriser explicitement

Une politique par liste d'autorisations répond à une question différente : au lieu d'énumérer seulement ce qui est interdit, on énumère les licences acceptées par la politique. Cette approche peut être plus stricte et éviter qu'une licence nouvelle ou inconnue soit acceptée simplement parce qu'elle n'était pas dans la deny-list.

Exemple pédagogique de stratégie allow-list - la vraie liste doit venir de la politique juridique interne

```
- name: Dependency Review
  uses: actions/dependency-review-action@v4
  with:
    fail-on-severity: high
    allow-licenses: MIT, BSD-2-Clause, BSD-3-Clause, Apache-2.0
```

4.6 Vulnérabilités : comprendre le seuil

`fail-on-severity` exprime un seuil de blocage. Il faut faire raisonner les participants : un seuil bas augmente la sensibilité mais peut bloquer davantage ; un seuil haut laisse passer des vulnérabilités moins sévères. Ce choix appartient à la politique sécurité du produit, pas au développeur isolé ni à l'IA.

Seuil pédagogique	Interprétation simple	Question de gouvernance
critical	Bloquer uniquement les alertes les plus graves.	Acceptons-nous les high sans action immédiate ?
high	Bloquer high et critical.	Que faisons-nous des moderate ?
moderate	Bloquer moderate, high, critical.	Le bruit reste-t-il gérable ?
low	Politique très stricte sur toute vulnérabilité détectée.	Avons-nous un processus d'exception ?

4.7 De l'échec de CI à la décision

Un check rouge n'est pas la fin du raisonnement. Il doit déclencher une action explicite : supprimer la dépendance, choisir une autre version, demander une exception documentée si le processus l'autorise, changer de bibliothèque ou corriger la configuration si le signal est faux. La règle "obligation de validation avant merge" signifie que la PR ne doit pas être fusionnée tant que les contrôles requis ne sont pas satisfaits selon la gouvernance du dépôt.

Workflow décisionnel à dessiner

Résultat Dependency Review	Action
Aucune nouvelle dépendance problématique	Continuer avec les autres checks de la PR.
Vulnérabilité au-dessus du seuil	Changer la version/composant ou passer par le processus d'exception sécurité.
Licence interdite / non autorisée	Retirer ou remplacer la dépendance ; sinon validation juridique formelle selon procédure.
Licence inconnue	Bloquer la décision et obtenir l'information fiable.
Check non exécuté / erreur technique	Ne pas considérer la PR comme validée ; diagnostiquer le workflow.

4.8 Atelier : étude d'un workflow

CONSIGNE

Projetez le workflow ci-dessous. Les participants doivent répondre à cinq questions : quand s'exécute-t-il ? qu'est-ce qui peut faire échouer la PR ? quelle règle de licence est utilisée ? quel est le risque d'une licence inconnue ? quelle amélioration proposer si la politique de l'entreprise préfère une liste d'autorisations ?

```
name: Dependency Review
on: [pull_request]

permissions:
  contents: read

jobs:
  review:
```

```
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v6
      - uses: actions/dependency-review-action@v4
        with:
          fail-on-severity: moderate
          deny-licenses: GPL-3.0-only, AGPL-3.0-only
```

Correction détaillée

- Le workflow est déclenché par une pull request.
- Une vulnérabilité de sévérité moderate ou supérieure peut faire échouer le contrôle.
- La politique présentée est une deny-list de deux identifiants SPDX.
- Une licence absente de la deny-list n'est pas automatiquement "approuvée juridiquement" : l'outil ne remplace pas la politique et la validation.
- Si l'organisation souhaite n'accepter qu'un ensemble fermé, remplacer la logique de deny-list par une allow-list approuvée, après vérification de la compatibilité de configuration.

4.9 Erreurs fréquentes

Erreur	Pourquoi c'est dangereux	Mieux
"Le check est vert donc la dépendance est bonne."	Le check ne juge pas l'utilité architecturale ni tous les aspects juridiques.	Séparer conformité automatisée, revue technique et validation juridique.
Copier une liste de licences depuis Internet.	La politique peut ne pas correspondre à l'entreprise.	Utiliser la liste officiellement validée.
Baisser le seuil pour faire passer la PR.	On transforme une politique de sécurité en obstacle contournable.	Passer par le processus d'exception documenté.
Ignorer une erreur du workflow.	Absence de preuve de contrôle.	Traiter la panne de CI comme un contrôle non effectué.
Laisser Copilot choisir la licence acceptable.	L'IA n'est pas l'autorité juridique.	Copilot peut aider à collecter ; l'organisation décide.

4.10 À retenir

RÈGLE

Une politique devient crédible quand elle produit une décision reproductible dans la PR : passer, bloquer, ou déclencher une exception explicitement gouvernée.

15:30-17:30 - Atelier final : simulation complète sur un ticket

OBJECTIF DE LA SÉQUENCE

Faire appliquer le flux complet de J1 à J3 : cadrer un ticket, utiliser Copilot avec des instructions, modifier du C++ de manière contrôlée, exécuter les tâches locales, vérifier les dépendances/licences, préparer une PR, interpréter la CI et réaliser une revue humaine.

Le déroulé indique que les participants doivent traiter un ticket complet. Le ticket ci-dessous est un scénario pédagogique construit autour du robot porte-palette pour réutiliser les contraintes déjà travaillées sans introduire un nouveau domaine.

5.1 Ticket pédagogique

TICKET ROBOT-247 - Ajouter un motif explicite de refus de déplacement

Aujourd'hui, `canStartMotion()` retourne seulement un booléen. La supervision ne sait pas pourquoi le robot refuse un déplacement. Modifier le contrat pour retourner un résultat explicite sans contourner la couche Safety. Ajouter les tests de batterie, freins, capteur frontal, erreur critique et état Idle. Ne pas ajouter de dépendance externe. Préparer la PR avec les preuves de build et de test.

5.2 Critères d'acceptation

- Le code compile avec la version C++ prévue par le projet.
- Le résultat distingue au minimum les motifs demandés par le ticket.
- La règle de batterie traite correctement la valeur frontière.
- Les règles safety restent dans les frontières prévues ; aucune modification non demandée de la couche Safety.
- Les tests couvrent succès et refus.
- Aucune dépendance externe n'est introduite.
- La PR indique les commandes locales exécutées et leurs résultats.
- La revue IA ne remplace pas la revue humaine.

5.3 Étape 1 - Lire les instructions avant de prompter

Le participant commence par lire ou faire afficher les instructions du dépôt. L'exercice doit éviter le réflexe "je demande du code immédiatement". Il faut d'abord identifier les contraintes déjà définies : version C++, architecture, tests, dépendances et règles safety.

QUESTION AU GROUPE

Quelles informations du ticket sont déjà couvertes par les instructions du dépôt ? Quelles informations restent spécifiques au ticket et doivent donc être dans le prompt ?

5.4 Étape 2 - Construire le prompt de travail

```
Context: pallet robot C++ project. Follow the repository instructions.
```

```
Task: ROBOT-247. Replace the ambiguous boolean result of canStartMotion()
with an explicit result that lets supervision understand why motion is refused.

Constraints:
- Do not modify the Safety layer behavior.
- Do not add an external dependency.
- Cover battery threshold, brakes, front sensor, critical error and Idle state.
- Keep the change small and list impacted consumers before editing.

Expected output:
1. Proposed interface change.
2. Impacted files/consumers.
3. Implementation.
4. Tests including boundary and negative cases.
5. Commands to build and test.
6. Risks or assumptions that still require human confirmation.
```

5.5 Étape 3 - Revoir la proposition avant de l'appliquer

Question de revue	Exemple de réponse attendue
Le contrat est-il plus explicite ?	Oui si le résultat porte un motif nommé plutôt qu'un booléen ambigu.
Une couche est-elle contournée ?	Non : la vérification respecte les frontières du projet.
Une dépendance est-elle ajoutée ?	Non pour ce ticket.
La frontière batterie est-elle claire ?	Le test à 20 et juste en dessous évite une erreur > / >=.
Les tests prouvent-ils tous les refus ?	Chaque motif demandé a un cas négatif.
L'IA a-t-elle inventé une exigence ?	Toute hypothèse doit être signalée et décidée par l'humain.

5.6 Exemple de code cible - pédagogique

motion_decision.hpp

```
#pragma once

enum class MotionRefusal {
    None,
    BatteryTooLow,
    BrakesUnavailable,
    FrontSensorUnavailable,
    CriticalErrorPresent,
    NotIdle
};

struct RobotState {
    int batteryPercent;
    bool brakesAvailable;
    bool frontSensorAvailable;
    bool criticalErrorPresent;
    bool idle;
};

struct MotionDecision {
    bool allowed;
```

```
    MotionRefusal refusal;
};
```

motion_eligibility.hpp

```
#pragma once
#include "motion_decision.hpp"

class MotionEligibility {
public:
    MotionDecision evaluate(const RobotState& state) const {
        if (state.criticalErrorPresent) {
            return {false, MotionRefusal::CriticalErrorPresent};
        }
        if (!state.idle) {
            return {false, MotionRefusal::NotIdle};
        }
        if (state.batteryPercent < 20) {
            return {false, MotionRefusal::BatteryTooLow};
        }
        if (!state.brakesAvailable) {
            return {false, MotionRefusal::BrakesUnavailable};
        }
        if (!state.frontSensorAvailable) {
            return {false, MotionRefusal::FrontSensorUnavailable};
        }
        return {true, MotionRefusal::None};
    }
};
```

5.7 Tests autonomes à montrer

main.cpp - démonstration exécutable C++17

```
#include <cassert>
#include <iostream>
#include "motion_eligibility.hpp"

int main() {
    MotionEligibility rule;

    assert(rule.evaluate({20, true, true, false, true}).allowed);

    auto battery = rule.evaluate({19, true, true, false, true});
    assert(!battery.allowed);
    assert(battery.refusal == MotionRefusal::BatteryTooLow);

    auto brakes = rule.evaluate({80, false, true, false, true});
    assert(brakes.refusal == MotionRefusal::BrakesUnavailable);

    auto sensor = rule.evaluate({80, true, false, false, true});
    assert(sensor.refusal == MotionRefusal::FrontSensorUnavailable);

    auto critical = rule.evaluate({80, true, true, true, true});
    assert(critical.refusal == MotionRefusal::CriticalErrorPresent);

    auto state = rule.evaluate({80, true, true, false, false});
    assert(state.refusal == MotionRefusal::NotIdle);

    std::cout << "ROBOT-247: tests passed\n";
```

```
}
```

5.8 Étape 4 - Exécuter le parcours local

Les participants lancent la tâche `pre-PR` ou l'équivalent réel du projet. Le formateur exige une preuve simple : sortie du build, sortie des tests, sortie du contrôle licence. Une phrase "ça devrait passer" ne vaut pas une preuve.

Contrôle	Preuve à conserver dans l'exercice	Décision si échec
Build	Commande + code retour / sortie sans erreur.	Corriger le code ou la configuration.
Tests	Liste ou sortie des tests exécutés.	Corriger le comportement ou le test après analyse.
Licence	Aucune nouvelle dépendance ou scan conforme.	Retirer, remplacer ou déclencher validation.

5.9 Étape 5 - Préparer la PR

Exemple de description de PR

```
## ROBOT-247 - explicit motion refusal

### What changed
- Replaced the ambiguous boolean motion eligibility result with MotionDecision.
- Added explicit refusal reasons for battery, brakes, front sensor, critical error and state.

### Impact
- Consumers of canStartMotion() must read MotionDecision.
- Safety behavior was not modified.
- No external dependency added.

### Local validation
- Build: PASSED
- Tests: PASSED
- Licence check: PASSED / no new dependency

### Review focus
- Verify error semantics and consumer migration.
- Verify that the Safety boundary is unchanged.
```

5.10 Étape 6 - Simuler la CI et la Dependency Review

Comme le ticket n'ajoute pas de dépendance, la revue de dépendances doit normalement ne pas signaler de nouvel ajout. Le formateur peut ensuite injecter une variante : "un participant propose une petite bibliothèque externe pour sérialiser l'erreur". Le groupe doit immédiatement arrêter la modification et appliquer la politique de dépendances avant de discuter du code.

Variante d'incident pédagogique

ÉVÉNEMENT

Copilot propose d'ajouter `magic-json-helper` pour sérialiser le résultat. La bibliothèque n'existe pas dans la liste de dépendances validées et sa licence n'est pas fournie dans le ticket.

RÉPONSE ATTENDUE

Ne pas ajouter la dépendance. Identifier le besoin réel, chercher si le projet possède déjà un mécanisme, collecter les informations de licence/version si la dépendance reste utile, puis passer par le processus de validation. L'argument "Copilot l'a proposée" n'a aucune valeur d'approbation.

5.11 Étape 7 - Revue IA puis revue humaine

La revue IA peut préparer le travail : détecter un consommateur oublié, un test de frontière absent, une incohérence de nom ou une erreur de logique. La revue humaine tranche : la sémantique des refus est-elle conforme au besoin ? L'ordre des règles est-il correct ? La supervision doit-elle afficher tous les motifs ? L'impact sur les autres modules a-t-il été traité ?

Suggestion de revue	Vérification humaine	Décision possible
"Fusionner CriticalErrorPresent et NotIdle en Unavailable."	Le ticket demande-t-il des motifs distincts ? La supervision en a-t-elle besoin ?	Refuser si cela détruit l'information métier.
"Passer la batterie minimale à > 20."	Le comportement attendu à 20 est-il défini ?	Refuser si le test frontière prouve >= 20.
"Ajouter une bibliothèque pour mapper les erreurs."	Existe-t-elle déjà ? licence ? coût ? nécessité ?	Refuser ou déclencher processus dépendance.
"Ajouter un test de critical error."	Le cas est dans les critères d'acceptation ?	Accepter : améliore la preuve.

5.12 Grille d'évaluation de l'atelier final

Critère	0 - insuffisant	1 - partiel	2 - acquis
Cadrage	Prompt vague ou hors contraintes.	Contexte présent mais critères incomplets.	Prompt précis, contraintes et preuves demandées.
Respect architecture	Frontière contournée.	Doute non traité.	Frontières respectées et impacts identifiés.
Code C++	Ne compile pas / contrat ambigu.	Fonctionne mais manque de clarté.	Contrat explicite et cohérent.
Tests	Absents.	Cas nominaux seulement.	Frontières + refus couverts.
Dépendances/licences	Ajout non validé.	Règle connue mais preuve incomplète.	Politique appliquée et preuve conservée.
Pré-PR	Contrôles non exécutés.	Une partie seulement.	Build + tests + licence exécutés.
PR / traçabilité	Description vague.	Résumé sans preuves.	Impact, preuves, risques et focus de revue.
Revue humaine	Accepte l'IA sans vérifier.	Vérification partielle.	Suggestion, vérification et décision séparées.

5.13 Debrief de fin d'atelier

- À quel moment l'IA vous a-t-elle réellement fait gagner du temps ?
- À quel moment un contrôle déterministe était-il plus important qu'un prompt ?
- Quelle erreur aurait pu passer si vous aviez seulement demandé "review my code" ?
- Quelle information doit absolument figurer dans le dépôt plutôt que rester dans la tête d'un développeur ?
- Quel contrôle local mérite d'être automatisé dans la CI ?
- Quelle décision n'aurait jamais dû être déléguée à l'IA ?

5.14 Synthèse J3

SYNTHÈSE

Le flux cible du cours est maintenant complet : règles du dépôt -> prompt cadré -> proposition IA -> code contrôlé -> build/tests/licence en local -> pull request -> Dependency Review et CI -> revue humaine -> merge seulement lorsque les contrôles obligatoires sont satisfaits.

6. Démonstration complète et exécutable - mini dépôt J3

Cette démonstration réunit les fichiers nécessaires à l'atelier dans une petite structure autonome. Elle ne prétend pas reproduire le build industriel réel. Elle sert à montrer comment les éléments du jour s'alignent : instructions IA, code, test, tâches locales et workflow de PR.

6.1 Arborescence

```
j3-demo/
├── .github/
│   ├── copilot-instructions.md
│   ├── instructions/
│   │   └── cpp.instructions.md
│   └── workflows/
│       └── dependency-review.yml
├── .vscode/
│   ├── settings.json
│   └── tasks.json
├── include/
│   ├── motion_decision.hpp
│   └── motion_eligibility.hpp
└── main.cpp
```

6.2 Compilation directe

```
g++ -std=c++17 -Wall -Wextra -pedantic -Iinclude main.cpp -o j3_demo
./j3_demo

# Résultat attendu :
ROBOT-247: tests passed
```

6.3 Ce qu'il faut faire observer

- Le code peut être compilé sans dépendance externe.
- Les tests sont de simples assertions pour garder la démonstration autonome.
- La configuration VS Code doit pointer vers les commandes réelles du dépôt si on l'utilise en production.
- Le workflow de Dependency Review agit sur la PR ; il ne "teste" pas le comportement C++.
- Le fichier d'instructions Copilot documente les règles, mais seul le build/test/CI produit la preuve technique.

Annexe A - Canevas `.github/copilot-instructions.md`

```
# Project instructions

## Project context
- ...

## Architecture
- ...

## C++
- ...

## Safety
- ...

## Tests and validation
- ...

## Dependencies and licences
- ...

## Review
- ...
```

Annexe B - Canevas d’instruction ciblée

```
---
name: ...
description: ...
applyTo: "..."
---

- Rule 1
- Rule 2
- Rule 3
```

Annexe C - Checklist pré-PR

- Le ticket et ses critères d’acceptation ont été relus.
- Les instructions du dépôt ont été prises en compte.
- Le changement reste dans le périmètre demandé.
- Les frontières architecturales sont respectées.
- Le build local a réellement été exécuté.
- Les tests pertinents ont réellement été exécutés.
- Tout ajout de dépendance a suivi la politique de validation.
- Le contrôle licence local a été exécuté si le projet le prévoit.
- La PR indique impacts, preuves et risques connus.
- Aucune affirmation “testé / validé” n’est faite sans preuve.

Annexe D - Fiche de proposition de dépendance

Champ	À renseigner
Nom / version	
Source officielle	
Usage prévu	
Licence SPDX	
Type d'usage : build/test/runtime	
Alternative déjà présente dans le dépôt ?	
Vulnérabilités connues vérifiées ?	
Owner interne	
Validation sécurité	
Validation juridique	
Décision finale	

Annexe E - Glossaire J3

Terme	Définition simple
Custom instructions	Instructions persistantes fournies à l'assistant pour guider ses réponses dans un périmètre donné.
Repository-wide	À l'échelle du dépôt ; s'applique au projet dans son ensemble.
Path-specific	Instruction ciblée sur des fichiers correspondant à un chemin ou motif.
Workspace	Dossier ou ensemble de dossiers ouverts comme environnement de travail dans VS Code.
Task	Commande déclarée et lançable depuis VS Code.
Problem matcher	Configuration qui transforme la sortie d'un outil en diagnostics affichables dans VS Code.
Licence permissive	Famille pédagogique de licences généralement moins contraignantes sur la redistribution, sans signifier absence d'obligations.
Copyleft	Principe de licence qui impose certaines obligations de partage ou de maintien de libertés dans des conditions définies par la licence.
Faible copyleft	Copyleft dont la portée est souvent plus limitée, selon la licence et le mode d'intégration.
SPDX	Standard et liste d'identifiants permettant de désigner précisément les licences et exceptions.
Dependency Review	Analyse des changements de dépendances introduits par une pull request.

Terme	Définition simple
Vulnérabilité	Faiblesse de sécurité connue ou potentielle susceptible d'être exploitable.
Pull Request	Proposition de changement soumise à revue avant fusion.
CI	Exécution automatisée de contrôles sur les changements intégrés ou proposés.
Merge	Fusion du changement dans la branche cible.
Allow-list	Liste de valeurs explicitement autorisées.
Deny-list	Liste de valeurs explicitement interdites.

Annexe F - Questions de relance formateur

Moment	Question	But
Copilot	"Cette règle doit-elle être dans le prompt ou dans le dépôt ?"	Distinguer contexte durable et tâche ponctuelle.
Instructions	"Comment prouver que la règle est respectée ?"	Faire émerger la limite de la guidance.
VS Code	"Cette task appelle-t-elle la vraie commande du projet ?"	Éviter la duplication de chaîne de build.
Licences	"SPDX nous dit-il si nous avons le droit d'utiliser le composant ?"	Distinguer identification et autorisation.
Dependency Review	"Un check vert suffit-il à approuver la dépendance ?"	Distinguer automatisation et gouvernance.
CI	"Que signifie une erreur du workflow ?"	Faire comprendre : contrôle non effectué, donc pas de preuve.
Atelier final	"Quelle décision reste humaine ici ?"	Réaffirmer la responsabilité.

Annexe G - Erreurs fréquentes de J3

Erreur	Correction à enseigner
Mettre toutes les règles dans un prompt énorme.	Mettre le durable dans les instructions de dépôt et garder le ticket dans le prompt.
Croire que Copilot respecte toujours les instructions.	Les considérer comme guidance ; vérifier avec outils et revue.
Versionner des préférences VS Code personnelles.	Ne versionner que ce qui sert le projet.
Dupliquer les commandes de build dans tasks.json et la CI.	Appeler les mêmes scripts canoniques.
Considérer SPDX comme une autorisation.	SPDX identifie ; la politique décide.
Ajouter une dépendance puis demander la validation après coup.	Valider avant intégration.
Contourner un check de CI pour gagner du temps.	Utiliser le processus d'exception gouverné.
Interpréter un check vert comme validation du code métier.	Chaque check couvre un risque spécifique.

Annexe H - Auto-évaluation de fin de J3

Je suis capable de...	Oui	À revoir
Expliquer la différence entre instruction de dépôt et prompt ponctuel.		
Créer un canevas `.github/copilot-instructions.md`.		
Décrire le rôle d'une instruction ciblée.		
Expliquer le rôle de `.vscode/tasks.json`.		
Organiser un parcours local build -> tests -> licence avant PR.		
Différencier licence permissive, copyleft et faible copyleft au niveau pédagogique.		
Expliquer à quoi sert un identifiant SPDX.		
Décrire le rôle de GitHub Dependency Review.		
Expliquer ce que signifie `fail-on-severity`.		
Expliquer pourquoi une politique de licence ne doit pas dépendre seulement de l'IA.		
Traiter un ticket complet avec preuves de validation.		
Séparer suggestion IA, vérification technique et décision humaine.		

Annexe I - Sources techniques officielles vérifiées

Le périmètre pédagogique de J3 provient du déroulé fourni par le formateur. Les points d'outillage suivants ont été vérifiés dans les documentations officielles disponibles en septembre 2026 :

- GitHub Docs - repository custom instructions for GitHub Copilot : `.github/copilot-instructions.md` et instructions ciblées `.github/instructions/*.instructions.md`.
- Visual Studio Code Docs - custom instructions : instructions toujours actives et ciblées ; les instructions guident le modèle mais ne constituent pas un mécanisme déterministe d'enforcement.
- Visual Studio Code Docs - Tasks : tâches workspace dans `.vscode/tasks.json`, schéma 2.0.0, types de tâches personnalisées `shell` / `process`.
- SPDX - License List et identifiants SPDX pour nommer les licences de manière standardisée.
- GitHub Dependency Review Action - analyse des dépendances introduites en PR, seuil `fail-on-severity`, politiques de licences ; `deny-licenses` annoncé comme déprécié dans le dépôt de l'action.

Annexe J - Checklist de préparation du formateur

- Préparer un dépôt ou dossier pédagogique contenant `.github`, `.vscode`, `include` et un `main.cpp`.
- Verifier que le compilateur C++17 utilisé en salle accepte la démonstration.
- Préparer une version volontairement vague de `copilot-instructions.md` pour l'atelier de correction.

- Préparer un tasks.json avec une erreur de commande pour montrer la différence entre configuration et preuve.
- Préparer cinq fiches de dépendances fictives pour l'atelier licences.
- Projeter un workflow Dependency Review et faire lire chaque ligne par le groupe.
- Préparer une variante de PR qui introduit une dépendance non validée.
- Préparer le ticket ROBOT-247 et la grille d'évaluation.
- Insister sur la dépréciation de `deny-licenses` sans sortir du périmètre du déroulé.
- Finir par la chaîne complète guidance -> local -> PR/CI -> revue humaine.

J3 - ÉDITION ÉTENDUE

Copilot, VS Code, licences, CI et mise en pratique complète

Approfondissements sur l’environnement contrôlé, les preuves et le flux de pull request

Objet de cette extension

Cette édition étendue développe le J3 sans changer son périmètre : instructions Copilot, tâches VS Code, politique de licences, Dependency Review et simulation finale. Les ajouts renforcent la compréhension de la chaîne guidance -> vérification locale -> CI -> revue humaine.

Principe de densification

Le contenu existant est conservé comme socle. Les pages suivantes approfondissent chaque séquence avec des explications supplémentaires, des variantes de démonstration, des contre-exemples, des questions de groupe, des exercices corrigés et des mises en situation. L'objectif n'est pas d'ajouter des pages pour elles-mêmes : chaque ajout doit aider le formateur à expliquer, faire pratiquer, faire verbaliser ou contrôler une compétence du déroulé pédagogique.

Mode d’utilisation conseillé

- Lire l'approfondissement correspondant avant la séquence, puis sélectionner les exemples adaptés au niveau réel du groupe.
- Ne pas tout réciter : utiliser les blocs comme réserve d'explications, de questions et de contre-exemples.
- Conserver le fil rouge du robot porte-palette pour relier les notions et éviter l'impression d'une collection de recettes.
- Faire distinguer systématiquement ce que l'IA propose, ce qu'un outil déterministe vérifie, et ce qu'un humain décide.
- Revenir aux productions attendues du déroulé : cartographies, checklists, fichiers d'instructions, contrats, tâches locales et dossier de ticket.

Approfondissement 1 - Guidance, vérification locale et enforcement CI

À DIRE TRÈS SIMPLEMENT
Une règle écrite conseille, une commande locale vérifie, la CI peut imposer le contrôle à toute l'équipe.

Explication approfondie. Le J3 doit faire comprendre une progression de force. Les instructions de Copilot réduisent les propositions incohérentes, mais elles sont interprétées par un modèle. Une tâche locale exécute une commande déterministe, mais un développeur peut l'oublier. Une CI associée à la pull request rejoue les contrôles de façon partagée et peut bloquer l'intégration selon la configuration. Ces trois niveaux sont complémentaires.

Faire construire une matrice avec chaque règle du projet et le meilleur niveau de contrôle. Exemple : 'C++17' peut être rappelé dans les instructions et imposé par le build. 'Aucune dépendance externe sans validation' peut être rappelé dans Copilot et vérifié par la Dependency Review. 'Ne pas contourner Safety' demande souvent revue d'architecture et tests ; le simple fichier d'instructions ne suffit pas.

Application au robot porte-palette. Pour ROBOT-247, Copilot reçoit la consigne de ne pas modifier Safety ; les tests vérifient le comportement attendu ; la PR montre le diff ; la revue humaine confirme que la frontière n'a pas été contournée.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
« C'est dans `copilot-instructions.md`, donc personne ne peut enfreindre la règle. »

VERSION MIEUX MAÎTRISÉE
Écrire la règle pour guider l'assistant, puis choisir un contrôle déterministe ou humain qui détecte l'écart réellement important.

Pièges et nuances à anticiper

- Automatiser une règle mal définie.
- Supposer que la CI couvre tout le comportement métier.
- Créer des tâches locales différentes des commandes réelles de CI.
- Ne pas expliquer qui décide après un check rouge.

Questions au groupe

- Quelle règle peut être automatisée ?
- Quelle règle nécessite une revue humaine ?
- Pourquoi garder à la fois tâche locale et CI ?

Mini-activité guidée
Distribuer dix règles et demander de cocher guidance, local, CI, revue humaine. Plusieurs réponses peuvent être cumulées, mais chaque choix doit être justifié.

Correction et débrief
Les règles importantes apparaissent souvent à plusieurs niveaux. La guidance réduit les écarts en amont, les contrôles détectent, la revue interprète et décide.

À RETENIR : La robustesse vient de couches de contrôle complémentaires, pas d'un seul mécanisme.

Approfondissement 2 - Instructions de dépôt et instructions ciblées

À DIRE TRÈS SIMPLEMENT
Les règles communes vont au niveau du dépôt ; les règles spécifiques vont près des fichiers concernés.

Explication approfondie. Un fichier global trop long devient difficile à maintenir et peut noyer les priorités. À l'inverse, répéter les mêmes règles dans chaque prompt augmente les oublis. La séparation entre instructions globales et ciblées permet d'appliquer le bon contexte au bon endroit : architecture, version et validation au niveau global ; conventions de C++ ou de tests sur les chemins concernés.

Le formateur peut faire trier les règles par portée. Une règle globale doit être vraie pour tout le dépôt. Une règle ciblée s'applique à une catégorie de fichiers. Une information spécifique au ticket reste dans le prompt. Ce tri évite de transformer les instructions en base documentaire complète.

Application au robot porte-palette. `Mission` ne doit pas appeler les drivers directement : règle globale d'architecture. Les tests doivent couvrir les valeurs frontières : instruction ciblée sur `tests/**`. Le motif précis demandé par ROBOT-247 appartient au ticket et au prompt.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Copier le ticket courant dans les instructions globales, puis oublier de le retirer.

VERSION MIEUX MAÎTRISÉE
Maintenir des règles stables dans le dépôt et garder les objectifs temporaires dans les demandes ponctuelles.

Pièges et nuances à anticiper

- Mettre des secrets dans les instructions versionnées.
- Écrire des règles trop spécifiques à un seul ticket.
- Multiplier les fichiers ciblés avec des contradictions.
- Oublier de relire l'instruction lorsque l'architecture évolue.

Questions au groupe

- Cette règle est-elle stable pour tout le dépôt ?
- Est-elle spécifique à un type de fichier ?
- Doit-elle plutôt rester dans le ticket ?

Mini-activité guidée
Fournir quinze règles et trois emplacements : global, ciblé tests/C++, prompt. Les participants doivent justifier le placement et repérer deux règles qui devraient devenir des checks automatisés.

Correction et débrief
Le meilleur emplacement minimise la répétition tout en conservant la pertinence. Les règles critiques ne deviennent pas garanties par leur seul placement.

À RETENIR : Le contexte persistant doit être stable, partagé et proportionné à sa portée.

Approfondissement 3 - VS Code tasks : une façade reproductible vers les commandes du projet

À DIRE TRÈS SIMPLEMENT
Une tâche VS Code ne remplace pas le build ; elle donne un bouton ou un raccourci vers la vraie commande.

Explication approfondie. L'intérêt pédagogique des tasks est de rendre les contrôles faciles à exécuter avant une PR. Le risque est de confondre configuration d'éditeur et preuve : si `tasks.json` contient une mauvaise commande, son existence ne prouve rien. La tâche doit donc appeler les commandes réelles du dépôt, être versionnée lorsque cela a du sens et rester cohérente avec la CI.

On enseigne la task comme un alias documenté : label, type shell, command, group et éventuelles dépendances. Une tâche `pre-PR` peut chaîner build, test et contrôle licence. Le formateur doit montrer volontairement une commande cassée pour faire comprendre la différence entre 'configuration présente' et 'contrôle réussi'.

Application au robot porte-palette. Le développeur peut lancer `Robot: pre-PR` avant d'ouvrir la pull request. La tâche compile en C++17, exécute le binaire de test et lance le contrôle local disponible. La CI rejouera ensuite les contrôles adaptés au serveur.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Une task nommée `test` qui affiche simplement 'tests ok'.

VERSION MIEUX MAÎTRISÉE
Une task qui appelle le script ou la commande de test réelle, échoue avec un code de sortie non nul et affiche la commande exécutée.

Pièges et nuances à anticiper

- Chemins absolus propres au poste du formateur.
- Commandes Windows/Linux non portables sans explication.
- Task locale plus permissive que la CI.
- Masquer les erreurs avec `|| true`.

Questions au groupe

- Quelle commande réelle est derrière la task ?
- Comment savoir si elle a échoué ?
- Pourquoi la CI doit-elle rester indépendante de l'éditeur ?

Mini-activité guidée
Présenter trois `tasks.json`, dont un qui masque les erreurs. Faire repérer lequel donne une fausse confiance et le corriger.

Correction et débrief
La tâche correcte propage le code de sortie et appelle les mêmes scripts de base que le processus partagé. L'éditeur améliore l'ergonomie, pas la vérité du contrôle.

À RETENIR : Une task est utile quand elle rend une vraie commande reproductible et visible.

Approfondissement 4 - Politique de licences : identifier, qualifier, décider

À DIRE TRÈS SIMPLEMENT
Connaître le nom d'une licence aide à analyser une dépendance ; cela ne suffit pas à l'autoriser.

Explication approfondie. Le cours doit éviter deux raccourcis : 'open source = autorisé' et 'licence permissive = aucune obligation'. La politique de l'entreprise détermine les usages acceptables. Le rôle du développeur est de fournir les informations nécessaires : composant, version, source, licence identifiée, type d'usage et alternative éventuelle. Les cas ambigus sont escaladés au processus approprié.

SPDX fournit un vocabulaire normalisé pour nommer les licences. La matrice pédagogique peut regrouper les familles en permissif, copyleft et informations insuffisantes, mais sans prétendre rendre un avis juridique universel. Le formateur insiste sur la distinction entre identification automatique et décision de conformité.

Application au robot porte-palette. Ajouter une bibliothèque de télémétrie peut sembler sans rapport avec la logique de mouvement, mais elle entre dans la chaîne logicielle livrée. La PR doit rendre visible cette nouvelle dépendance et déclencher le contrôle prévu.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
« C'est MIT, donc on n'a rien à faire. »

VERSION MIEUX MAÎTRISÉE
« La licence déclarée est MIT. Notre politique l'autorise-t-elle pour cet usage ? Quelles notices devons-nous conserver et quelle version introduisons-nous ? »

Pièges et nuances à anticiper

- Prendre un nom de dépôt pour une preuve de licence.
- Ignorer les dépendances transitives.
- Supposer que les dépendances de test sont toujours hors politique.
- Donner un avis juridique définitif à partir d'un exercice pédagogique.

Questions au groupe

- Quelle information manque si le ticket dit seulement 'open source' ?
- Pourquoi noter la version exacte ?
- Qui décide en cas de licence non couverte par la politique ?

Mini-activité guidée
Classer cinq dépendances fictives avec trois états : potentiellement acceptable, validation explicite requise, informations insuffisantes. Interdire les réponses sans justification.

Correction et débrief
La qualité de la réponse se juge à la démarche : identifier, décrire l'usage, consulter la politique, escalader si nécessaire. L'exercice ne remplace pas la validation réelle de l'entreprise.

À RETENIR : Identifier une licence est une entrée du processus de décision, pas l'autorisation finale.

Approfondissement 5 - Dependency Review : comprendre ce qui change dans la PR

À DIRE TRÈS SIMPLEMENT
La Dependency Review cherche ce qu'une pull request ajoute ou modifie dans les dépendances et applique des règles définies par l'équipe.

Explication approfondie. Le contrôle est puissant parce qu'il se place au moment de la proposition de changement. Il peut détecter des vulnérabilités ou licences incompatibles selon la configuration. Mais il ne remplace ni l'architecture ni le jugement juridique ni les tests métier. Il répond à une question précise : quelles dépendances changent et respectent-elles les règles automatisées choisies ?

Le workflow est lu ligne par ligne : déclencheur `pull_request`, permissions nécessaires, checkout éventuel, action de dependency review, paramètres de sévérité et politique de licences. La formation doit aussi montrer le chemin après un échec : lire le message, identifier la dépendance, vérifier le contexte, puis décider de retirer, remplacer ou faire valider.

Application au robot porte-palette. ROBOT-247 n'a pas besoin d'une nouvelle dépendance. Si une PR en introduit une malgré la consigne, la review doit rendre ce changement visible. Le reviewer vérifie alors si l'ajout est justifié et conforme.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Traiter un check vert comme preuve que la dépendance est techniquement nécessaire et juridiquement parfaite.

VERSION MIEUX MAÎTRISÉE
Utiliser le check comme preuve qu'un ensemble de règles automatisées a été satisfait, puis conserver la revue d'architecture et la décision humaine pour le reste.

Pièges et nuances à anticiper

- Ne pas comprendre la portée exacte du workflow.
- Configurer un seuil sans expliquer sa signification.
- Ignorer un check rouge en le rendant optionnel.
- Croire que le workflow teste le comportement C++.

Questions au groupe

- Que compare la Dependency Review ?
- Que ne prouve pas un check vert ?
- Quelle action suivre après un échec de licence ?

Mini-activité guidée
Donner un workflow court et demander au groupe de surligner déclencheur, action, règle de vulnérabilité, règle de licence et point de décision humaine.

Correction et débrief
Le participant doit être capable d'expliquer chaque ligne avec des mots simples et de décrire le flux d'un check rouge jusqu'à la décision d'intégration.

À RETENIR : La Dependency Review automatise un contrôle de dépendances dans la PR ; elle ne remplace pas les autres formes de validation.

Approfondissement 6 - Local et CI : lutter contre la divergence

À DIRE TRÈS SIMPLEMENT
Ce qui passe sur le poste doit être rejoué dans un environnement partagé, mais les deux chemins doivent rester cohérents.

Explication approfondie. La duplication des commandes entre local et CI peut dériver : versions différentes, options de compilation, scripts distincts. Un développeur croit alors avoir validé un changement qui échoue en PR. Le meilleur moyen de réduire ce risque est de partager les scripts ou commandes de base et de garder la task VS Code comme façade ergonomique.

Faire dessiner la chaîne : source -> script/build -> task locale -> workflow CI. Les deux derniers doivent converger vers les mêmes briques. Les différences nécessaires, par exemple environnement ou secrets, sont documentées. Le groupe apprend ainsi à chercher la source de vérité technique plutôt qu'à recopier des commandes.

Application au robot porte-palette. Si local compile avec C++20 mais la CI avec C++17, Copilot peut proposer une syntaxe acceptée localement et refusée en PR. La version cible doit être déclarée dans les instructions et imposée par la commande de build.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Maintenir une commande dans `tasks.json` et une autre dans le YAML sans script commun ni vérification de version.

VERSION MIEUX MAÎTRISÉE
Versionner des commandes ou scripts partagés ; la task et la CI les invoquent avec des paramètres contrôlés.

Pièges et nuances à anticiper

- Dépendre d'un outil installé uniquement sur le poste du formateur.
- Masquer un échec local que la CI considère bloquant.
- Utiliser des versions de compilateur non alignées.
- Ne pas documenter les différences nécessaires.

Questions au groupe

- Quelle est la source de vérité du build ?
- Comment détecter une divergence local/CI ?
- Que doit-on afficher dans la PR pour faciliter le diagnostic ?

Mini-activité guidée
Présenter un cas où local passe et CI échoue. Faire comparer les commandes, versions et variables. Les participants doivent isoler l'écart minimal.

Correction et débrief
La solution n'est pas de désactiver la CI mais de réconcilier les chemins. La configuration locale doit aider à anticiper le contrôle partagé.

À RETENIR : Le local donne un feedback rapide ; la CI fournit une vérification partagée et reproductible.

Approfondissement 7 - Pull request : transformer le changement en dossier de preuve

À DIRE TRÈS SIMPLEMENT
Une bonne PR ne dit pas seulement ce qui a changé ; elle explique pourquoi, comment cela a été vérifié et ce qui reste à décider.

Explication approfondie. La PR est le point de rencontre entre ticket, code, tests, dépendances et revue. Dans un flux assisté par IA, elle doit éviter l'effet 'Copilot a fait le travail'. Le développeur reste auteur de la proposition intégrée. La description doit donc résumer le besoin, les interfaces impactées, les preuves locales exécutées, les dépendances ajoutées et les incertitudes restantes.

Un modèle simple : contexte, changement, impacts, tests/commandes, dépendances, risques, revue. Le reviewer peut alors reproduire les contrôles et concentrer son attention sur les zones à risque. La CI complète ce dossier par des résultats automatisés.

Application au robot porte-palette. ROBOT-247 modifie le contrat de `canStartMotion`. La PR liste les consommateurs, montre les nouveaux motifs, indique la frontière batterie testée et précise qu'aucune dépendance externe n'a été ajoutée.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Description : « Fix ROBOT-247 with Copilot ». Aucun test ni impact n'est mentionné.

VERSION MIEUX MAÎTRISÉE
Description structurée avec ticket, contrat modifié, consommateurs, commandes exécutées, résultats, dépendances et points de revue.

Pièges et nuances à anticiper

- Coller une longue conversation IA dans la PR au lieu d'une synthèse.
- Dire 'tests passed' sans commande.
- Oublier de signaler un test non exécuté.
- Masquer un changement d'interface dans un refactoring massif.

Questions au groupe

- Quelles preuves le reviewer doit-il pouvoir reproduire ?
- Quel impact de contrat doit être visible ?
- Que faut-il écrire si un contrôle n'a pas pu être lancé ?

Mini-activité guidée
Donner une PR minimale et demander aux participants de l'enrichir avec six rubriques. Puis faire une revue croisée entre binômes.

Correction et débrief
La PR réussie réduit le coût de revue : elle oriente vers les risques et permet de vérifier les faits. Elle ne doit pas cacher les limites ou incertitudes.

À RETENIR : La PR est un dossier de changement et de preuves, pas un simple bouton de merge.

Approfondissement 8 - Atelier final : séparer chaque décision humaine des automatismes

À DIRE TRÈS SIMPLEMENT
À chaque étape du ticket, demandez : qui propose, qui vérifie et qui décide ?

Explication approfondie. La simulation finale a pour but d'intégrer les trois jours. Un participant peut connaître chaque outil séparément et pourtant ne pas savoir organiser le flux. Le formateur doit donc faire verbaliser la chaîne complète : lecture du ticket, instructions, prompt, proposition, revue du diff, tests locaux, dépendances, PR, CI, revue humaine et merge.

Une technique efficace consiste à tenir trois colonnes au tableau : IA, outil déterministe, humain. Pour chaque action de l'atelier, les participants placent la responsabilité. L'IA propose une interface et des tests ; le compilateur et les tests

produisent des résultats ; la Dependency Review applique une politique automatisée ; l'humain interprète, demande des corrections et autorise ou refuse selon le processus.

Application au robot porte-palette. ROBOT-247 devient le scénario de synthèse. Le participant ne doit pas seulement produire `MotionResult` ; il doit expliquer pourquoi le contrat est meilleur, quels consommateurs changent, quelles commandes ont été exécutées et pourquoi la PR peut ou non être fusionnée.

Pour animer cette notion, il est utile de faire raisonner le groupe en chaîne : quelle est la donnée de départ, quelle décision est prise, sur quelle hypothèse cette décision repose, et quelle preuve permet ensuite de l'accepter. Cette manière de questionner évite de juger une proposition uniquement sur son apparence. Dans un contexte industriel, une réponse bien formulée n'a de valeur que si elle peut être reliée à un besoin, à des contraintes explicites et à des contrôles observables.

VERSION FRAGILE
Évaluer uniquement le fait que le programme affiche 'tests passed'.

VERSION MIEUX MAÎTRISÉE
Évaluer la qualité du raisonnement et la traçabilité du processus en plus du résultat technique.

Pièges et nuances à anticiper

- Aider trop vite pendant l'atelier et masquer les lacunes.
- Noter seulement la syntaxe C++.
- Accepter une dépendance inutile parce que le code fonctionne.
- Laisser l'IA rédiger la conclusion de conformité.

Questions au groupe

- Quelle décision reste humaine ici ?
- Quelle preuve est produite par la machine ?
- Quelle information du ticket n'a pas été transformée en test ?

Mini-activité guidée
À trois moments de l'atelier, interrompre le groupe et demander de remplir la matrice proposition/vérification/décision. Cela rend visibles les délégations implicites.

Correction et débrief
Le participant réussit lorsqu'il peut expliquer chaque étape et ses limites. Une solution techniquement correcte mais non traçable doit être considérée incomplète pour l'objectif du cours.

À RETENIR : La compétence finale est un processus maîtrisé, pas l'utilisation isolée d'un assistant.

Études de cas supplémentaires

Ces études de cas servent à faire transférer les règles vues dans le cours. Chaque cas est conçu pour obliger le groupe à justifier une décision plutôt qu'à réciter une définition.

Cas 1 - Task locale qui masque l’échec

La tâche VS Code exécute `./tests || true`, puis affiche « pre-PR complete ».

Questions d’analyse

- Pourquoi l’ergonomie est-elle trompeuse ?
- Quel signal doit remonter ?
- Que se passe-t-il en CI ?

Correction argumentée
Le `|| true` transforme un échec en succès. La tâche doit propager le code de sortie et arrêter la chaîne. Un contrôle local utile ne doit pas masquer ce que la CI considérera comme bloquant.

Cas 2 - Dépendance ajoutée pour trois lignes de code

Copilot propose une bibliothèque externe pour formater un petit message.

Questions d’analyse

- Quelle question d’architecture poser ?
- Quelle règle de dépôt s’applique ?
- Quel contrôle PR doit détecter l’ajout ?

Correction argumentée
L’ajout doit être justifié et validé ; une alternative sans dépendance peut être préférable. La Dependency Review rend l’ajout visible mais la décision d’architecture reste humaine.

Cas 3 - Licence indiquée par le README seulement

Une dépendance affirme « open source » dans son README mais aucun identifiant ni fichier de licence n’est trouvé.

Questions d’analyse

- Peut-on l’autoriser ?
- Quelle information demander ?
- Quel état de la matrice choisir ?

Correction argumentée
Les informations sont insuffisantes. Ne pas intégrer automatiquement ; identifier la licence depuis une source fiable et appliquer la politique réelle.

Cas 4 - Copilot respecte le prompt mais contourne une frontière

Le code généré passe les tests locaux mais appelle directement un driver depuis Mission.

Questions d’analyse

- Pourquoi les tests peuvent-ils passer ?
- Quelle instruction aurait dû guider ?
- Quel contrôle humain reste nécessaire ?

Correction argumentée
Un test comportemental peut ne pas détecter une violation d’architecture. Les instructions peuvent guider, mais la revue du diff et les règles d’architecture restent nécessaires.

Cas 5 - CI verte mais test manquant

Le workflow compile et exécute les tests existants. Aucun test ne couvre `battery=20`.

Questions d’analyse

- Que prouve la CI ?
- Que ne prouve-t-elle pas ?
- Comment améliorer la PR ?

Correction argumentée
La CI prouve que les contrôles configurés réussissent. Elle ne prouve pas que la couverture des exigences est complète. Le reviewer doit comparer les tests aux critères d’acceptation.

Cas 6 - Check rouge de vulnérabilité

Dependency Review échoue sur une nouvelle dépendance alors que le ticket ne la demandait pas.

Questions d’analyse

- Quelles options ?
- Peut-on simplement baisser le seuil ?
- Qui décide ?

Correction argumentée
On identifie la dépendance et sa nécessité. Si elle est inutile, on la retire. Modifier la politique pour faire passer la PR sans analyse serait un anti-pattern. Toute exception suit le processus de l’organisation.

Ateliers longs et corrections

Atelier 1 - Construire un parcours pre-PR cohérent

Contexte
Le dépôt possède un build manuel, des tests et une politique de dépendances.

Consigne
Créer un plan de tâches VS Code et un workflow de PR qui réutilisent les mêmes commandes de base, puis expliquer ce qui reste uniquement humain.

Déroulement formateur

- Écrire d’abord les commandes sources hors VS Code.
- Créer les tasks comme façade.
- Dessiner le workflow CI qui réutilise ces commandes.
- Ajouter la Dependency Review comme contrôle distinct.
- Lister les décisions non automatisables dans l’exercice.

Correction détaillée
La correction montre que les commandes de build/test sont communes. La task locale donne un feedback rapide ; la CI rejoue dans un contexte partagé. L’architecture et les exceptions de politique restent soumises à revue.

Critères de réussite

- Aucune task ne masque les erreurs.
- La version C++ cible est explicite.
- La CI ne dépend pas de l’éditeur.
- Les contrôles de dépendance sont séparés des tests métier.

Atelier 2 - Simulation ROBOT-247 de bout en bout

Contexte
Le ticket demande un motif de refus explicite sans modifier Safety ni ajouter de dépendance.

Consigne
Produire prompt, diff, tests, résultats locaux, description de PR, lecture de CI et décision finale.

Déroulement formateur

- Faire lire d’abord les instructions du dépôt.
- Faire lister les consommateurs avant modification.
- Exiger le cas frontière batterie.
- Interdire une dépendance externe.
- Faire rédiger la PR puis une revue IA structurée.
- Terminer par une décision humaine argumentée.

Correction détaillée
Le dossier final doit relier chaque exigence à une preuve. La revue IA peut signaler des points, mais le participant doit confirmer les faits. Le merge n’est possible que si les contrôles obligatoires et la revue attendue sont satisfaits.

Critères de réussite

- Contrat explicite.
- Tests nominaux, négatifs et frontière.
- Commandes réellement exécutées documentées.
- Aucune dépendance non validée.
- Décision humaine justifiée.

Tableaux de référence complémentaires

Matrice règle -> mécanisme de contrôle

Règle	Guidance Copilot	Local	CI	Humain
C++17	Oui	Build	Build	Surveille exceptions
Tests requis	Rappel	Task test	Check test	Évalue couverture
Pas de nouvelle dépendance non validée	Oui	Contrôle possible	Dependency Review	Décide pertinence / exception
Ne pas contourner Safety	Oui	Tests partiels	Tests partiels	Revue architecture
Signaler incertitudes	Oui	-	-	Vérifie honnêteté de la PR

Checklist de PR contrôlée

Rubrique	Contenu attendu
Ticket / objectif	Pourquoi le changement existe
Contrat	Interfaces, types, erreurs, unités ou timeouts modifiés
Impacts	Consommateurs et fichiers concernés
Preuves locales	Commandes exécutées + résultat
Dépendances	Ajouts, versions, licence/politique
CI	Checks attendus et statut
Revue	Points à vérifier humainement

Démonstrations et code complémentaires

Exemple `.vscode/tasks.json` pédagogique

Les tâches appellent des commandes réelles et la tâche `pre-PR` les enchaîne. Les commandes doivent être adaptées au dépôt réel.

```
{
  "version": "2.0.0",
  "tasks": [
    {"label":"robot: build", "type":"shell", "command":"g++ -std=c++17 -Wall -Wextra -pedantic -Iinclude main.cpp -o robo
    {"label":"robot: test", "type":"shell", "command":"./robot_demo", "dependsOn":["robot: build"]},
    {"label":"robot: pre-PR", "dependsOrder":"sequence", "dependsOn":["robot: build","robot: test"]}
  ]
}
```

Points à commenter

- La task est une façade : la commande de build reste visible.
- Un échec doit remonter par un code de sortie non nul.
- Les chemins et commandes doivent être adaptés au système réel.
- La CI doit rester capable d’exécuter les contrôles sans VS Code.

Exemple de workflow Dependency Review - lecture pédagogique

Exemple volontairement court destiné à la lecture en cours. La politique réelle de l’entreprise prime.

```
name: Dependency Review
on: [pull_request]
permissions:
  contents: read

jobs:
  dependency-review:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/dependency-review-action@v4
        with:
          fail-on-severity: high
          allow-licenses: MIT, Apache-2.0, BSD-3-Clause
```

Points à commenter

- Le déclencheur est la pull request.
- Le check applique une politique automatisée à la modification de dépendances.
- La liste de licences est un exemple pédagogique et non une politique universelle.
- Un check vert ne remplace pas la décision d’architecture ni l’analyse juridique des cas ambigus.

Banque de questions de vérification

1. Quelle règle peut être automatisée ?

Réponse attendue : La robustesse vient de couches de contrôle complémentaires, pas d'un seul mécanisme. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

2. Quelle règle nécessite une revue humaine ?

Réponse attendue : La robustesse vient de couches de contrôle complémentaires, pas d'un seul mécanisme. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

3. Cette règle est-elle stable pour tout le dépôt ?

Réponse attendue : Le contexte persistant doit être stable, partagé et proportionné à sa portée. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

4. Est-elle spécifique à un type de fichier ?

Réponse attendue : Le contexte persistant doit être stable, partagé et proportionné à sa portée. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

5. Quelle commande réelle est derrière la task ?

Réponse attendue : Une task est utile quand elle rend une vraie commande reproductible et visible. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

6. Comment savoir si elle a échoué ?

Réponse attendue : Une task est utile quand elle rend une vraie commande reproductible et visible. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

7. Quelle information manque si le ticket dit seulement 'open source' ?

Réponse attendue : Identifier une licence est une entrée du processus de décision, pas l'autorisation finale. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

8. Pourquoi noter la version exacte ?

Réponse attendue : Identifier une licence est une entrée du processus de décision, pas l'autorisation finale. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

9. Que compare la Dependency Review ?

Réponse attendue : La Dependency Review automatise un contrôle de dépendances dans la PR ; elle ne remplace pas les autres formes de validation. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

10. Que ne prouve pas un check vert ?

Réponse attendue : La Dependency Review automatise un contrôle de dépendances dans la PR ; elle ne remplace pas les autres formes de validation. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

11. Quelle est la source de vérité du build ?

Réponse attendue : Le local donne un feedback rapide ; la CI fournit une vérification partagée et reproductible. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

12. Comment détecter une divergence local/CI ?

Réponse attendue : Le local donne un feedback rapide ; la CI fournit une vérification partagée et reproductible. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

13. Quelles preuves le reviewer doit-il pouvoir reproduire ?

Réponse attendue : La PR est un dossier de changement et de preuves, pas un simple bouton de merge. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

14. Quel impact de contrat doit être visible ?

Réponse attendue : La PR est un dossier de changement et de preuves, pas un simple bouton de merge. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

15. Quelle décision reste humaine ici ?

Réponse attendue : La compétence finale est un processus maîtrisé, pas l'utilisation isolée d'un assistant. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

16. Quelle preuve est produite par la machine ?

Réponse attendue : La compétence finale est un processus maîtrisé, pas l'utilisation isolée d'un assistant. Le formateur doit demander une justification reliée au besoin, aux contraintes et aux preuves disponibles.

Synthèse générale de la journée - version approfondie

La compétence recherchée n'est pas de connaître une liste de règles par cœur. Le participant doit être capable de regarder une situation nouvelle et de reconstruire le raisonnement : identifier le risque, expliciter le contrat, cadrer la demande, obtenir une proposition, organiser les preuves, puis décider. Le fil rouge du robot permet de rendre ce raisonnement concret, mais la méthode doit pouvoir être transférée à d'autres systèmes C++ industriels.

- La robustesse vient de couches de contrôle complémentaires, pas d'un seul mécanisme.
- Le contexte persistant doit être stable, partagé et proportionné à sa portée.
- Une task est utile quand elle rend une vraie commande reproductible et visible.
- Identifier une licence est une entrée du processus de décision, pas l'autorisation finale.
- La Dependency Review automatise un contrôle de dépendances dans la PR ; elle ne remplace pas les autres formes de validation.
- Le local donne un feedback rapide ; la CI fournit une vérification partagée et reproductible.
- La PR est un dossier de changement et de preuves, pas un simple bouton de merge.
- La compétence finale est un processus maîtrisé, pas l'utilisation isolée d'un assistant.

Annexe approfondie 1 - Guide d’animation par séquence

Ce guide aide le formateur à transformer le support en rythme de salle. Pour chaque séquence, la logique reste la même : faire émerger le problème, introduire la notion seulement ensuite, faire produire une décision par les participants, puis terminer par une preuve ou un artefact réutilisable.

Séquence	Ouverture conseillée	Activité centrale	Trace de fin
09:00-10:15 paramétrage Copilot	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
10:15-11:30 VS Code tasks	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
11:30-12:30 licences	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
13:30-15:30 Dependency Review	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite
15:30-17:30 atelier final	Question-problème reliée au robot	Faire comparer une version fragile et une version mieux maîtrisée	Une règle, une checklist ou une preuve écrite

Conseil d’animation : ne pas donner immédiatement la réponse. Laisser d’abord le groupe formuler ce qu’il faudrait vérifier. Lorsque plusieurs réponses sont possibles, demander quel fait permettrait de trancher. Cette habitude construit le réflexe central de la formation : ne pas remplacer une incertitude par une affirmation plausible.

- Faire reformuler la règle à la fin de chaque bloc.
- Conserver au tableau les décisions qui seront réutilisées plus tard dans la journée.
- Faire expliciter la différence entre une hypothèse, un fait observé et une décision.
- Demander régulièrement : « quelle preuve avons-nous réellement ? »
- Si le groupe va trop vite vers le code, revenir au contrat et aux critères d’acceptation.

Annexe approfondie 2 - FAQ formateur et réponses de secours

Question 1. Quelle règle peut être automatisée ?

Réponse courte : La robustesse vient de couches de contrôle complémentaires, pas d'un seul mécanisme.

Réponse développée : Le J3 doit faire comprendre une progression de force. Les instructions de Copilot réduisent les propositions incohérentes, mais elles sont interprétées par un modèle. Une tâche locale exécute une commande déterministe, mais un développeur peut l'oublier. Une CI associée à la pull request rejoue les contrôles de façon partagée et peut bloquer l'intégration selon la configuration. Ces trois niveaux sont complémentaires. Le formateur peut ensuite revenir sur le fil rouge : Pour ROBOT-247, Copilot reçoit la consigne de ne pas modifier Safety ; les tests vérifient le comportement attendu ; la PR montre le diff ; la revue humaine confirme que la frontière n'a pas été contournée.

Question 2. Cette règle est-elle stable pour tout le dépôt ?

Réponse courte : Le contexte persistant doit être stable, partagé et proportionné à sa portée.

Réponse développée : Un fichier global trop long devient difficile à maintenir et peut noyer les priorités. À l'inverse, répéter les mêmes règles dans chaque prompt augmente les oublis. La séparation entre instructions globales et ciblées permet d'appliquer le bon contexte au bon endroit : architecture, version et validation au niveau global ; conventions de C++ ou de tests sur les chemins concernés. Le formateur peut ensuite revenir sur le fil rouge : `Mission` ne doit pas appeler les drivers directement : règle globale d'architecture. Les tests doivent couvrir les valeurs frontières : instruction ciblée sur `tests/**`. Le motif précis demandé par ROBOT-247 appartient au ticket et au prompt.

Question 3. Quelle commande réelle est derrière la task ?

Réponse courte : Une task est utile quand elle rend une vraie commande reproductible et visible.

Réponse développée : L'intérêt pédagogique des tasks est de rendre les contrôles faciles à exécuter avant une PR. Le risque est de confondre configuration d'éditeur et preuve : si `tasks.json` contient une mauvaise commande, son existence ne prouve rien. La tâche doit donc appeler les commandes réelles du dépôt, être versionnée lorsque cela a du sens et rester cohérente avec la CI. Le formateur peut ensuite revenir sur le fil rouge : Le développeur peut lancer `Robot: pre-PR` avant d'ouvrir la pull request. La tâche compile en C++17, exécute le binaire de test et lance le contrôle local disponible. La CI rejouera ensuite les contrôles adaptés au serveur.

Question 4. Quelle information manque si le ticket dit seulement 'open source' ?

Réponse courte : Identifier une licence est une entrée du processus de décision, pas l'autorisation finale.

Réponse développée : Le cours doit éviter deux raccourcis : 'open source = autorisé' et 'licence permissive = aucune obligation'. La politique de l'entreprise détermine les usages acceptables. Le rôle du développeur est de fournir les informations nécessaires : composant, version, source, licence identifiée, type d'usage et alternative éventuelle. Les cas ambigus sont escaladés au processus approprié. Le formateur peut ensuite revenir sur le fil rouge : Ajouter une bibliothèque de télémétrie peut sembler sans rapport avec la logique de mouvement, mais elle entre dans la chaîne logicielle livrée. La PR doit rendre visible cette nouvelle dépendance et déclencher le contrôle prévu.

Question 5. Que compare la Dependency Review ?

Réponse courte : La Dependency Review automatise un contrôle de dépendances dans la PR ; elle ne remplace pas les autres formes de validation.

Réponse développée : Le contrôle est puissant parce qu'il se place au moment de la proposition de changement. Il peut détecter des vulnérabilités ou licences incompatibles selon la configuration. Mais il ne remplace ni l'architecture ni le jugement juridique ni les tests métier. Il répond à une question précise : quelles dépendances changent et respectent-elles les règles automatisées choisies ? Le formateur peut ensuite revenir sur le fil rouge : ROBOT-247 n'a pas besoin d'une nouvelle dépendance. Si une PR en introduit une malgré la consigne, la review doit rendre ce changement visible. Le reviewer vérifie alors si l'ajout est justifié et conforme.

Question 6. Quelle est la source de vérité du build ?

Réponse courte : Le local donne un feedback rapide ; la CI fournit une vérification partagée et reproductible.

Réponse développée : La duplication des commandes entre local et CI peut dériver : versions différentes, options de compilation, scripts distincts. Un développeur croit alors avoir validé un changement qui échoue en PR. Le meilleur moyen de réduire ce risque est de partager les scripts ou commandes de base et de garder la task VS Code comme façade ergonomique. Le formateur peut ensuite revenir sur le fil rouge : Si local compile avec C++20 mais la CI avec C++17, Copilot peut proposer une syntaxe acceptée localement et refusée en PR. La version cible doit être déclarée dans les instructions et imposée par la commande de build.

Si une question dépasse le périmètre de la journée, ne pas improviser une architecture ou une règle future. La bonne réponse pédagogique consiste à reconnaître le périmètre, reformuler ce qui est déjà établi et indiquer clairement ce qui doit être confirmé par la politique, l'architecture ou les spécialistes de l'entreprise.

Annexe approfondie 3 - Grille de diagnostic d’une production participant

Dimension	Question de contrôle	Signal satisfaisant	Signal d’alerte
Besoin	Le participant sait-il expliquer le problème avant la solution ?	Objectif et contraintes reformulés	Code produit sans besoin explicite
Hypothèses	Les inconnues sont-elles signalées ?	Questions ouvertes visibles	Valeurs inventées ou implicites
Contrat	Entrées, sorties et erreurs sont-elles compréhensibles ?	Comportement testable	Booléens / nombres ambigus
Preuves	Les tests ou contrôles sont-ils reliés aux exigences ?	Frontières et cas négatifs couverts	Seulement un cas nominal
IA	Le rôle de l’assistant est-il correctement limité ?	Proposition / aide à la revue	Validation déléguée à l’outil
Décision	La conclusion est-elle argumentée ?	Acceptation, modification ou refus justifié	« ça marche » sans critères
Traçabilité	Peut-on retrouver les éléments importants ?	Ticket, tests, revue, décision reliés	Connaissance uniquement dans le chat

Cette grille n’est pas un barème obligatoire. Elle sert à diagnostiquer rapidement où se situe la difficulté : technique, contrat, preuve ou raisonnement. Elle permet aussi d’éviter de sur-noter une solution qui compile mais dont le processus reste fragile.

Annexe approfondie 4 - Variantes selon le niveau du groupe

Situation du groupe	Adaptation	Ce qu'il ne faut pas supprimer
Groupe peu expérimenté	Réduire la taille du code, garder un seul changement à la fois, verbaliser chaque terme	La distinction proposition / preuve / décision
Groupe expérimenté	Ajouter des ambiguïtés de contrat et demander plusieurs solutions	La justification des choix et les preuves
Groupe très orienté outils	Commencer par le problème avant d'ouvrir Copilot ou VS Code	Le pourquoi métier et le rôle humain
Pas d'accès réseau	Utiliser des sorties IA préparées à l'avance et simuler la revue	L'analyse critique et les contrôles déterministes
Temps réduit	Supprimer un cas secondaire, pas les étapes de raisonnement	Une activité pratique + correction + synthèse
Groupe hétérogène	Binômes mélangeant niveaux, rôles alternés auteur/reviewer	La participation de chacun et la verbalisation

Une adaptation de niveau ne doit pas transformer la formation en démonstration passive. Même avec des débutants, les participants doivent prendre des décisions simples et expliquer leur raisonnement. Avec des experts, on augmente surtout l'ambiguïté des cas et la profondeur de justification plutôt que d'ajouter des outils hors programme.

- Version courte : une seule erreur injectée à la fois.
- Version avancée : deux contraintes contradictoires à détecter.
- Version atelier : un binôme produit, un autre révise avec la checklist.
- Version paperboard : aucune machine pendant dix minutes, uniquement contrat et tests attendus.

Annexe approfondie 5 - Plan B et scénarios de secours pour le formateur

Une formation centrée sur des outils peut être fragilisée par un problème de compte, de réseau, de licence, d'extension ou de configuration. Le contenu pédagogique doit rester exécutable même si l'assistant ou la plateforme n'est pas disponible. Les scénarios ci-dessous permettent de continuer sans changer les objectifs de la journée.

Incident	Plan B pédagogique	Objectif conservé
Copilot indisponible	Projeter deux réponses IA préparées, dont une volontairement imparfaite	Évaluer et revoir une proposition
GitHub indisponible	Utiliser un diff local et une fiche PR papier	Traçabilité, impacts et décision
CI indisponible	Lire un log de workflow préparé avec un check vert et un check rouge	Interpréter un contrôle automatisé
Compilateur indisponible	Faire prédire les résultats et analyser les tests sur papier	Relier règles, code et preuves
Temps de build trop long	Utiliser la mini-démo autonome du support	Conserver la logique de contrôle
Question hors politique interne	Revenir au principe générique et signaler la validation nécessaire	Ne pas inventer une règle d'entreprise

Préparer ces éléments avant le cours évite de transformer un incident d'outillage en perte pédagogique. Le fil conducteur doit rester le même : une proposition est analysée, les contraintes sont explicitées, des preuves sont recherchées et la décision finale reste attribuée au bon rôle.

CHECKLIST AVANT DE QUITTER LA SALLE
Le groupe peut expliquer au moins trois règles de la journée avec ses propres mots ; il peut citer une erreur typique ; il sait donner un exemple de preuve ; il sait dire ce qui reste une décision humaine ; et il a produit au moins un artefact réutilisable dans la suite de la formation.